

Note — Calcolo Numerico

`federico.poloni@unipi.it`

Versione del 5 febbraio 2026

Queste note sono nate per fare da ‘scaletta’ per me mentre tengo le lezioni; sono anche a vostra disposizione per essere usate come dispense per lo studio.

Scelta degli argomenti, dimostrazioni ed esempi sono in parte presi dalle dispense di Luca Gemignani, Paolo Ghelardoni, Cecilia Magherini. Dei Large Language Model (GPT 4.1 e GPT 5) sono stati utilizzati per scrivere il codice Latex che genera alcune delle immagini.

Queste note vengono generate in versioni diverse (con più o meno argomenti) a seconda del corso di laurea a cui sono destinate.

Quella che state leggendo è la versione per il corso di laurea in **informatica**.

Capitolo 1

Motivazione

In questo corso, vedremo algoritmi per risolvere al computer alcuni problemi che avete visto nei corsi di analisi e algebra lineare. Come mai questi algoritmi richiedono un trattamento speciale?

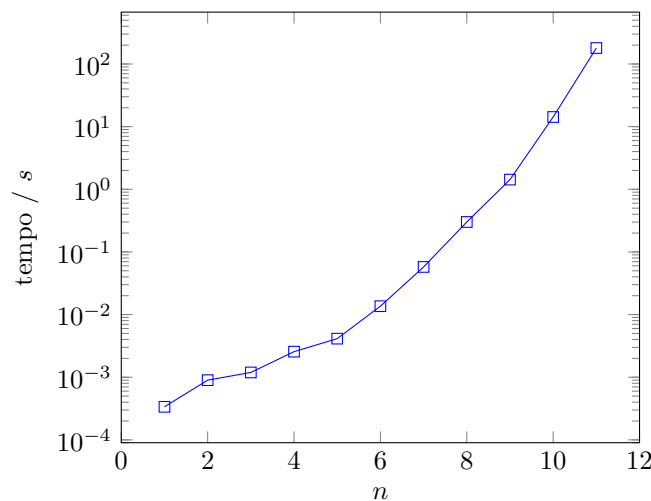
Problemi senza soluzione esatta Alcuni problemi non hanno una soluzione esatta data da una formula, per esempio $\int_0^1 e^{-x^2} dx$, ma si riescono a calcolare approssimazioni di queste soluzioni: per esempio, un integrale si può approssimare tramite somme di aree di rettangolini.

Algoritmi lenti Non sempre gli algoritmi che si vedono in un corso ‘teorico’ funzionano bene su un computer. Ad esempio, il calcolo del determinante di una matrice $A \in \mathbb{R}^{n \times n}$ tramite la formula di Laplace lungo la prima riga:

$$\det A = \sum_{j=1}^n (-1)^{j+1} a_{1j} A^{(1,j)}$$

Qui $A^{(1,j)}$ il determinante della sottomatrice $(n-1) \times (n-1)$ che si ottiene eliminando da A la riga 1 e la colonna j . Per calcolare il determinante di una matrice 11×11 con questa formula bisogna calcolare 11 determinanti di matrici 10×10 , che a loro volta richiedono ognuno il calcolo di 10 determinanti di matrici 9×9 , e così via.

A meno che non ci siano proprietà particolari nella matrice A , come entrate uguali a zero, il calcolo di un determinante con questa formula è molto lento, non solo per un umano ma anche per un computer. Esempio su un computer moderno::



Per calcolare il determinante di una matrice 11×11 ci vogliono più di 100 secondi! Un algoritmo così lento è poco utile.

Algoritmi inaccurati Un altro fattore è *come* il computer implementa certe operazioni. Numeri come $1/3$ o $\sqrt{2}$ hanno infinite cifre; non possiamo fare le operazioni “in colonna”. Ci sono diverse soluzioni possibili, ma per la maggior parte delle applicazioni l’esperienza ha insegnato che la strategia migliore è tenere (essenzialmente) solo un certo numero di cifre significative, come si fa su una calcolatrice. Questo significa che le operazioni sono approssimate: per esempio $\frac{1}{3} + \frac{1}{3} + \frac{1}{3} \approx 0.33333 + 0.33333 + 0.33333 = 0.99999 \neq 1$. Gli algoritmi vanno implementati con attenzione per evitare che queste approssimazioni un grosso impatto sul risultato. Per esempio: l’eliminazione di Gauss, o anche formule semplici come quella per risolvere un’equazione di secondo grado.

Importanza Risolvere problemi computazionali al computer è diventato sempre più importante negli ultimi anni. Pensate per esempio a chi deve progettare un aereo. Quanto spesso deve essere la parete di metallo da usare nelle ali? Se è troppo spessa, il nostro aereo consuma più carburante del necessario. Se è troppo sottile, l’ala potrebbe rompersi. Una volta c’era un solo modo di scoprirlo: costruendo un prototipo di aereo. Oggi, la maggior parte di questi esperimenti possono essere rimpiazzati da simulazioni al computer. Va trovato il modo più efficiente di farle (un aereo è un oggetto molto complesso), ed è fondamentale capire quando gli errori diventano così grandi da compromettere il risultato.

Programmazione L’altro scopo importante di questo corso è come *programmare* un computer, cioè scrivere delle istruzioni per far eseguire sequenze specifiche di operazioni: non vogliamo calcolare noi le somme di Riemann di $\int_0^1 e^{x^2} dx$, o il determinante di una matrice 11×11 , ma vogliamo imparare a fare in modo che il computer esegua questo calcolo per noi. Non vogliamo affidarci a un’app o un sito già fatto per ogni singolo problema, ma imparare come *programmare* il computer in modo da saper risolvere da soli qualunque nuovo problema ci capiti davanti.

Capitolo 2

Equazioni non lineari (zeri di funzione)

Un esempio pratico Consideriamo l'equazione

$$T = \frac{1}{\mu - \lambda 2^{-T}},$$

dove $\mu, \lambda > 0$ sono costanti, e T è un'incognita che dobbiamo determinare. Questa equazione compare nella cosiddetta *teoria delle code*, che si occupa di studiare sistemi che coinvolgono dei buffer. Per esempio, consideriamo un server web: ogni tanto arrivano utenti che richiedono di caricare una pagina. Il server lavora con un processo solo, cioè, risponde a una sola richiesta alla volta; se una seconda richiesta arriva quando il server sta ancora rispondendo alla prima, il client deve mettersi in attesa. Questa equazione serve per calcolare il tempo medio di attesa T . Il termine μ dice (raccontato molto informalmente) quanto il server è veloce a gestire richieste; e il termine $\lambda 2^{-T}$ dice quanto spesso arrivano richieste dai client. Questo termine dipende a sua volta dalla media T , nella nostra equazione: più un server è lento, più gli utenti si stufano e tendono ad abbandonarlo.

Per trovare il tempo medio di attesa T , dobbiamo risolvere un'equazione; questa non è un'equazione né di primo né di secondo grado, ma una più complicata, che non ammette una formula risolutiva. Nonostante questo, vogliamo comunque determinare un'approssimazione della soluzione. Possiamo riscrivere questo problema in termini di una funzione: la soluzione è il punto in cui

$$f(x) = \frac{1}{\mu - \lambda 2^{-x}} - x$$

tocca l'asse delle ascisse. Questa è una famiglia di problemi molto generale, che andiamo a studiare.

Il problema Data una funzione $f : [a, b] \rightarrow \mathbb{R}$, vogliamo calcolare uno *zero della funzione*, cioè un punto α tale che $f(\alpha) = 0$. Un altro esempio semplice è il calcolo di radici n -esime: calcolare $\sqrt[n]{K}$ è equivalente a trovare uno zero della funzione $f(x) = x^n - K$.

Ci potrebbero essere più soluzioni in un dato intervallo (a priori anche infinite!). Un modo conveniente di dimostrare che esiste *almeno* una soluzione in un dato intervallo $[a, b]$ è mostrare che è un *intervallo di separazione*, cioè $f(a)$ e $f(b)$ hanno segni opposti: $f(a) > 0, f(b) < 0$ oppure $f(a) < 0, f(b) > 0$. Un modo semplice di scrivere questa condizione è $f(a)f(b) < 0$. Difatti, ricordiamo questo teorema che avete visto ad analisi.

Teorema 2.1 (Teorema di esistenza degli zeri). *Sia $f \in C^0([a, b])$. Se $f(a)$ e $f(b)$ hanno segno opposto (uno positivo e uno negativo), allora esiste almeno un reale $\alpha \in (a, b)$ tale che $f(\alpha) = 0$.*

Detto in altro modo, se f è una funzione continua e $[a, b]$ è un intervallo di separazione, allora esso contiene uno zero della funzione.

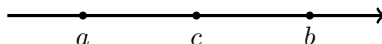
Questo risultato non ci assicura che lo zero sia unico. Possiamo concludere che è unico per esempio se riusciamo a mostrare che f è strettamente crescente o decrescente in $[a, b]$; per esempio, usando le derivate, quando la funzione è derivabile.

Esempio: siano fissati $n \geq 2, K > 0$, e consideriamo la funzione $f(x) = x^n - K$. Si ha $f(0) = -K < 0$, mentre per x sufficientemente grande si ha $f(x) > 0$ (difatti $x \rightarrow \infty$). Per esempio, se $K > 1$, abbiamo $f(K) = K^n - K > 0$. Quindi $[0, K]$ è un intervallo di separazione, ed esiste una soluzione nell'intervallo. Questa soluzione è unica? La funzione f ha derivata $f'(x) = nx^{n-1}$; quindi $f'(x) > 0$ per ogni $x > 0$, e $f'(0) = 0$. Questo permette di concludere che la soluzione è una sola.

2.1 Metodo di bisezione

È un metodo che funziona con poche ipotesi sulla f : serve solo che $f \in C^0([a, b])$, e che $[a, b]$ sia un intervallo di separazione.

Descrizione del metodo L'idea del metodo è la seguente: voglio costruire una sequenza di intervalli di separazione sempre più piccoli. Per prima cosa, calcolo il punto medio tra a e b , che chiameremo c . Se $f(c) = 0$, allora ho trovato uno zero della funzione e posso fermarmi. Altrimenti, uno dei due intervalli $[a, c]$ e $[c, b]$ è un intervallo di separazione, come si può vedere facilmente considerando i casi possibili per i segni. A questo punto, posso ripetere la procedura per questo intervallo e continuare.

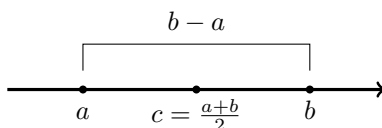


Associamo degli indici ai punti con cui lavoriamo, in modo da poter parlare più agevolmente dei punti a, b, c in passi diversi. Chiameremo $a_0 = a, b_0 = b$ i due punti iniziali, e c_0 il loro punto medio. A questo punto, dopo aver fatto il primo passo abbiamo i punti $[a_1, b_1] = [a_0, c_0]$ oppure $[c_0, b_0]$. Chiamiamo c_1 il loro punto medio, e così via.

Questa convenzione è particolarmente comoda per ragionare teoricamente sui passi dell'algoritmo: l'indice 0 si usa per i punti di partenza, da cui iniziamo l'algoritmo, e l'indice k denota i punti calcolati al k -esimo passo. (Vedremo però che il linguaggio di programmazione che usiamo, Matlab, spesso ci costringerà a partire da 1 anziché da 0.)

Due semplici formule saranno fondamentali per proseguire:

- La formula che dà il punto medio tra due numeri reali a e b è $c = \frac{a+b}{2}$;
- La formula che dà la distanza tra a e b è $b - a$ (supponendo che b sia il punto più a destra; altrimenti possiamo scrivere $|b - a|$).



```

 $a_0 = a, b_0 = b;$ 
for  $k = 0, 1, \dots$  do
     $c_k = \frac{a_k + b_k}{2};$ 
    if  $f(c_k) = 0$  then
         $\alpha = c_k;$ 
        break;
    else if  $f(a_k)f(c_k) < 0$  then
         $a_{k+1} = a_k, b_{k+1} = c_k;$ 
    else
         $a_{k+1} = c_k, b_{k+1} = b_k;$ 
end

```

Algoritmo 1: Il metodo di bisezione

Esempio Prendiamo $f(x) = x^2 - 2$. L'intervallo $[a, b] = [0, 2]$ è un intervallo di separazione, quindi contiene (almeno) uno zero α di f . Sappiamo già in realtà che ne contiene esattamente uno, $\alpha = \sqrt{2}$.

k	a_k	c_k	b_k	$f(a_k)$	$f(c_k)$	$f(b_k)$
0	0	1	2	-2	-1	2
1	1	1.5	2	-1	0.25	2
2	1	1.25	1.5	-1	-0.4375	0.25
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

```

1  % Script Matlab: metodo di bisezione
2
3  f = @(x) x^2 - 2;
4  a = 0; b = 2;
5  n = 10;
6
7  % salviamo questi valori in variabili
8  % per non doverli ricalcolare piu' volte
9  fa = f(a);
10 fb = f(b);
11
12 if fa*fb >= 0

```

```

13     error('Non ho un intervallo di separazione')
14 end
15
16 for k = 1:n
17     c = (a+b) / 2;
18     fc = f(c);
19     % per visualizzare cosa sta succedendo
20     valori = [a, c, b, fa, fc, fb]
21
22     if fc == 0    % caso fortunato
23         break
24     end
25     if fa*fc < 0
26         b = c;    % [a,b] <-- [a,c]
27         fb = fc;  % si puo' omettere (perche'?)
28     else
29         a = c;    % [a,b] <-- [c,b]
30         fa = fc;  % si puo' omettere (perche'?)
31     end
32 end

```

Costo computazionale Una delle domande importanti quando abbiamo un metodo numerico per calcolare qualcosa è: quanto tempo ci mette? Approssimativamente, il tempo impiegato da un computer per eseguire un calcolo dipende dal numero di operazioni coinvolte. Per esempio, calcolare la somma $1 + 2 + 3 + \dots + 10$ richiede 9 somme; calcolare $1 + 2 + 3 + \dots + 20$ richiede 19 somme, cioè circa il doppio, e quindi ci aspettiamo che ci metta il doppio del tempo.

Nel caso del metodo di bisezione, la difficoltà maggiore nel calcolare il tempo è che non sappiamo quante operazioni sono necessarie per calcolare la f . Spesso il metodo viene usato con funzioni f date da formule molto complicate, per cui anche solo calcolare la f in un punto è dispendioso. Per questo ha senso chiedersi quante volte viene calcolato il valore della funzione f in ogni passo. Nei passi del metodo sono necessarie anche poche altre operazioni, quelle per calcolare $\frac{a_k+b_k}{2}$, ma il loro costo è tipicamente trascurabile rispetto a quello del calcolo della f , quindi possiamo ignorarle.

Guardando la tabella più sopra, possiamo vedere che, a parte il passo iniziale in cui abbiamo calcolato la f tre volte, in a_0, b_0, c_0 , in ogni passo successivo abbiamo calcolato il valore in *un* solo nuovo punto, cioè c_k . Difatti, i valori di $f(a_k)$ e $f(b_k)$ sono già noti dal passo precedente, e ci è bastato ricopiarli dalla riga precedente della tabella.

Per questo diciamo che il costo per passo del metodo di bisezione è di una valutazione della funzione.

Errore analitico L'altra domanda chiave per un algoritmo numerico è: quanto vicino andiamo alla soluzione vera del problema? È semplice vedere che ad ogni

passo la lunghezza dell'intervallo si dimezza:

$$\begin{aligned}
b_1 - a_1 &= \frac{b_0 - a_0}{2} \\
b_2 - a_2 &= \frac{b_1 - a_1}{2} = \frac{b_0 - a_0}{4} \\
b_3 - a_3 &= \frac{b_2 - a_2}{2} = \frac{b_1 - a_1}{4} = \frac{b_0 - a_0}{8} \\
&\vdots = \vdots \\
b_k - a_k &= \frac{b_{k-1} - a_{k-1}}{2} = \frac{b_{k-1} - a_{k-1}}{4} = \dots = \frac{b_0 - a_0}{2^k}
\end{aligned}$$

Se ci fermiamo dopo n iterazioni del metodo, abbiamo un intervallo di ampiezza $\frac{b_0 - a_0}{2^n}$, e sappiamo che esso contiene uno zero α della funzione. A questo punto, se vogliamo restituire *un* valore reale che sia la migliore approssimazione possibile di α con le informazioni a nostra disposizione, la cosa migliore da fare è restituire il punto medio $c_n = \frac{a_n + b_n}{2}$. Difatti α è un punto di cui sappiamo solo che sta nell'intervallo (a_n, b_n) , di ampiezza $\ell = \frac{b_0 - a_0}{2^n}$; quindi la distanza tra α e il punto medio dell'intervallo è al massimo $\ell/2$. Se restituissi a_n o b_n invece nel caso peggiore l'errore è quasi uguale a ℓ .

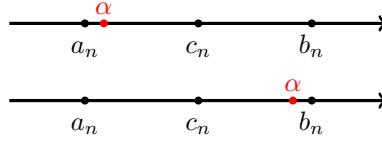


Figura 2.1: Nella figura sopra, α è molto vicino a a_n , quindi $|\alpha - b_n|$ è quasi uguale a ℓ , la lunghezza dell'intervallo $[a_n, b_n]$. Nella figura sotto, invece, α è molto vicino a b_n , quindi $|\alpha - a_n|$ è quasi uguale a ℓ . In entrambi i casi, $|\alpha - c_n| \leq \ell/2$.

Quindi, dato il valore c_n restituito dopo n passi del metodo di bisezione, siamo sicuri che esiste uno zero α di f tale che

$$|c_n - \alpha| \leq \frac{b_0 - a_0}{2^{n+1}}. \quad (2.1)$$

Criterio di arresto Possiamo trasformare il calcolo fatto sopra in un criterio per scegliere quante iterazioni fare. Fissiamo una tolleranza $\varepsilon > 0$; vogliamo arrestare il metodo quando siamo sicuri di aver determinato il valore di *uno* zero α della funzione f (non per forza l'unico!) a meno di questa tolleranza. Per avere questa sicurezza, il termine di destra della (2.1) deve essere minore di ε . Quindi prendiamo il più piccolo intero n tale che

$$\frac{b_0 - a_0}{2^{n+1}} \leq \varepsilon \iff 2^{n+1} \geq \frac{b_0 - a_0}{\varepsilon} \iff n \geq \log_2 \left(\frac{b_0 - a_0}{\varepsilon} \right) - 1.$$

Criterio di arresto euristico ()** Un'altra idea naturale è che possiamo fermarci quando $f(c_k)$ è “sufficientemente piccolo”. Abbiamo già accennato al

fatto che su un computer i calcoli non vengono effettuati esattamente, ma solo con un certo numero di cifre significative. In particolare, la condizione di arresto $f(c_k) = 0$ potrebbe non essere mai verificata esattamente, ma quando $f(c_k)$ è molto piccolo potremmo decidere di essere soddisfatti e fermare il calcolo. È però vero che se abbiamo trovato un valore c per cui $f(c)$ è piccolo, allora siamo vicini a uno zero α ? È facile costruire funzioni per cui questo non succede: basta prendere una funzione positiva che diminuisce fino ad arrivare molto vicino a zero e poi risale. Però possiamo introdurre un *criterio euristico*, cioè un criterio non rigoroso (e di cui non è garantito il successo).

Supponiamo di avere calcolato la funzione in un punto c_k , ottenendo $f(c_k)$. Una delle idee fondamentali dell'analisi è che se “zoomiamo il grafico” e ne guardiamo una sezione sufficientemente piccola, una funzione f che sia *differenziabile* si avvicina molto alla sua retta tangente. Pertanto, se f è differenziabile in c_k ,

$$f(x) \approx y = f(c_k) + f'(c_k)(x - c_k), \quad (2.2)$$

dove a destra dell'uguale abbiamo l'equazione della retta tangente in c_k . Un modo di trasformare questa relazione in un'uguaglianza esatta è tramite lo sviluppo di Taylor: f è una funzione di classe $\mathcal{C}^2$, allora per $x - c_k$ sufficientemente piccolo vale

$$f(x) = f(c_k) + f'(c_k)(x - c_k) + \frac{1}{2}f''(\xi)(x - c_k)^2,$$

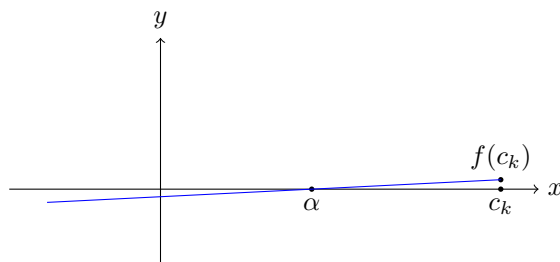
dove ξ è un punto compreso nell'intervallo tra c_k e x (in qualunque ordine essi siano). Ci aspettiamo che l'ultimo termine sia piccolo, se stiamo guardando in una sezione abbastanza piccola del grafico: se $x - c_k$ è piccolo, $(x - c_k)^2$ lo è molto di più.

Supponiamo che ci sia uno zero $x = \alpha$ vicino a c_k ; siamo in grado di valutare la distanza $|\alpha - c_k|$ usando le formule sopra? Possiamo ricavare dalla (2.2)

$$|c_k - \alpha| \approx \frac{|f(c_k)|}{|f'(c_k)|}.$$

Rovesciando il ragionamento, se $\frac{|f(c_k)|}{|f'(c_k)|} \leq \varepsilon$ ci possiamo aspettare (ma non è garantito!) che c_k disti circa ε da uno zero α della funzione. Questo procedimento non è rigoroso però: può essere che la funzione f vari più di quanto ci aspettiamo; questo succede per esempio quando $f''(\xi)$ è grande, o la funzione non è $\mathcal{C}^2$.

Notiamo il fatto interessante che non importa che f sia piccolo in assoluto in c_k , ma che sia piccolo *rispetto alla sua derivata*. Valore piccolo e derivata piccola vuol dire semplicemente che la nostra funzione è molto “piatta”, non per forza che siamo vicini a uno zero, come in figura: la funzione assume un valore $f(c_k)$ piccolo, ma c_k è lontano da α .



È utile cercare di avere un criterio di arresto che non dipenda solo dal valore della funzione, perché i valori assunti dalla funzione possono variare molto a seconda delle applicazioni: se la funzione f per esempio misurasse le distanze tra gli atomi di una molecola, la scala sull'asse delle y sarà di circa $f(x) \approx 10^{-10}$ già di suo.

Ci manca ancora un ingrediente per completare questo criterio di arresto. Il valore di $f(c_k)$ viene calcolato lungo l'algoritmo, ma $f'(c_k)$ no, e calcolarlo può essere molto più complicato. Possiamo però rimpiazzarlo con un'altra approssimazione: se $c_k \in [a_k, b_k]$, allora

$$f'(c_k) \approx \frac{f(b_k) - f(a_k)}{b_k - a_k}.$$

Difatti questo è un rapporto incrementale fatto tra due punti sufficientemente vicini a c_k (IMMAGINE). Ricordiamo anche un altro teorema di analisi, il teorema di Lagrange:

Teorema 2.2. *Se $f : [a, b] \rightarrow \mathbb{R}$ è una funzione di classe $\mathcal{C}^1$, allora esiste un punto $\xi \in (a, b)$ tale che*

$$f'(\xi) = \frac{f(b) - f(a)}{b - a}.$$

(Mettiamo “di classe $\mathcal{C}^1$ ” tra le ipotesi per semplicità, ma forse nel corso di analisi avete visto questo teorema con delle ipotesi leggermente diverse.) Questo risultato dice che la quantità che stiamo calcolando è la derivata di f in un punto ξ , diverso da c_k (e dallo ξ del teorema precedente!), ma non troppo lontano da esso.

Quindi un possibile criterio di arresto è: se vogliamo calcolare α con un errore (circa) ε , possiamo fermare il metodo quando vale la disuguaglianza

$$|f(c_k)| \leq \left| \frac{f(b_k) - f(a_k)}{b_k - a_k} \right| \varepsilon$$

(criterio di arresto sul residuo).

Vantaggi e svantaggi del metodo di bisezione Vantaggi:

- Si applica a molte funzioni: richiede solo la continuità di f .
- Funziona *sicuramente* se partiamo da un intervallo di separazione, restituendo un intervallo piccolo a piacere che contiene uno zero della funzione: come vedremo, non tutti i metodi offrono una garanzia di convergenza.

Svantaggi:

- Serve avere a disposizione un intervallo di separazione da cui iniziare.
- La convergenza è lenta rispetto ad altri metodi (vedremo più avanti come misurarla).

2.2 Metodo del punto fisso

Un altro algoritmo che ci permette di cercare zeri di funzioni è il *metodo del punto fisso*, o di *iterazione funzionale*. Per usare questo metodo, è necessario fare delle manipolazioni algebriche per riscrivere l'equazione dalla forma $f(x) = 0$ alla forma $x = \Phi(x)$, per un'opportuna funzione Φ . Per esempio, se stiamo cercando uno zero della funzione $f(x) = x^3 - 2$, possiamo riscrivere l'equazione $f(x)$ in vari modi; per esempio:

- $x = x^3 - 2 + x$;
- $x = x - \frac{1}{6}(x^3 - 2)$;
- $x = \frac{2}{x^2}$.

Ognuno di queste scritture porta a una definizione diversa di $\Phi(x)$, la funzione che sta a destra dell'uguale. In tutti questi casi, uno zero α di $f(\alpha) = 0$ è anche un *punto fisso* di Φ , cioè soddisfa $\alpha = \Phi(\alpha)$. Tutte queste possibilità portano a scelte diverse di $\Phi(x)$, e quindi a metodi che hanno (potenzialmente) comportamenti molto diversi. Alcuni di questi metodi potrebbero generare successioni che convergono a α ed altri no. Studiamo in generale questi metodi.

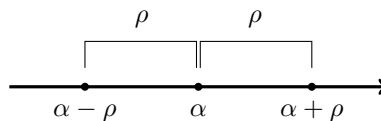
Il metodo del punto fisso funziona in questo modo: fissata una certa $\Phi : [a, b] \rightarrow \mathbb{R}$ e scelto $x_0 \in [a, b]$, costruiamo la successione

$$x_{n+1} = \Phi(x_n), \quad n = 0, 1, 2, \dots \quad (2.3)$$

Teorema del punto fisso Per dimostrare un risultato di convergenza per questo metodo, ci sono due protagonisti fondamentali che dobbiamo introdurre. Il primo è che dobbiamo lavorare su un intervallo definito in modo particolare. Fissiamo una distanza (o raggio) $\rho > 0$, e consideriamo l'insieme dei punti che hanno distanza da α minore o uguale a ρ , che possiamo scrivere in due modi:

$$I = \{x \in \mathbb{R} : |x - \alpha| \leq \rho\} = [\alpha - \rho, \alpha + \rho]. \quad (2.4)$$

La situazione è simile a quella che avevamo nel metodo di bisezione, con un segmento e il suo punto medio, solo che ora le quantità che ci interessano di più sono il centro e il raggio.



Il secondo protagonista è la quantità

$$L = \max_{x \in I} |\Phi'(x)|, \quad (2.5)$$

cioè il massimo (in valore assoluto) della derivata seconda di Φ su tutto l'intervallo. Notiamo innanzitutto che questo massimo esiste: ricordiamo il seguente teorema dell'analisi.

Teorema 2.3 (Teorema di Weierstrass / di esistenza del massimo). *Sia $[a, b]$ un intervallo chiuso e limitato, e sia f una funzione di classe $\mathcal{C}^0$ (cioè continua) su $[a, b]$. Allora, esiste (almeno) un punto di massimo nell'intervallo I .*

In questo caso, l'intervallo I è chiuso e limitato, per come l'abbiamo definito in (2.4); se Φ è una funzione di classe $\mathcal{C}^1$, allora la sua derivata Φ' esiste ed è continua, e anche la funzione $f(x) = |\Phi'(x)|$ è continua, in quanto composizione di due funzioni continue: la funzione valore assoluto, e la derivata Φ' . Quindi le ipotesi del teorema sono verificate e la quantità L esiste.

Con questi due ingredienti a disposizione, possiamo enunciare questo teorema.

Teorema 2.4 (Convergenza locale del metodo del punto fisso). *Siano dati $\alpha \in \mathbb{R}$, $\rho > 0$, e Φ una funzione tale che $\Phi(\alpha) = \alpha$. Supponiamo che Φ sia di classe $\mathcal{C}^1$ sull'intervallo $I = \{x \in \mathbb{R} : |x - \alpha| \leq \rho\}$, e supponiamo inoltre che la quantità L definita in (2.5) sia strettamente minore di 1. Allora, il metodo (2.3) è tale che per ogni $x_0 \in I$ si ha*

- $x_k \in I$ per ogni $k \geq 0$;
- $\lim_{k \rightarrow \infty} x_k = \alpha$.

Notare che il primo punto è importante perché il metodo sia ben definito: altrimenti, Φ potrebbe non essere definita nel punto x_k !

Dimostrazione. Consideriamo la quantità $|x_k - \alpha|$, cioè la distanza tra x_k e la soluzione esatta del nostro problema α . Questa quantità viene chiamata l'errore al passo k commesso dal metodo. La prima cosa che facciamo nella nostra dimostrazione è trovare una relazione tra l'errore al passo k e quello al passo $k + 1$. Si ha

$$\frac{|x_{k+1} - \alpha|}{|x_k - \alpha|} = \left| \frac{\Phi(x_k) - \Phi(\alpha)}{x_k - \alpha} \right| = |\Phi'(\xi_k)|, \quad (2.6)$$

per un certo punto ξ_k . Abbiamo usato il fatto che $x_{k+1} = \Phi(x_k)$, che $\alpha = \Phi(\alpha)$, e il teorema di Lagrange, che avete visto nel corso di analisi: esso ci dice che esiste un punto ξ_k nell'intervallo aperto di estremi x_k e α (non importa in che ordine sono) tale che $\frac{\Phi(x_k) - \Phi(\alpha)}{x_k - \alpha} = \Phi'(\xi_k)$. Chiamiamo questo punto ξ_k , indicando esplicitamente l'indice k , per sottolineare il fatto che per ogni iterazione k si ha un punto ξ_k diverso.

Ora vogliamo trasformare la (2.6) in una formula per l'errore esplicita, in cui l'errore al passo k non dipende dall'errore al passo $k - 1$. Dimosteremo per induzione che per ogni $k \geq 0$ si ha

$$|x_k - \alpha| \leq L^k \rho. \quad (2.7)$$

Ricordandoci come è fatta una dimostrazione per induzione, dobbiamo dimostrare queste due cose:

- Per $k = 0$, la (2.7) diventa $|x_0 - \alpha| \leq \rho$, che è vera perché è equivalente a dire che $x_0 \in I$.
- Supponiamo la (2.7) vera per un certo k , e dimostriamola per $k + 1$. Vogliamo usare la relazione (2.6) per dimostrare che l'errore viene ridotto di un fattore L . Per questo, il punto delicato è dimostrare che $\xi_k \in I$. Innanzitutto, per l'ipotesi induttiva abbiamo $|x_k - \alpha| < L^k \rho$; poiché $L < 1$, si ha anche $L^k \rho < \rho$ e quindi $|x_k - \alpha| < \rho$, il che vuol dire che $x_k \in I$. Poiché ξ_k è compreso tra x_k e α , anche esso appartiene all'intervallo I , e quindi abbiamo $|\Phi'(\xi_k)| \leq L$. Possiamo dunque scrivere usando la relazione (2.6)

$$|x_{k+1} - \alpha| = |\Phi'(\xi_k)| |x_k - \alpha| \leq L \cdot L^k \rho = L^{k+1} \rho. \quad (2.8)$$

Questo completa la dimostrazione per induzione della (2.7). Da questa relazione si ha che $x_k \in I$ per ogni k , come abbiamo già osservato nella dimostrazione. Inoltre

$$0 \leq |x_k - \alpha| \leq L^k \rho$$

da cui per il teorema dei carabinieri segue che $\lim_{k \rightarrow \infty} x_k = \alpha$. $\square$

Esempio 2.5. Consideriamo la funzione $\Phi(x) = x - \frac{1}{6}(x^3 - 2)$. Questa ha come punto fisso $\alpha = \sqrt[3]{2} \approx 1.26$. La sua derivata è $\Phi'(x) = 1 - \frac{1}{6}3x^2 = 1 - \frac{1}{2}x^2$. Si ha allora $|\Phi'(x)| < 1$ per $x \in (-2, 0) \cup (0, 2)$. Se prendiamo $r = 0.5$, abbiamo che su tutto l'intervallo

$$[\alpha - \rho, \alpha + \rho] \approx [1.26 - 0.5, 1.26 + 0.5]$$

si ha $|\Phi'(x)| < 1$. Possiamo quindi applicare il teorema, e concludere che il metodo converge per ogni scelta di x_0 in questo intervallo. Al posto di 0.5, possiamo scegliere qualunque ampiezza $r < 2 - \sqrt[3]{2} \approx 0.74$ e la soluzione è analoga.

Esempio 2.6. Consideriamo invece la funzione $\Phi(x) = \frac{2}{x^2}$, che ha sempre come punto fisso $x = \sqrt[3]{2}$. Questa volta si ha $\Phi'(x) = -\frac{4}{x^3}$, quindi $\Phi'(\alpha) = -2$. Non riusciremo mai a trovare un intervallo in cui applicare il teorema, perché già nel punto α che deve essere il centro dell'intervallo non vale la relazione $|\Phi'(x)| < 1$. Si può difatti vedere che il metodo di punto fisso con questa funzione non converge ad α : anche partendo da un x_0 molto vicino ad α (ma diverso da esso) le iterate si allontanano da esso.

Osservazione 2.7. Se $\Phi(x) \in \mathcal{C}^1([a, b])$ e $|\Phi'(\alpha)| < 1$ per un certo punto fisso $\alpha \in (a, b)$, allora per continuità possiamo prendere un r sufficientemente piccolo da avere $|\Phi'(x)| < 1$ per ogni $x \in I = [\alpha - \rho, \alpha + \rho]$. Quindi le ipotesi del teorema di convergenza locale 2.4 sono verificate per questo ρ . Non è semplice stabilire a priori quanto dev'essere piccolo ρ .

Esercizio 2.8. Esercizio: consideriamo l'equazione di punto fisso $x = \cos x$ (con x in radianti). Quante soluzioni positive ha? Sappiamo individuare esplicitamente un intervallo I in cui si applica il teorema del punto fisso?

Svolgimento: una soluzione corrisponde a uno zero di $f(x) = x - \cos x$. Questa funzione è tale che $f(0) = -1$, $f(\pi/2) = \pi/2$, quindi esiste almeno uno zero in $(0, \pi/2)$. La derivata di questa funzione è $f'(x) = 1 + \sin(x) \geq 0$, che si annulla solo in punti isolati, quindi la funzione è strettamente crescente. Pertanto c'è *solo* uno zero in $(0, \pi/2)$. Inoltre, non ci sono altri zeri in $[\pi/2, \infty)$ perché la funzione è strettamente positiva: su questo intervallo vale $f(x) \geq x - 1 > 0$. La derivata $\Phi'(x)$ è *strettamente* minore di 1 (in valore assoluto) al di fuori dei punti $-\pi/2, \pi/2$, e in generale tutti i punti della forma $\frac{1}{2}\pi + k\pi$, per $k \in \mathbb{Z}$. Quindi dobbiamo prendere un intervallo centrato in $\alpha \approx 0.7391$ che non contenga $-\pi/2 \approx -1.5708$ né $\pi/2 \approx 1.5708$.

Esercizio 2.9. Consideriamo il metodo del punto fisso applicato a una funzione lineare $\Phi(x) = mx + q$. Sotto quali condizioni su m e q il metodo converge?

Esercizio 2.10. Appliciamo il metodo del punto fisso all'equazione $x = \Phi(x) = \frac{1}{3} + \frac{2}{3}x^2$. Quali sono gli zeri della funzione? Per quali valori di x_0 possiamo assicurare che il metodo converge usando il teorema visto?

2.3 Convergenza lineare di un metodo iterativo

In questa sezione vediamo un modo di definire quanto velocemente converge un metodo numerico come quelli che abbiamo visto; questo è importante in modo da poter confrontare metodi diversi. Abbiamo una successione di approssimazioni x_n di una certa soluzione esatta α , ognuna calcolata da quella precedente. La successione si dice *convergente* se l'errore $e_n := |x_n - \alpha|$ tende a 0. Partiamo con un esempio che mostra cosa succede in un caso reale; in Figura 2.2 abbiamo considerato diversi metodi iterativi per risolvere l'equazione $x^3 - 2 = 0$, e abbiamo tracciato per ognuno di essi un grafico dell'errore e_n rispetto a n . La prima osservazione importante è che è utile usare una *scala logaritmica* sull'asse delle y , come nel grafico in basso: con una scala lineare, non si riesce a valutare la velocità di convergenza dei metodi se non nelle primissime iterazioni, perché poi è tutto schiacciato sullo zero.

La seconda osservazione che possiamo fare è che nel secondo grafico i punti sono approssimativamente allineati secondo una retta. Vediamo perché questo succede.

Convergenza lineare L'esempio più semplice che possiamo fare è quello in cui e_n è una successione geometrica:

$$e_0 = a, e_1 = ar, e_2 = ar^2, e_3 = ar^3, \dots$$

Ad ogni passo, l'errore precedente viene moltiplicato per r . Ovviamente dobbiamo chiedere che $r < 1$, altrimenti e_n non converge a 0. In un grafico con scala logaritmica sulle y (come quello della Figura 2.2), la successione e_k corrisponde a punti allineati lungo una retta di coefficiente angolare $\log r$ (che è negativo se $r < 1$): difatti la differenza tra le ordinate di due punti successivi è il valore costante $\log e_{n+1} - \log e_n = \log r < 0$.

Molti metodi hanno un comportamento simile a questo al limite, quindi introduciamo una definizione che formalizza questo comportamento. Diciamo che un metodo iterativo *converge linearmente* (o ha *ordine di convergenza 1*) quando

$$\lim_{n \rightarrow \infty} \frac{e_{n+1}}{e_n} = r \in (0, 1). \quad (2.9)$$

Questo vuol dire che per n sufficientemente grande (la (2.9) contiene un simbolo di limite!) la successione degli errori si comporta come una successione geometrica. Questo corrisponde al comportamento che vediamo nel grafico per alcune delle successioni nella figura 2.2: a parte i primissimi, i punti tendono ad essere allineati.

La convergenza è più veloce quanto più piccolo è r , che si chiama *tasso di convergenza*; in inglese *convergence rate*. Più informalmente, spesso si parla di *velocità di convergenza* maggiore o minore.

Attenzione: se la (2.9) vale con $r > 1$, allora vuol dire che gli errori si comportano come una successione geometrica di ragione > 1 ; quindi in particolare non vanno a zero. Se il limite nella (2.9) vale $r = 1$, può voler dire o che la successione converge a zero molto lentamente (ad esempio, $e_n = 1/n$), o che non converge proprio a zero. Questo può succedere perché abbiamo identificato erroneamente la radice α a cui si ha convergenza, come in questo esempio.

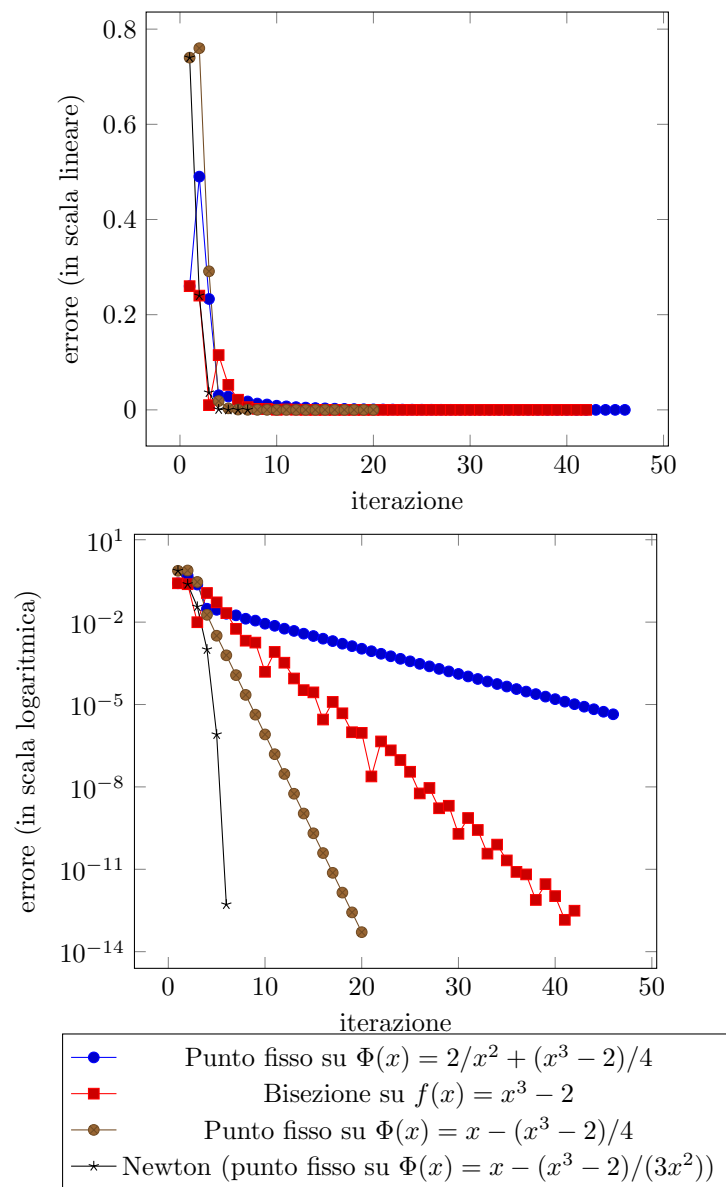


Figura 2.2: Convergenza di diversi metodi iterativi per trovare uno zero di $f(x) = x^3 - 2$.

Esempio 2.11. Per calcolare $\alpha = \sqrt{2}$, consideriamo il metodo di punto fisso $x_{n+1} = x + \frac{1}{2}(x^2 - 2)$, con $x_0 = -2$. Se calcoliamo con Matlab gli errori $e_n = |x_n - \alpha|$ prodotti dal metodo, otteniamo

1	3.4142	2.4142	2.9142	2.7892	2.8439	2.8219
2	2.8311	2.8273	2.8289	2.8282	2.8285	2.8284
3	2.8284	2.8284	2.8284	2.8284	...	

Notiamo che gli errori non stanno convergendo a zero. I valori assunti da $\frac{e_{n+1}}{e_n}$ sono

1	0.5000	1.5000	0.9167	1.0398	0.9846	1.0065
2	0.9973	1.0011	0.9995	1.0002	0.9999	1.0000
3	1.0000	1.0000	1.0000	1.0000	...	

In particolare questi numeri mostrano che $\frac{e_{n+1}}{e_n}$ converge a $r = 1$, nella pratica. Questo succede perché in realtà la successione x_k converge non a $\alpha = \sqrt{2}$, ma all'altra soluzione dell'equazione $x^2 - 2 = 0$, che è $-\sqrt{2}$.

Tasso di convergenza del metodo di punto fisso Riprendendo il calcolo fatto nell'equazione (2.8), abbiamo

$$\lim_{n \rightarrow \infty} \frac{|x_{n+1} - \alpha|}{|x_n - \alpha|} = \lim_{n \rightarrow \infty} |\Phi'(\xi_n)| = |\Phi'(\alpha)| \in [0, 1).$$

L'ultima uguaglianza vale perché abbiamo $0 \leq |\xi_n - \alpha| \leq |x_n - \alpha|$, e quindi $\xi_n \rightarrow \alpha$ per il teorema dei carabinieri. Inoltre usiamo il fatto che $x \mapsto |\Phi'(x)|$ è una funzione continua, per dire che se $\xi_n \rightarrow \alpha$ allora $|\Phi'(\xi_n)| \rightarrow |\Phi'(\alpha)|$. Questa formula ci dice che il metodo del punto fisso ha tasso di convergenza $r = |\Phi'(\alpha)|$.

Quindi quando $|\Phi'(\alpha)| < 1$ il metodo del punto fisso ha convergenza *almeno lineare*. Diciamo “almeno”, perché può succedere che $|\Phi'(\alpha)| = 0$. In questo caso, si ha quindi anche

$$\lim_{n \rightarrow \infty} \frac{e_{n+1}}{e_n} = 0.$$

Questo comportamento implica una convergenza più veloce di quella di qualunque retta su un grafico in scala logaritmica: per ogni $r > 0$ si ha da un certo n in poi $\frac{e_{n+1}}{e_n} < r$, e quindi la successione degli errori va a zero più velocemente di una retta di pendenza $\log r$. Questo per qualunque r ; per esempio, i punti potrebbero formare una parabola con la concavità verso il basso, o un'altra curva la cui derivata tende a $-\infty$. Quando una successione soddisfa $\lim_{n \rightarrow \infty} \frac{e_{n+1}}{e_n} = 0$, si dice che ha convergenza *superlineare*. Un esempio è la quarta successione in Figura 2.2, che difatti converge più velocemente degli altri metodi. Studieremo questo metodo nel dettaglio più avanti.

Esercizio 2.12. Consideriamo il metodo del punto fisso applicato a una funzione lineare $\Phi(x) = mx + q$. Quanto vale il tasso di convergenza r ?

Esercizio 2.13. Quanto vale r per i diversi metodi del punto fisso studiati più sopra per $f(x) = x^3 - 2$?

Esercizio 2.14. Supponiamo di avere una funzione con uno zero $\alpha = 1$, e di avere un metodo numerico che produce una successione x_k che converge invece a $\lim x_k = 2$ (per esempio, per un errore di programmazione). Quanto vale il limite r in (2.9)?

Dall'esercizio precedente, ricaviamo che $r \geq 1$ può essere un'indicazione che il metodo non sta convergendo, o sta convergendo alla soluzione sbagliata!

Tasso di convergenza del metodo di bisezione Nel caso del metodo di bisezione, non riusciamo a dimostrare una convergenza secondo la definizione (2.9). Difatti può anche succedere che l'errore e_k cresca da un passo all'altro: è facile costruire una situazione in cui lo zero α è più vicino a c_k che a c_{k+1} . Ne abbiamo visto un esempio proprio quando abbiamo visto il nostro primo esempio per calcolare $\alpha = \sqrt{2}$: applicando il metodo di bisezione a quel problema, $\alpha = \sqrt{2}$ è più vicino a $c_2 = 1.5$ che a $c_3 = 1.25$. Il comportamento non monotono si vede chiaramente anche nella Figura 2.2.

Riusciamo però a dimostrare che $e_n \leq \varepsilon_n$, dove ε_n è una successione che converge linearmente: difatti abbiamo visto che

$$e_n \leq \underbrace{\frac{b_0 - a_0}{2^{n+1}}}_{=\varepsilon_n},$$

e

$$\lim_{n \rightarrow \infty} \frac{\varepsilon_{n+1}}{\varepsilon_n} = \lim_{n \rightarrow \infty} \frac{1}{2} = \frac{1}{2}.$$

Diciamo allora che il metodo di bisezione ha *convergenza pressoché lineare*. Su un grafico in scala logaritmica, gli errori e_n stanno al di sotto della retta data dai punti ε_n , e solitamente non se ne discostano troppo. Questo è confermato anche dalla nostra figura.

Criteri di arresto per il metodo del punto fisso Per un metodo come quello del punto fisso, il criterio di arresto più usato è quello di fermarsi quando due iterate successive sono vicine, cioè $|x_n - x_{n-1}| < \varepsilon$. È importante però osservare che questo non garantisce che $|x_n - \alpha| < \varepsilon$, quando r è vicino a 1 e la convergenza è lenta. Vediamo un esempio nella figura qui sotto.

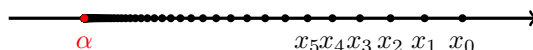


Figura 2.3: Una successione di punti che convergono ad α con tasso di convergenza $r = 0.9$. Osserviamo che anche quando $|x_{k+1} - x_k|$ è piccolo (per esempio per $k = 4$), l'errore $e_k = |x_k - \alpha|$ è molto più grande.

Esercizio 2.15. L'esempio della figura qui sopra è stato generato prendendo punti dati dalla formula $x_k = 10 \cdot 0.9^k$. Quanto vale il rapporto tra $|x_{k+1} - x_k|$ e $|x_k - \alpha|$ per questa successione? E per una successione geometrica generale $x_k = ar^k$ con tasso $r < 1$?

Vantaggi e svantaggi del metodo di punto fisso Svantaggi:

- Convergenza non garantita se non in un intorno della soluzione. È difficile calcolare quanto è grande questo intorno: i teoremi non ci danno una formula facile da calcolare.
- Il criterio di arresto non garantisce un errore $\leq \varepsilon$.

Vantaggi del metodo:

- A seconda della scelta della Φ (e quindi del valore di r), questo metodo può essere anche molto più veloce del metodo di bisezione.
- La convergenza è più regolare del metodo di bisezione, nel senso che la distanza da α scende sempre da un certo punto in poi.

2.4 Metodo di Newton

Il *metodo di Newton*, o *metodo delle tangenti*, è un metodo per la ricerca di zeri che offre convergenza più veloce, ma richiede di utilizzare non solo $f \in \mathcal{C}^1([a, b])$, ma anche la sua derivata f' .

L'idea è una di quelle classiche dell'analisi: quando guardiamo “abbastanza in piccolo”, ogni funzione derivabile è vicina alla sua retta tangente. Ad ogni passo, data un'approssimazione x_n di una soluzione, otteniamo x_{n+1} calcolando uno zero della *retta tangente* al grafico di $f(x)$ in x_n . L'equazione della retta tangente è

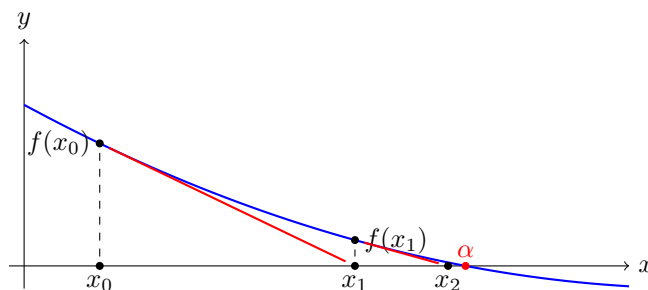
$$y = f(x_n) + f'(x_n)(x - x_n),$$

quindi x_{n+1} è il punto tale che

$$0 = f(x_n) + f'(x_n)(x_{n+1} - x_n),$$

ossia, risolvendo,

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}.$$



Il metodo di Newton quindi è definito dalla successione

$$\begin{cases} x_0 \text{ dato,} \\ x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)} \quad n = 0, 1, 2, \dots \end{cases}$$

Il metodo di Newton quindi è un metodo di punto fisso con $\Phi(x) = x - \frac{f(x)}{f'(x)}$. Notiamo che questa funzione non è definita se $f'(x) = 0$: quando incontriamo un'iterata x_n per cui si ha $f'(x_n) = 0$, il metodo di Newton non è applicabile, perché non riusciamo a calcolare l'iterata successiva.

Sotto l'ipotesi aggiuntiva $f'(\alpha) \neq 0$, riusciamo a dimostrare la convergenza locale del metodo di Newton.

Teorema 2.16. Sia $f \in C^2([a, b])$, e sia $\alpha \in (a, b)$ tale che $f(\alpha) = 0$ e $f'(\alpha) \neq 0$. Allora, il metodo di Newton converge localmente: cioè, esiste un intervallo $I = [\alpha - \rho, \alpha + \rho]$ tale che per ogni successione generata dal metodo con $x_0 \in I$ si ha

- $x_n \in I$ per ogni $n \geq 0$;
- $\lim_{n \rightarrow \infty} x_n = \alpha$.

Dimostrazione. Poiché $f'(\alpha) \neq 0$ e la funzione derivata $f'(x)$ è continua, in un intorno sufficientemente piccolo di α , che possiamo prendere della forma $[\alpha - \hat{\rho}, \alpha + \hat{\rho}]$, si ha $f'(x) \neq 0$; quindi la funzione $\Phi(x)$ è ben definita. Inoltre, possiamo calcolare

$$\Phi'(x) = 1 - \frac{f'(x)f'(x) - f(x)f''(x)}{(f'(x))^2} = \frac{f(x)f''(x)}{(f'(x))^2}; \quad (2.10)$$

pertanto se $f'(\alpha) \neq 0$ abbiamo $\Phi'(\alpha) = 0$. Come nell'Osservazione 2.7, possiamo affermare che in un intorno sufficientemente piccolo di α avremo $|\Phi'(x)| < 1$. Esiste quindi un intervallo $I = [\alpha - \rho, \alpha + \rho]$, con $\rho \leq \hat{\rho}$, in cui la $\Phi(x)$ esiste e $|\Phi'(x)| < 1$ per ogni $x \in I$. In questo intervallo possiamo applicare il teorema del punto fisso e quindi ottenere la tesi. $\square$

Poiché $r = \Phi'(\alpha) = 0$, il metodo di Newton ha convergenza superlineare.

Vantaggi e svantaggi del metodo di Newton Svantaggi:

- Richiede di conoscere non solo f ma anche f' : con Matlab, tipicamente dovremo passare due funzioni anziché una.
- Costo computazionale maggiore: ad ogni passo, dobbiamo valutare f e f' una volta.
- La convergenza è garantita solo localmente, come per il metodo di punto fisso.

Vantaggi:

- Nella maggior parte dei casi, converge molto più velocemente degli altri metodi.

Il comportamento che si osserva tipicamente con il metodo di Newton è un certo numero di iterazioni iniziali in cui l'errore non scende per nulla o quasi, e poi una convergenza molto veloce; appena x_n si avvicina a α e l'errore e_n va sotto una certa soglia, esso comincia ad andare a zero molto velocemente.

Ordine di convergenza di un metodo iterativo Abbiamo visto che il metodo di Newton ha convergenza superlineare, cioè, il limite $\lim_{n \rightarrow \infty} \frac{e_{n+1}}{e_n}$ è uguale a zero: questo vuol dire che la successione degli errori e_n va a zero più velocemente di una serie geometrica, qualunque sia la sua ragione r . In un grafico in scala logaritmica, la differenza di ordinata tra due punti successivi $\log e_{n+1} - \log e_n$ tende a $-\infty$. Fissato un qualunque $r > 0$, abbiamo che da un certo punto in poi $\lim_{n \rightarrow \infty} \frac{e_{n+1}}{e_n} < r$, e questo vuol dire che la successione degli errori e_n sta al di sotto di una retta con coefficiente angolare $\log r$. Questo

significa che un metodo con convergenza superlineare converge sempre (al limite) più velocemente di un metodo lineare.

Per valutare la velocità di convergenza di metodi superlineari in modo più preciso, introduciamo un'altra definizione: dato un $p > 1$, diciamo che un metodo iterativo ha *ordine di convergenza* p se vale

$$\lim_{n \rightarrow \infty} \frac{e_{n+1}}{e_n^p} = s \in (0, \infty).$$

Notare che non richiediamo che $s < 1$, a differenza della convergenza lineare: basta che s sia un limite finito, per ottenere un metodo con convergenza superlineare. Difatti, se un metodo ha ordine di convergenza $p > 1$ allora per qualunque $s < \infty$ vale

$$\lim_{n \rightarrow \infty} \frac{e_{n+1}}{e_n} = \lim_{n \rightarrow \infty} \frac{e_{n+1}}{e_n^p} \cdot \lim_{n \rightarrow \infty} e_n^{p-1} = s \cdot 0 = 0. \quad (2.11)$$

Esempio La successione $x_n = 3 \cdot 10^{-2^n}$ converge a 0 con ordine di convergenza 2: difatti si ha

$$\frac{e_{n+1}}{e_n^2} = \frac{3 \cdot 10^{-2^{n+1}} - 0}{(3 \cdot 10^{-2^n} - 0)^2} = \frac{3 \cdot 10^{-2^{n+1}}}{9 \cdot 10^{-2^{n+1}}} = \frac{1}{3}.$$

I primi elementi della successione sono $x_0 = 3 \cdot 10^{-1}$, $x_1 = 3 \cdot 10^{-2}$, $x_2 = 3 \cdot 10^{-4}$, $x_3 = 3 \cdot 10^{-8}$, $x_4 = 3 \cdot 10^{-16}$, cioè $x_n = 0,00 \dots 03$, dove prima del 3 ci sono 2^n zeri in totale (incluso quello prima della virgola). Il numero di zeri raddoppia ad ogni passo, e questo fa sì che la successione converga molto più velocemente che non una successione che converge linearmente. In un grafico in scala logaritmica, gli errori tendono a scendere come un'esponenziale, e la distanza (in verticale) tra due punti successivi $\log e_{n+1} - \log e_n$ raddoppia ad ogni passo.

Diciamo che un metodo numerico converge con ordine *almeno* p se vale

$$\lim_{n \rightarrow \infty} \frac{e_{n+1}}{e_n^p} = s \in [0, \infty).$$

Questa volta abbiamo incluso lo zero tra i possibili valori ammessi per s . Si può mostrare che questo concetto di “convergenza di ordine almeno k ” si comporta come ci aspettiamo dall'uso della parola “almeno”: se un metodo converge con ordine k , allora converge anche di ordine almeno h per ogni $h \leq k$. (La dimostrazione di questo fatto, che non vediamo, è analoga alla (2.11).)

Convergenza quadratica

Teorema 2.17 (Convergenza almeno quadratica del metodo di Newton). *Supponiamo $f \in \mathcal{C}^2([a, b])$. Se il metodo di Newton converge a $\alpha \in (a, b)$ e se $f'(\alpha) \neq 0$ (cioè se α è uno zero semplice), allora l'errore $e_n = |x_n - \alpha|$ soddisfa*

$$\lim_{n \rightarrow \infty} \frac{e_{n+1}}{e_n^2} = s \in [0, \infty).$$

(cioè questo limite esiste ed è finito.)

Dimostrazione. Usiamo uno sviluppo di Taylor (con resto di Lagrange) in x_n per scrivere l'uguaglianza

$$0 = f(\alpha) = f(x_n) + f'(x_n)(\alpha - x_n) + \frac{f''(\xi_n)}{2}(\alpha - x_n)^2 \quad (2.12)$$

che vale per uno ξ_n compreso tra α e x_n .

Dividiamo per $f'(x_n)$ per ottenere

$$\frac{f''(\xi_n)}{2f'(x_n)}(\alpha - x_n)^2 = -\frac{f(x_n)}{f'(x_n)} - \alpha + x_n = x_{n+1} - \alpha.$$

Allora

$$\frac{x_{n+1} - \alpha}{(x_n - \alpha)^2} = \frac{f''(\xi_n)}{2f'(x_n)}.$$

Questa uguaglianza può essere scritta per ogni n (con un valore diverso di ξ_n). Vogliamo ora passare al limite per $n \rightarrow \infty$. Come già detto sopra, sappiamo che $0 \leq |\xi_n - \alpha| \leq |x_n - \alpha|$, da cui per il teorema dei carabinieri concludiamo che $|\xi_n - \alpha| \rightarrow 0$ e quindi $\xi_n \rightarrow \alpha$. Allora, per la continuità di queste funzioni, $f''(\xi_n) \rightarrow f''(\alpha)$ e $f'(x_n) \rightarrow f'(\alpha)$ (che è diverso da zero); quindi

$$\lim_{n \rightarrow \infty} \frac{x_{n+1} - \alpha}{(x_n - \alpha)^2} = \lim_{n \rightarrow \infty} \frac{f''(\xi_n)}{2f'(x_n)} = \frac{f''(\alpha)}{2f'(\alpha)} = s \in [0, \infty).$$

□

Criterio di arresto Come per il metodo di bisezione, un criterio di arresto euristico per il metodo di Newton è: fermiamoci quando $\left| \frac{f(x_n)}{f'(x_n)} \right| \leq \varepsilon$; ci aspettiamo (senza garanzie!) che questo avvenga quando $|x_n - \alpha| \leq \varepsilon$. Per come è definito il metodo, questo equivale a $|x_n - x_{n+1}| \leq \varepsilon$, cioè, ci fermiamo quando due iterate successive sono abbastanza vicine: questo è il criterio di arresto che avevamo visto anche per il metodo del punto fisso. Avevamo visto che questo criterio funziona male se $r \approx 1$, ma nel caso del metodo di Newton $r = 0$, quindi non abbiamo questo problema.

Convergenza su intervalli C'è un risultato che ci permette di concludere la convergenza del metodo di Newton su un intero intervallo, senza la condizione scomoda da verificare che il punto iniziale sia “abbastanza vicino”. Questa volta per applicare il teorema dobbiamo usare un intervallo in cui lo zero α sia un estremo della funzione.

Teorema 2.18 (Convergenza su intervalli del metodo di Newton). *Sia f una funzione di classe C^2 su un intervallo $I = [a, b]$. Supponiamo che:*

- *f abbia un solo zero α in un estremo dell'intervallo ($\alpha = a$ o $\alpha = b$).*
- *$f(x)f''(x) \geq 0$ su tutti i punti dell'intervallo.*

Allora, a partire da qualunque $x_0 \in [a, b]$ si ha $x_k \in I$ e $\lim_{k \rightarrow \infty} x_k = \alpha$.

Dimostrazione. Visto che f è continua e non ha altri zeri all'interno di I , dev'essere sempre positiva o sempre negativa. Quindi ci sono quattro situazioni possibili:

1. $\alpha = a$, $f(x) > 0$, $f''(x) \geq 0$ in tutti i punti di $(\alpha, b]$;
2. $\alpha = a$, $f(x) < 0$, $f''(x) \leq 0$ in tutti i punti di $(\alpha, b]$;
3. $\alpha = b$, $f(x) > 0$, $f''(x) \geq 0$ in tutti i punti di $[a, \alpha)$;

4. $\alpha = b$, $f(x) < 0$, $f''(x) \leq 0$ in tutti i punti di $[a, \alpha)$.

Trattiamo solo il primo: gli altri sono analoghi, anche perché si ottengono dal primo riflettendo il grafico rispetto all'asse x e/o all'asse y .

Innanzitutto, dimostriamo per induzione che per ogni $n = 0, 1, 2, \dots$ si ha $x_n \in I$. Il caso base $x_0 \in I$ è vero per ipotesi; quindi assumiamo $x_n \in I$ e dimostriamo $x_{n+1} \in I$. Consideriamo l'equazione che definisce la retta tangente in x_n ,

$$y = T(x) = f(x_n) + f'(x_n)(x - x_n).$$

Ricordiamo lo sviluppo di Taylor (2.12)

$$0 = f(\alpha) = f(x_n) + f'(x_n)(\alpha - x_n) + \frac{f''(\xi_n)}{2}(\alpha - x_n)^2,$$

che vale uno ξ_n compreso tra α e x_n . Per le nostre ipotesi, $\frac{f''(\xi_n)}{2}(\alpha - x_n)^2 \geq 0$, quindi $f(x_n) + f'(x_n)(\alpha - x_n) = T(\alpha) \leq 0$. Poiché invece $T(x_n) = f(x_n) \geq 0$, il punto x_{n+1} in cui la retta incrocia l'asse delle ascisse è compreso tra α e x_n ; quindi in particolare $x_{n+1} \in I$. Questo completa la dimostrazione per induzione.

Quello che abbiamo scritto dimostra anche qualcosa di più forte: $x_{n+1} \leq x_n$ per ogni n . Ciò significa che la successione x_n è decrescente; in particolare quindi converge, e il suo limite è necessariamente α perché è l'unico zero in I . $\square$

Zeri di molteplicità superiore a 1 Cosa succede al metodo di Newton se la funzione a cui lo applichiamo ha $f'(\alpha) = 0$? Introduciamo innanzitutto una definizione: data una funzione $f \in \mathcal{C}^m$, diciamo che f ha uno zero di molteplicità m in α se vale

$$f(\alpha) = f'(\alpha) = f''(\alpha) = \dots = f^{(m-1)}(\alpha) = 0, \quad \text{ma } f^{(m)}(\alpha) \neq 0.$$

Esempio: la funzione $f(x) = x^2$ ha uno zero in $\alpha = 0$ di molteplicità $m = 2$.

Esempio: la funzione $f(x) = \sin(2x) - 2\sin(x)$ ha uno zero di molteplicità 3 in $x = 2\pi$. Difatti,

$$\begin{aligned} f(\alpha) &= \sin(2\alpha) - 2\sin(\alpha) = 0 - 0 = 0, \\ f'(\alpha) &= 2\cos(2\alpha) - 2\cos(\alpha) = 2 - 2 = 0, \\ f''(\alpha) &= -4\sin(2\alpha) + 2\sin(\alpha) = -0 + 0 = 0, \\ f'''(\alpha) &= -8\cos(2\alpha) + 2\cos(\alpha) = -8 + 2 \neq 0. \end{aligned}$$

È possibile dimostrare che questa definizione di molteplicità restituisce lo stesso risultato di quella che già conoscete per i polinomi: se un polinomio si fattorizza come

$$p(x) = (x - \alpha)^m g(x),$$

dove $g(x)$ è il prodotto di tutti gli altri fattori di grado 1 diversi da $(x - \alpha)$, allora α è uno zero di molteplicità m . Però la nostra definizione è più generale, in quanto si applica anche a funzioni che non sono polinomi.

È possibile anche dimostrare una fattorizzazione simile a quella che si ha per i polinomi.

Lemma 2.19. *Se $f \in \mathcal{C}^m([a, b])$ ha uno zero di molteplicità m in $\alpha \in (a, b)$, allora si ha $f(x) = (x - \alpha)^m g(x)$, dove $g(x)$ è una funzione continua in $[a, b]$ con $g(\alpha) \neq 0$.*

Dimostrazione. Definiamo $g(x) = \frac{f(x)}{(x-\alpha)^m}$. Questa funzione è definita su tutto $[a, b]$ tranne che in $x = \alpha$, dove si annulla il denominatore. Possiamo però estenderla per continuità. Per farlo, dobbiamo mostrare che esiste ed è finito il limite

$$\lim_{x \rightarrow \alpha} \frac{f(x)}{(x-\alpha)^m}.$$

Per calcolare questo limite, utilizziamo m volte il teorema di De l'Hopital:

$$\lim_{x \rightarrow \alpha} \frac{f(x)}{(x-\alpha)^m} = \lim_{x \rightarrow \alpha} \frac{f'(x)}{m(x-\alpha)^{m-1}} = \lim_{x \rightarrow \alpha} \frac{f''(x)}{m(m-1)(x-\alpha)^{m-2}} = \dots \quad (2.13)$$

$$= \lim_{x \rightarrow \alpha} \frac{f^{(m)}(x)}{m! \cdot 1} = \frac{f^{(m)}(\alpha)}{m!} = C \neq 0. \quad (2.14)$$

(Tutti i limiti tranne l'ultimo sono della forma $\frac{0}{0}$.) Quindi possiamo definire

$$g(x) = \begin{cases} \frac{f(x)}{(x-\alpha)^m} & x \neq \alpha, \\ C & x = \alpha. \end{cases}$$

In questo modo la funzione $g(x)$ è continua su tutto $[a, b]$, ha $g(\alpha) \neq 0$, e soddisfa $f(x) = (x-\alpha)^m g(x)$, come richiesto. $\square$

Dalla riga (2.13) segue anche che

$$\lim_{x \rightarrow \alpha} \frac{f(x)}{(x-\alpha)^m} = C, \quad \lim_{x \rightarrow \alpha} \frac{f'(x)}{(x-\alpha)^{m-1}} = Cm, \quad \lim_{x \rightarrow \alpha} \frac{f''(x)}{(x-\alpha)^{m-2}} = Cm(m-1). \quad (2.15)$$

Usando questi tre limiti, possiamo dimostrare che se α è uno zero di molteplicità $m > 1$, la convergenza del metodo di Newton è più lenta.

Teorema 2.20. Sia $f(x) \in C^m([a, b])$ una funzione con uno zero $\alpha \in (a, b)$ di molteplicità $m > 1$. Allora, il metodo di Newton converge localmente ad α , e la sua velocità di convergenza è lineare con tasso $\frac{m-1}{m}$.

Dimostrazione. Vogliamo applicare di nuovo i risultati di convergenza locale del metodo di punto fisso; questa volta però ci scontriamo con il problema che la funzione $\Phi(x) = x - \frac{f(x)}{f'(x)}$ non è definita in $x = \alpha$, perché il denominatore si annulla.

Possiamo però estendere anche questa funzione per continuità, cioè definire

$$\Phi(x) = \begin{cases} x - \frac{f(x)}{f'(x)} & x \neq \alpha, \\ \alpha & x = \alpha, \end{cases}$$

in modo appunto che α sia un punto fisso. Ci serve però che Φ sia di classe $\mathcal{C}^1$, quindi dobbiamo verificare due cose:

- che Φ sia derivabile in $x = \alpha$, cioè che esiste il limite

$$\lim_{x \rightarrow \alpha} \frac{\Phi(x) - \alpha}{x - \alpha};$$

questo assicura automaticamente anche che Φ è continua in α ;

- che la derivata Φ' sia una funzione continua, cioè

$$\lim_{x \rightarrow \alpha} \Phi'(x) = \Phi'(\alpha).$$

Partiamo dalla prima. Vogliamo utilizzare i primi due limiti calcolati in (2.15), quindi dividiamo numeratore e denominatore per $(x - \alpha)^m$:

$$\begin{aligned} \Phi'(\alpha) &= \lim_{x \rightarrow \alpha} \frac{\Phi(x) - \alpha}{x - \alpha} = \lim_{x \rightarrow \alpha} \frac{x - \frac{f(x)}{f'(x)} - \alpha}{x - \alpha} \\ &= \lim_{x \rightarrow \alpha} 1 - \frac{f(x)}{f'(x)(x - \alpha)} \\ &= 1 - \lim_{x \rightarrow \alpha} \frac{\frac{f(x)}{(x - \alpha)^m}}{\frac{f'(x)(x - \alpha)}{(x - \alpha)^m}} \\ &= 1 - \frac{C}{Cm} = \frac{m - 1}{m}. \end{aligned}$$

Usando questa derivata e la (2.10), otteniamo

$$\begin{aligned} \lim_{x \rightarrow \alpha} \Phi'(x) &= \lim_{x \rightarrow \alpha} \frac{f(x)f''(x)}{(f'(x))^2} \\ &= \lim_{x \rightarrow \alpha} \frac{\frac{f(x)}{(x - \alpha)^m} \cdot \frac{f''(x)}{(x - \alpha)^{m-2}}}{\left(\frac{f'(x)}{(x - \alpha)^{m-1}}\right)^2} \\ &= \frac{C \cdot Cm(m - 1)}{(Cm)^2} = \frac{m - 1}{m} = \Phi'(\alpha). \end{aligned}$$

Nei passaggi, abbiamo diviso numeratore e denominatore per $(x - \alpha)^{2m-2}$.

Quindi è possibile estendere $\Phi(x)$ a una funzione di classe $\mathcal{C}^1$ con $\Phi(\alpha) = \alpha$ e $\Phi'(\alpha) = \frac{m-1}{m} < 1$. Allora, per il teorema del punto fisso, il metodo di Newton converge (localmente) linearmente con tasso $\frac{m-1}{m}$. $\square$

Metodo di Newton modificato Data una funzione f che ha uno zero di molteplicità m in α , abbiamo visto che la funzione

$$\Phi(x) = x - \frac{f(x)}{f'(x)},$$

una volta estesa per continuità, soddisfa

$$\Phi'(\alpha) = 1 - \frac{1}{m}.$$

Questo suggerisce di fare una modifica al metodo: se definiamo

$$\Phi_m(x) = x - m \frac{f(x)}{f'(x)}$$

allora avremo

$$\Phi'_m(\alpha) = 1 - m \frac{1}{m} = 0.$$

Quindi se il valore della molteplicità m è noto a priori possiamo utilizzare il seguente (*metodo di Newton modificato*):

$$x_{n+1} = x_n - m \underbrace{\frac{f(x_n)}{f'(x_n)}}_{:=\Phi_m(x_n)} \quad n = 0, 1, 2, \dots \quad (2.16)$$

Si può dimostrare, con passaggi simili a quelli precedenti, che questo metodo converge (localmente) con ordine almeno 2, quando viene applicato a una funzione che ha uno zero di molteplicità m .

Esercizi Esempio: metodo di Newton su una funzione lineare $f(x) = ax + b$: ha convergenza in un passo.

Esempio: metodo di Newton su $f(x) = x^2 - a$: è anche un metodo per calcolare radici quadrate a mano. Per esempio, calcolando $\sqrt{17}$ con $x_0 = 4$, già x_2 ha 5 cifre significative esatte.

Esempio: metodo di Newton su $f(x) = x^2$: lo zero è di ordine 2, e la convergenza diventa lineare con ragione $\frac{1}{2}$.

Esempio: metodo di Newton modificato su $f(x) = x^2$ per recuperare convergenza quadratica (converge di nuovo in un passo).

2.5 Varianti del metodo di Newton

Non sempre si ha a disposizione (in modo facile da calcolare) la derivata della funzione f . Per questo esistono alcune varianti del metodo di Newton che cercano di evitare di calcolare derivate.

Metodo delle corde Si ottiene rimpiazzando la derivata $f'(x_n)$ con un valore costante fissato c :

$$\begin{cases} x_0 \text{ dato,} \\ x_{n+1} = x_n - \frac{f(x_n)}{c}, \quad n = 0, 1, 2, \dots \end{cases}$$

Per esempio, si può prendere $c = f'(x_0)$, se lo si conosce; questo richiede di calcolare la f' una volta sola anziché una volta per passo come nel metodo di Newton. Geometricamente, questo metodo corrisponde a rimpiazzare le rette tangenti che si usano nel metodo di Newton con rette che hanno coefficiente angolare costante c . Un esempio di questo metodo è l'iterazione di punto fisso che abbiamo già studiato $x_{k+1} = x_k - \frac{1}{4}(x_k^3 - 2)$, dove la c vale 4.

Anche il metodo delle corde è un metodo di punto fisso, con $\Phi(x) = x - \frac{1}{c}f(x)$, quindi possiamo studiarne la convergenza locale con le tecniche che abbiamo visto: $\Phi'(x) = 1 - \frac{1}{c}f'(x)$, e per avere $|\Phi'(\alpha)| < 1$ dobbiamo scegliere c in modo che $f'(\alpha) \in (0, 2c)$.

Metodo delle secanti È l'iterazione che si ottiene rimpiazzando, nel metodo di Newton, $f'(x_n)$ con il rapporto incrementale calcolato sugli ultimi due punti,

$$\frac{f(x_n) - f(x_{n-1})}{x_n - x_{n-1}}.$$

In questo modo si rimpiazza la derivata con una sua approssimazione più economica da calcolare: abbiamo già calcolato $f(x_n)$ e $f(x_{n-1})$ nei passi precedenti del metodo, quindi dobbiamo fare solo due sottrazioni e una divisione (che in realtà diventa una moltiplicazione, quando si va a scrivere il metodo per esteso).

Per applicare il metodo delle secanti, serve partire da *due* punti iniziali x_0 e x_1 :

$$\begin{cases} x_0, x_1 \text{ dati,} \\ x_{n+1} = x_n - \frac{f(x_n)(x_n - x_{n-1})}{f(x_n) - f(x_{n-1})}, \quad n = 1, 2, \dots \end{cases}$$

Anche questo metodo converge localmente, e il suo ordine di convergenza (sotto opportune ipotesi sulla funzione) è $p = \frac{1+\sqrt{5}}{2} \approx 1.618$. Non dimostriamo quest'ultimo risultato, che è più complesso: difatti questo non è un metodo del tipo $x_{n+1} = \Phi(x_n)$, perché ogni valore dipende dalle *due* iterate precedenti.

Capitolo 3

Aritmetica di macchina

Se provate a implementare i metodi precedenti su un computer con Matlab, noterete in molti casi un comportamento abbastanza particolare: i metodi spesso smettono di convergere attorno a 10^{-15} o 10^{-16} , come accade nella Figura 3.1. Per comprendere cosa sta succedendo, dobbiamo studiare come i numeri reali vengono rappresentati in un computer.

Rappresentazione in base Scegliamo un intero $\beta > 1$ che sarà la *base* della nostra rappresentazione. Questo teorema descrive la notazione scientifica che già conoscete:

Teorema 3.1 (rappresentazione scientifica in base β). *Fissato un intero $\beta > 1$ (la base), ogni numero reale $x \neq 0$ si può scrivere come*

$$x = \pm \beta^p \sum_{i=0}^{\infty} c_i \beta^{-i}, \quad (3.1)$$

dove i c_i (cifre) sono interi $0 \leq c_i < \beta$.

Questa scrittura è unica se aggiungiamo le condizioni che $c_1 \neq 0$, e che c_i non sono tutti uguali a $\beta - 1$ da un certo punto in poi.

Per esempio, $x = -764.88888 \dots$ (periodico) si scrive in base $\beta = 10$ come

$$x = -10^2(7 \cdot 10^0 + 6 \cdot 10^{-1} + 4 \cdot 10^{-2} + 8 \cdot 10^{-3} + 8 \cdot 10^{-4} + 8 \cdot 10^{-5} + \dots)$$

Questo è un modo più prolisso di scrivere la notazione scientifica che già conosciamo: $x = -7.6488888 \dots \cdot 10^2$.

Terminologia: p si chiama *esponente* di x , la quantità nella sommatoria in (3.1) si chiama *mantissa*.

Non ci interessa qui dimostrare questo teorema: “sappiamo che è vero” fin dalla scuola primaria, e più si va verso fatti base e più bisogna essere puntigliosi e formali nelle dimostrazioni. Però ci soffermiamo un attimo sulle condizioni di unicità.

La prima condizione $c_1 \neq 0$ serve a escludere scritture alternative con zeri iniziali, per esempio

$$-764.88888 = -10^4(0 \cdot 10^0 + 0 \cdot 10^{-1} + 7 \cdot 10^{-2} + 6 \cdot 10^{-3} + 4 \cdot 10^{-4} + \dots) = -10^4 \cdot 0.07648 \dots$$

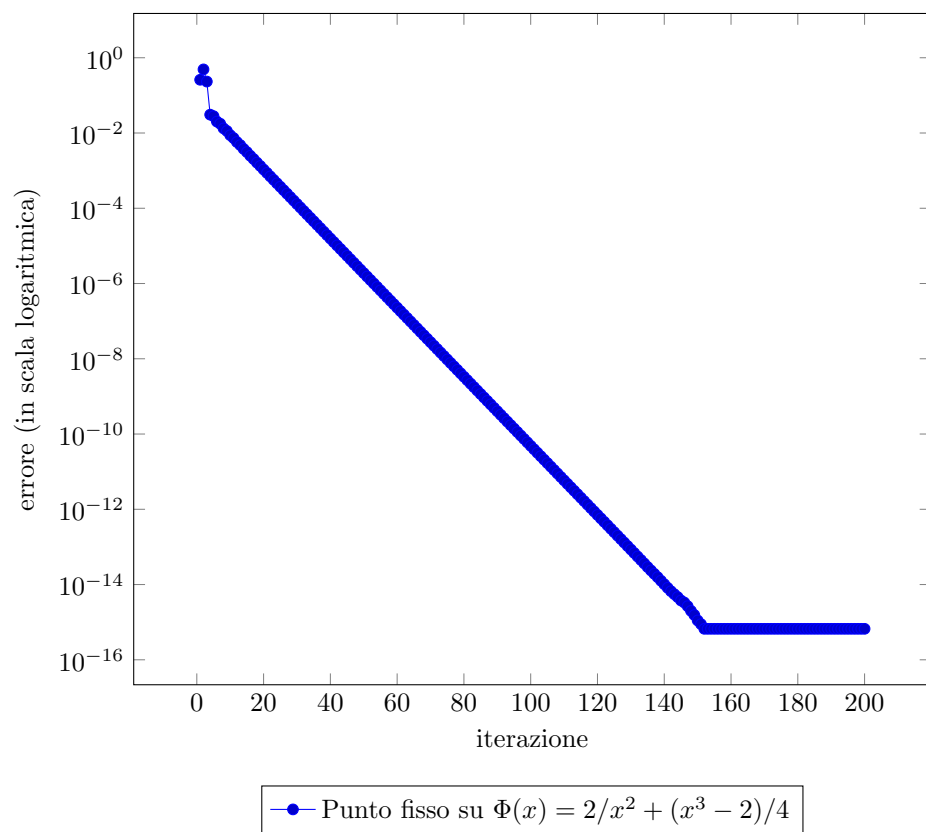


Figura 3.1: Stagnazione di un metodo iterativo attorno alla precisione di macchina.

La rappresentazione senza zeri iniziali si chiama *normalizzata*.

La seconda condizione esclude scritture come $0.999999\dots$ (periodico): se vi ricordate come si sommano le serie, $9 \cdot 10^{-1} + 9 \cdot 10^{-2} + 9 \cdot 10^{-3} + \dots$ è uguale a 1; quindi $0.999999\dots$ è un modo diverso di scrivere il numero 1 che vogliamo escludere.

Numeri di macchina Su un computer, possiamo rappresentare solo una quantità finita di numeri.

Definizione 3.2. L'insieme dei *numeri di macchina* (o *floating-point*) *normalizzati*, $\mathbb{F}(\beta, t, m, M)$ è l'insieme dei numeri della forma

$$\pm \beta^p \sum_{i=0}^{t-1} c_i \beta^{-i}, \quad p \in \{m, m+1, \dots, M\}$$

Ci sono due grossi cambiamenti rispetto alla (3.1): abbiamo t cifre (anziché infinite) nella mantissa, e un insieme finito per gli esponenti, da m a M .

Esempio Prendiamo i numeri di macchina costruiti con $\beta = 10$, $t = 3$, $m = -2$, $M = 3$.

Il numero di macchina positivo più piccolo è $10^{-2}(1 \cdot 10^0 + 0 \cdot 10^{-1} + 0 \cdot 10^{-2}) = 0.0100$. I numeri di macchina successivi si ottengono incrementando le cifre: $0.0101, 0.0102, \dots, 0.0999$. Altri numeri compresi tra due di questi, ad esempio 0.01005 , non si possono scrivere con sole $t = 3$ cifre significative. A questo punto abbiamo elencato tutti i numeri con esponente -2 ; il numero di macchina ancora successivo è un numero con esponente -1 : precisamente, $10^{-1}(1 \cdot 10^0 + 0 \cdot 10^{-1} + 0 \cdot 10^{-2}) = 0.100$. Si prosegue in questo modo, ottenendo

0.0100, 0.0101, 0.0102, ..., 0.0999	esponente $p = -2$
0.100, 0.101, 0.102, ..., 0.999	esponente $p = -1$
1.00, 1.01, 1.02, ..., 9.99	esponente $p = 0$
10.0, 10.1, 10.2, ..., 99.9	esponente $p = 1$
100, 101, 102, ..., 999	esponente $p = 2$
1000, 1010, 1020, ..., 9990	esponente $p = 3$

L'elenco che abbiamo fatto qui sopra contiene tutti i numeri di macchina normalizzati positivi dell'insieme $\mathbb{F}(10, 3, -2, 3)$. A questi vanno aggiunti quelli negativi, che sono gli stessi ma con un segno meno davanti.

Notiamo che i numeri con esponente -2 hanno distanza 10^{-4} l'uno dal successivo; i numeri con esponente -1 hanno distanza 10^{-3} l'uno dal successivo, fino ai numeri con esponente 3 che hanno distanza 10.

Esercizio 3.3. Qual è il più piccolo intero positivo che *non* appartiene all'insieme $\mathbb{F}(10, 3, -2, 3)$ riportato qui sopra?

Quindi i numeri di macchina non sono equispaziati: i numeri più vicini allo zero (con p piccolo) hanno una spaziatura minore tra l'uno e l'altro. Vedremo che queste spaziature variabili sono proprio quello che consente un'approssimazione con un errore *relativo* basso.

Numero di macchina più piccolo, più grande, successivo Il numero positivo più piccolo rappresentabile, che chiamiamo ω , si ottiene scegliendo $p = m$ e mantissa $1000 \dots 0$. Il numero più grande rappresentabile, Ω , si ottiene scegliendo $p = M$ e mantissa con tutte cifre $\beta - 1$.

Dato un numero di macchina normalizzato positivo

$$x = \beta^p \left(c_0 \beta^0 + c_1 \beta^1 + \dots + c_{t-1} \beta^{-(t-1)} \right),$$

il numero di macchina immediatamente successivo è quello che si ottiene aggiungendo 1 all'ultima cifra c_{t-1} , cioè $x + \beta^{p-(t-1)}$. Questo è chiaro se l'ultima cifra è diversa da $\beta - 1$; se l'ultima cifra è $\beta - 1$, aggiungendo 1 all'ultima cifra ci sono dei riporti: ad esempio, aggiungendo 0.0001 a 0.0999 otteniamo 0.1000. Quando si hanno questi riporti, però, l'ultima cifra è sempre 0, quindi siamo sempre in grado di scrivere questo numero $x + \beta^{p-(t-1)}$ come un numero di macchina, eventualmente cambiando esponente.

Approssimazione con numeri di macchina

Teorema 3.4. *Dato un numero reale $x \in [-\Omega, -\omega] \cup [\omega, \Omega]$, esiste un numero di macchina $\tilde{x}$ tale che*

$$\frac{|\tilde{x} - x|}{|x|} < \beta^{1-t}. \quad (3.2)$$

Dimostrazione. Possiamo assumere che x sia positivo (il caso negativo è analogo), e che non sia esso stesso un numero di macchina (altrimenti il risultato è ovvio, basta prendere $\tilde{x} = x$). Allora x è compreso tra due numeri di macchina successivi, chiamiamoli $\underline{x}$ e $\overline{x}$. In particolare, $\underline{x}$ si ottiene troncando la rappresentazione (3.1), cioè arrestando la sommatoria a t anziché a ∞ , e ha lo stesso esponente p del numero x : per esempio, il troncamento di $10^{-1} \cdot 7.648888 \dots$ è $10^{-1} 7.64$.

Possiamo scegliere se prendere $\tilde{x} = \underline{x}$ (troncamento o arrotondamento verso 0), $\tilde{x} = \overline{x}$ (arrotondamento verso infinito), o quello dei due che ha un errore minore (arrotondamento al più vicino); tutti questi forniscono arrotondamenti che soddisfano la (3.2). In ogni caso, visto che x e $\tilde{x}$ sono entrambi compresi nell'intervallo $(\underline{x}, \overline{x})$, si ha

$$|\tilde{x} - x| < \overline{x} - \underline{x} = \beta^{p-(t-1)}. \quad (3.3)$$

Inoltre, visto che la prima cifra di x è almeno 1 e le altre sono positive o nulle,

$$x \geq \beta^p. \quad (3.4)$$

Dividendo membro a membro queste due disuguaglianze (notare che i versi sono quelli giusti per farlo!) si ha la tesi. $\square$

Errore relativo Data un'approssimazione $\tilde{x}$ di un numero reale $x \neq 0$, il suo *errore relativo* è

$$\varepsilon = \frac{\tilde{x} - x}{x}. \quad (3.5)$$

È un tipo di errore naturale da considerare: l'errore assoluto $\tilde{x} - x$ da solo non dice nulla: possiamo fare degli esempi dalla vita reale di errori su lunghezze e prezzi. Per esempio, aver misurato una lunghezza con un errore di 0.5 cm da

solo non vuol dire nulla: è molto diverso se questa lunghezza è la distanza dalla terra alla luna, o una tolleranza di fabbricazione sulla cover del vostro cellulare.

Possiamo riscrivere la (3.5) come $\tilde{x} = x(1 + \varepsilon)$. Quindi la (3.2) dice che per ogni x in quegli intervalli esiste un numero di macchina $\tilde{x} = x(1 + \varepsilon)$ che lo approssima con un errore relativo che soddisfa $|\varepsilon| \leq \beta^{1-t}$. La quantità $u = \beta^{1-t}$ (detta *precisione di macchina*) non dipende da x , ma solo dall'insieme di numeri di macchina scelto.

Valori non normalizzati Su un computer, una variabile può assumere anche alcuni valori speciali, oltre ai numeri di macchina visti sopra:

- Lo zero: difficile fare senza, e non è un numero normalizzato!
- $+\infty, -\infty, -0$: vengono aggiunti, con regole aritmetiche ispirate dall'analisi come $1/-\infty = -0$, per far sì che gli algoritmi possano funzionare anche in alcuni casi limite.
- NaN, che viene restituito come una sorta di codice d'errore da alcune operazioni non valide come $0/0$ e $\infty - \infty$.
- Alcuni numeri in più compresi tra 0 e ω , chiamati *numeri denormalizzati*; non ci interessano qui.

Numeri a doppia precisione Esiste uno standard (IEEE 754) per l'aritmetica di macchina; praticamente tutti i computer e i linguaggi di programmazione si conformano a quello. Lo standard specifica quali parametri β, t, m, M scegliere e il risultato di ogni operazione. Il formato più comune è quello noto come **double**, **float64** o **binary64**; esso corrisponde a scegliere $\beta = 2, t = 53, m = -1022, M = 1023$. Ogni numero viene memorizzato in 64 bit (=valori 0 oppure 1).

s	$eeee\dots e$	$d_1d_2d_3\dots d_{52}$
-----	---------------	-------------------------

Il primo bit codifica il segno, i successivi 11 bit l'esponente, e i successivi 52 sono le cifre della mantissa (d_0 è sempre uguale a 1 e non serve memorizzarla).

Con questi valori, il Teorema 3.4 ci dice che ogni numero con valore assoluto compreso tra $\omega \approx 2.2 \cdot 10^{-308}$ e $\Omega \approx 1.8 \cdot 10^{308}$ può venire approssimato con precisione di macchina $u \approx 2.2 \cdot 10^{-16}$. Matlab (che useremo per programmare in questo corso) usa questo formato.

C'è anche un altro formato comune, chiamato **single** o **float32**, che ha precisione di macchina $u \approx 10^{-8}$. Con questo formato gli errori relativi sono maggiori, ma è possibile memorizzare più numeri nello stesso spazio di memoria (32 bit l'uno).

Quando scriviamo (per esempio) $x = 0.3$ in Matlab, questo numero non viene memorizzato esattamente: non appartiene a $\mathbb{F}(2, 53, -1022, 1023)$, ma anzi in base 2 è il numero periodico $0.0100\overline{1}$. Questo numero periodico viene troncato a $t = 53$ cifre binarie. Quindi il numero con cui il computer lavora non è *esattamente* 0.3, bensì il numero in base 2

$$\begin{aligned}\tilde{x} &= 0.01001100110011001100110011001100110011001100110011001100110011 \\ &= 2^{-2} \cdot \underbrace{1.00110011001100110011001100110011001100110011001100110011}_{53 \text{ cifre binarie nella mantissa}}\end{aligned}$$

cioè, in base 10,

$$\tilde{x} = 0.2 \underbrace{999999999999999}_{15 \text{ volte } 9} 88897769753748434595763683319091796875,$$

che ha un errore relativo $|\tilde{x} - 0.3|/0.3$ minore di $u = 2.2 \cdot 10^{-16}$, come promesso dal Teorema 3.4.

Un buon sito che potete usare per visualizzare lo sviluppo in base 2 dei numeri e fare qualche esperimento è <https://www.exploringbinary.com/floating-point-converter/>.

Operazioni di macchina Una volta memorizzati due numeri, possiamo chiedere al computer di calcolarne la somma: per esempio se scriviamo in Matlab $x = 0.3$; $y = 0.4$; $x + y$ viene visualizzato il risultato.

Sappiamo già che il computer memorizza approssimazioni $\tilde{x}$ e $\tilde{y}$ di 0.3 e 0.4, che soddisfano $\tilde{x} = 0.3(1 + \varepsilon_1)$, $\tilde{y} = 0.4(1 + \varepsilon_2)$, con $|\varepsilon_i| \leq u$. Ma c'è una terza fonte di errore: anche se $\tilde{x}$ e $\tilde{y}$ sono numeri di macchina, la loro somma $\tilde{x} + \tilde{y}$ potrebbe non esserlo; quindi va approssimata anche lei con un numero di macchina. L'operazione "calcola la somma di $\tilde{x}$ e $\tilde{y}$ e rimpiazzala con il numero di macchina più vicino" viene indicata con $\tilde{x} \oplus \tilde{y}$. Analogamente definiamo $\ominus, \odot, \oslash$. Quindi $\tilde{x} \oplus \tilde{y} = (\tilde{x} + \tilde{y})(1 + \varepsilon)$, per un opportuno errore ε che soddisfa $|\varepsilon| \leq u$, e analogamente per le altre operazioni.

Underflow/overflow Ci sono due casi particolari da citare: eseguendo operazioni con numeri di macchina, si può ottenere come risultato un numero più grande di Ω in valore assoluto. In questo caso, viene restituito $+\infty$ o $-\infty$. Questo fenomeno si chiama *overflow*. Similmente, quando un'operazione produce un numero più piccolo di ω in valore assoluto, viene restituito 0 (*underflow*).

Vediamo qui sotto alcuni esempi interessanti del risultato di operazioni di macchina.

Esempio 3.5. Consideriamo di nuovo il sistema di numerazione con $\beta = 10, t = 3, m = -2, M = 3$. Qual è il risultato delle seguenti operazioni (arrotondando sempre al numero di macchina più vicino)?

- $100 \ominus 0.01$;
- $1.01 \ominus 1.01$;
- $(7.24 \oplus 0.04) \oplus 10$;
- $7.24 \oplus (0.04 \oplus 10)$;
- $(1 \oslash 3) \oplus (1 \oslash 3) \oplus (1 \oslash 3)$;
- $5000 \oplus 5000$;
- $0.01 \odot 0.01$;
- $50 \oplus 0.01$.

Le risposte sono queste:

- $100 \ominus 0.01$ è il numero di macchina più vicino al risultato esatto 99.99, cioè 100 stesso.

- $1.01 \ominus 1.01$: poiché $1.01 - 1.01 = 0$, viene restituito 0.
- Il risultato di $7.24 \oplus 0.04$ è il numero di macchina più vicino a $7.24 + 0.04 = 7.28$, cioè 7.28. Il risultato di $7.28 \oplus 10$ è il numero di macchina più vicino a 17.28, cioè 17.3.
- Ora dobbiamo fare le operazioni in ordine diverso. Il risultato di $10 \oplus 0.04$ è il numero di macchina più vicino a 10.04, cioè 10. Il risultato di $7.24 \oplus 10$ è il numero di macchina più vicino a 17.24, cioè 17.2. È interessante notare che l'ordine diverso produce risultati diversi: per l'operazione $\oplus$ non vale sempre la proprietà associativa!
- $(1 \oslash 3) \oplus (1 \oslash 3) \oplus (1 \oslash 3) = 0.333 \oplus 0.333 \oplus 0.333 = 0.999$, che è diverso dal risultato esatto $1/3 + 1/3 + 1/3 = 1$.
- $5000 \oplus 5000 = +\infty$, visto che il risultato esatto 10000 supera il più grande numero di macchina $\Omega = 9990$ (un esempio di overflow).
- $0.01 \odot 0.01 = 0$, visto che il risultato esatto $0.01 \cdot 0.01 = 10^{-4}$ è più piccolo del più piccolo numero di macchina positivo $\omega = 0.01$ (un esempio di underflow).

Commenti Perché è stato scelto questo sistema per rappresentare i numeri reali sul computer? Esistono alternative, come rappresentare i numeri tramite razionali, cioè rapporti di interi (ma i denominatori diventano spesso *molto* grandi anche quando si fanno operazioni semplici) o tenere traccia degli errori calcolando esplicitamente degli intervalli di inclusione per ogni quantità calcolata (ma gli intervalli diventano spesso *molto* grandi anche quando si fanno operazioni semplici). Il sistema dei numeri di macchina è quello che si è affermato come il più comodo nelle applicazioni, ed è diventato lo standard. È un sistema che richiede di effettuare continuamente delle approssimazioni; per questo è importante comprendere teoricamente il loro impatto.

Capitolo 4

Analisi dell'errore

Errore analitico ed errore di macchina Siamo ora in grado di studiare nel dettaglio l'errore commesso nella soluzione di problemi numerici. Supponiamo di dover calcolare una quantità $\alpha \neq 0$ tramite una successione x_n che converge ad α . Il primo errore che commettiamo è il cosiddetto *errore analitico*, che commettiamo quando rimpiazziamo α con x_n . In generale, rimpiazziamo la quantità desiderata α con una quantità effettivamente calcolata γ , per esempio se decidiamo di fare cinque passi del metodo di Newton avremo $\gamma = x_5$. Possiamo definire l'errore analitico come

$$e_{an} = \frac{\gamma - \alpha}{\alpha}. \quad (4.1)$$

Il secondo errore che commettiamo è il cosiddetto *errore di macchina*, dovuto al fatto che nel calcolare γ usiamo non numeri e operazioni esatte, ma le loro approssimazioni ottenute con numeri in virgola mobile. Per esempio, se vogliamo applicare il metodo di Newton alla funzione $f(x) = x^2 - 2$, su un computer otterremo la successione

$$\begin{cases} \tilde{x}_0 = \text{approssimazione con numeri di macchina di } x_0, \\ \tilde{x}_{n+1} = \tilde{x}_n \ominus (\tilde{x}_n \odot \tilde{x}_n \ominus 2) \oslash (2 \odot \tilde{x}_n). \end{cases}$$

Questo processo produce un risultato $\tilde{\gamma}$ che è, solitamente, diverso da γ . (Come già fatto nel capitolo precedente, usiamo la tilde per indicare un'approssimazione ottenuta tramite aritmetica di macchina.) È importante notare che questo $\tilde{\gamma}$ *non* è necessariamente il numero di macchina più vicino a γ , ma può essere più grande a causa dell'accumulo di molti errori successivi (come in uno degli esempi precedenti: $1 \oslash 3 \oplus 1 \oslash 3 \oplus 1 \oslash 3 \neq 1$).

Definiamo l'errore di macchina come

$$e_{mac} = \frac{\tilde{\gamma} - \gamma}{\gamma}.$$

L'effetto combinato dei due errori è il cosiddetto *errore totale*

$$e_{tot} = \frac{\tilde{\gamma} - \alpha}{\alpha}.$$

Vediamo ora un teorema che consente di darne un'espressione approssimata. Essa è approssimata perché assumiamo che tutti questi errori siano piccoli, e

ignoriamo termini che contengono il prodotto di due errori: un po' come quando approssimiamo $\exp(x) \approx 1 + x$ per valori di x piccoli, ignorando i termini di grado superiore $x^2, x^3, \dots$ perché, essendo il prodotto di più copie di x , sono molto più piccoli di x .

Nel seguito, usiamo il simbolo $a \doteq b$ per indicare che a e b sono uguali a patto di ignorare “termini di ordine superiore”. Cosa siano esattamente questi termini di ordine superiore dipende dal contesto: in questo caso, termini che sono il prodotto di due o più errori.

Teorema 4.1.

$$e_{tot} \doteq e_{an} + e_{mac}.$$

Dimostrazione. Notiamo innanzitutto che riarrangiando la (4.1) si ottiene

$$\gamma = \alpha(1 + e_{an}).$$

Ora possiamo scrivere

$$\begin{aligned} e_{tot} &= \frac{\tilde{\gamma} - \alpha}{\alpha} = \frac{\tilde{\gamma} - \gamma}{\alpha} + \frac{\gamma - \alpha}{\alpha} \\ &= e_{mac} \frac{\gamma}{\alpha} + e_{an} = e_{mac}(1 + e_{an}) + e_{an} \\ &= e_{mac} + e_{mac}e_{an} + e_{an} \doteq e_{mac} + e_{an}. \end{aligned} \quad \square$$

In questa dimostrazione abbiamo usato solo il fatto che abbiamo fatto due approssimazioni successive una dopo l'altra: prima abbiamo rimpiazzato α con γ , poi γ con $\tilde{\gamma}$. Questo quindi è un risultato generale che ci dice che gli errori relativi commessi successivamente si sommano (al prim'ordine).

Questo risultato è già sufficiente a spiegare qualitativamente il comportamento che osserviamo nella figura 3.1. Nell'analizzare i metodi abbiamo già stimato l'errore analitico $e_{an} = |x_n - \alpha|$, e sappiamo che si comporta come cr^k , dove k è il numero di passi, quindi scende come una retta su un grafico in scala semi-logaritmica. Nelle prime 150 iterazioni l'errore analitico è più grande di 10^{-16} , ed è la quantità dominante nella somma degli errori; più avanti, l'errore analitico è minore, e quello dominante è l'errore di macchina, che è dell'ordine di 10^{-16} e cresce solo molto lentamente con il numero di iterazioni.

Approssimazione al prim'ordine dell'errore di macchina In realtà l'errore di macchina ε_{mac} non è sempre dell'ordine di 10^{-16} : l'accumulo di errori successivi può far sì che esso diventi molto più grande. Vediamo ora come analizzare questo errore su un esempio semplice.

Supponiamo di avere dati due numeri reali x, y (non necessariamente dei numeri di macchina), e di voler calcolare la quantità $\gamma = f(x, y) = x^2 - y^2$. Per farlo, calcoliamo in quest'ordine

$$A = x \cdot x, \quad B = y \cdot y, \quad \gamma = A - B.$$

Il primo errore che commettiamo è quello che facciamo approssimando x e y con dei numeri di macchina: otteniamo $\tilde{x} = x(1 + \varepsilon_x)$, $\tilde{y} = y(1 + \varepsilon_y)$, con $|\varepsilon_x|, |\varepsilon_y| \leq u$. Successivamente, effettuiamo le tre operazioni (due moltiplicazioni e una sottrazione) con i numeri di macchina:

$$\tilde{A} = \tilde{x} \odot \tilde{x}, \quad \tilde{B} = \tilde{y} \odot \tilde{y}, \quad \tilde{\gamma} = \tilde{A} \ominus \tilde{B}.$$

Nel calcolare $\tilde{A}$, si combinano un nuovo errore di macchina ε_1 , ottenuto quando approssimiamo il risultato esatto $\tilde{x}^2$ con un numero di macchina, e gli errori precedenti che hanno modificato x in $\tilde{x}$:

$$\begin{aligned}\tilde{A} &= \tilde{x} \odot \tilde{x} = \tilde{x}^2(1 + \varepsilon_1) = x^2(1 + \varepsilon_x)^2(1 + \varepsilon_1) \\ &= x^2(1 + 2\varepsilon_x + \underbrace{\varepsilon_x^2}_{\mathcal{O}(u^2)})(1 + \varepsilon_1) \\ &= x^2(1 + 2\varepsilon_x + \varepsilon_1 + \underbrace{2\varepsilon_x\varepsilon_1}_{\mathcal{O}(u^2)} + \mathcal{O}(u^2)).\end{aligned}$$

Possiamo evitare di calcolare nel dettaglio i termini che contengono il prodotto di due o più errori; ci basta dire che sono dell'ordine di $\mathcal{O}(u^2)$ per avere una stima dell'errore al primo ordine. Questo ci dice che l'errore relativo su A è

$$\varepsilon_A = \frac{\tilde{A} - A}{A} = 2\varepsilon_x + \varepsilon_1 + \mathcal{O}(u^2) :$$

ricordiamo difatti che $\varepsilon_A = \frac{\tilde{A} - A}{A}$ con qualche manipolazione algebrica diventa $\tilde{A} = A(1 + \varepsilon_A)$.

In modo analogo si ha

$$\tilde{B} = \tilde{y} \odot \tilde{y} = B(1 + \varepsilon_B), \quad \varepsilon_B = 2\varepsilon_y + \varepsilon_2 + \mathcal{O}(u^2),$$

dove abbiamo chiamato ε_2 l'errore che facciamo rimpiazzando il risultato del secondo prodotto $\tilde{y} \odot \tilde{y}$ con un numero di macchina. Già questi errori ε_A e ε_B possono essere leggermente più grandi della precisione di macchina, fino a $3u$. Ma l'errore maggiore si verifica quando consideriamo anche la sottrazione finale. Abbiamo

$$\tilde{\gamma} = \tilde{A} \ominus \tilde{B} = (\tilde{A} - \tilde{B})(1 + \varepsilon_3).$$

Con un po' di manipolazioni, possiamo scrivere anche questo risultato in una forma simile a quella sopra: il risultato esatto $\gamma = x^2 - y^2$, moltiplicato per un fattore $(1 + e_{mac})$:

$$\begin{aligned}\tilde{\gamma} &= (\tilde{A} - \tilde{B})(1 + \varepsilon_3) \\ &= \frac{\gamma}{x^2 - y^2} \left(x^2(1 + \varepsilon_A) - y^2(1 + \varepsilon_B) \right) (1 + \varepsilon_3) \\ &= \gamma \left(\underbrace{\frac{x^2 - y^2}{x^2 - y^2}}_{=1} + \frac{x^2}{x^2 - y^2} \varepsilon_A - \frac{y^2}{x^2 - y^2} \varepsilon_B \right) (1 + \varepsilon_3) \\ &= \gamma \left(1 + \frac{x^2}{x^2 - y^2} \varepsilon_A - \frac{y^2}{x^2 - y^2} \varepsilon_B + \varepsilon_3 + \mathcal{O}(u^2) \right) \\ &= \gamma \left(1 + \frac{x^2}{x^2 - y^2} (2\varepsilon_x + \varepsilon_1) - \frac{y^2}{x^2 - y^2} (2\varepsilon_y + \varepsilon_2) + \varepsilon_3 + \mathcal{O}(u^2) \right) \\ &= \gamma \left(1 + \frac{2x^2}{x^2 - y^2} \varepsilon_x + \frac{2x^2}{x^2 - y^2} \varepsilon_1 - \frac{2y^2}{x^2 - y^2} \varepsilon_y - \frac{y^2}{x^2 - y^2} \varepsilon_2 + \varepsilon_3 + \mathcal{O}(u^2) \right).\end{aligned}$$

Questo ci dice che l'errore di macchina è

$$e_{mac} = \frac{\tilde{\gamma} - \gamma}{\gamma} = \frac{2x^2}{x^2 - y^2} \varepsilon_x + \frac{x^2}{x^2 - y^2} \varepsilon_1 - \frac{2y^2}{x^2 - y^2} \varepsilon_y - \frac{y^2}{x^2 - y^2} \varepsilon_2 + \varepsilon_3 + \mathcal{O}(u^2).$$

Usando solo il fatto che $|\varepsilon_i| \leq u$ per $i = x, y, 1, 2, 3$, la stima migliore che possiamo dare è quella che otteniamo usando la disuguaglianza triangolare: dividendo tutto per γ , abbiamo

$$\begin{aligned} \frac{|\tilde{\gamma} - \gamma|}{|\gamma|} &\leq \frac{2x^2}{|x^2 - y^2|} |\varepsilon_x| + \frac{x^2}{|x^2 - y^2|} |\varepsilon_1| + \frac{2y^2}{|x^2 - y^2|} |\varepsilon_y| + \frac{y^2}{|x^2 - y^2|} |\varepsilon_2| + |\varepsilon_3| \\ &\leq \frac{2x^2}{|x^2 - y^2|} u + \frac{x^2}{|x^2 - y^2|} u + \frac{2y^2}{|x^2 - y^2|} u + \frac{y^2}{|x^2 - y^2|} u + u. \end{aligned} \quad (4.2)$$

Questo calcolo ci permette di individuare per quali valori di x e y l'errore di macchina diventa molto grande: in particolare, quando x^2 e y^2 sono molto vicini tra loro, il denominatore $x^2 - y^2$ è molto più piccolo del numeratore, e quindi e_{mac} diventa molto grande. Questo è un fenomeno comune: l'errore di macchina tipicamente diventa molto alto quando facciamo sottrazioni tra due numeri molto vicini tra loro.

Questo calcolo è stato lungo e macchinoso anche per un'espressione semplice come $x^2 - y^2$; potete immaginare come le cose si complicano ancora di più quando eseguiamo sequenze più lunghe di operazioni. C'è però almeno una parte di questo calcolo che possiamo effettuare facilmente: quella dei coefficienti di fronte a ε_x e ε_y .

Numero di condizionamento Concentriamoci sull'errore ε_x , e supponiamo per un attimo di ignorare gli errori $\varepsilon_y, \varepsilon_1, \varepsilon_2, \varepsilon_3$, ponendoli uguali a zero. Allora, possiamo notare che quello che stiamo calcolando è $\tilde{\gamma} = f(\tilde{x}, y)$: l'unico errore che commettiamo è quello dovuto a rimpiazzare il valore iniziale x con $\tilde{x}$, e tutte le altre operazioni sono esatte. Abbiamo a disposizione uno strumento dell'analisi per dare un'espressione approssimata di questo errore: lo sviluppo di Taylor. Consideriamo la funzione $g(x) = f(x, y)$, dove nascondiamo la dipendenza dalla y perché la stiamo considerando costante. Se questa funzione è di classe $\mathcal{C}^1$, possiamo farne uno sviluppo di Taylor in x per ottenere

$$g(\tilde{x}) \doteq g(x) + \frac{\partial g(x)}{\partial x} (\tilde{x} - x).$$

La differenza $\tilde{x} - x$ è uguale a $x(1 + \varepsilon_x) - x = x\varepsilon_x$, e abbiamo ignorato (con la notazione $\doteq$) il termine che dipende da $(\tilde{x} - x)^2$, perché contiene il termine ε_x^2 e quindi è molto più piccolo. Allora otteniamo

$$\frac{\tilde{\gamma} - \gamma}{\gamma} = \frac{g(\tilde{x}) - g(x)}{g(x)} \doteq \frac{\frac{\partial g(x)}{\partial x} x \varepsilon_x}{g(x)} \quad (4.3)$$

Questa formula ci dice che quando calcoliamo una funzione $g(x)$ nel punto $\tilde{x}$ anziché x , l'errore relativo sul risultato $\frac{\tilde{\gamma} - \gamma}{\gamma}$ è uguale (al prim'ordine) a quello sul dato in ingresso

$$\varepsilon_x = \frac{\tilde{x} - x}{x}$$

moltiplicato per un fattore

$$\frac{\partial g}{\partial x} \frac{x}{g(x)}.$$

Il valore assoluto di questa quantità, cioè

$$\kappa_{g,x} = \left| \frac{\partial g}{\partial x} \frac{x}{g(x)} \right|$$

viene detto *numero di condizionamento* della funzione g nel punto x . Esso è una sorta di ‘fattore di amplificazione’ dell’errore, che ci dice di quanto un errore cresce (o decresce) quando effettuiamo operazioni.

Esempio La funzione $f(x) = \frac{x}{1-x}$ ha numero di condizionamento

$$\kappa_{f,x} = \left| \frac{f'(x)x}{f(x)} \right| = \left| \frac{\frac{1}{(1-x)^2}x}{\frac{x}{1-x}} \right| = \frac{1}{|1-x|}.$$

Questa quantità è grande quando $x \approx 1$. Un errore relativo piccolo sull’argomento x , per esempio $x = 0.9991$, $\tilde{x} = 0.999$, causa un errore relativo grande sul risultato della funzione $\frac{f(\tilde{x})-f(x)}{f(x)}$.

Una funzione che ha un numero di condizionamento grande viene detta *mal condizionata*.

Possiamo riconoscere nel nostro esempio (4.2), con $g(x) = f(x, y) = x^2 - y^2$, che il coefficiente davanti a ε_x è proprio

$$\kappa_{g,x} = \left| \frac{\partial g}{\partial x} \right| \frac{|x|}{|g(x)|} = |2x| \frac{|x|}{|x^2 - y^2|}.$$

Analogamente, il coefficiente davanti a ε_y è uguale a

$$\left| \frac{\partial (x^2 - y^2)}{\partial y} \right| \frac{|y|}{|x^2 - y^2|} = |-2y| \frac{|y|}{|x^2 - y^2|}.$$

Errore inerente, algoritmico, e stabilità (*) Notare che l’errore di macchina dipende non dalla funzione che vogliamo calcolare, ma dalla sequenza di operazioni che usiamo per calcolarla: per esempio $\gamma = f(x, y) = x^2 - y^2 = (x+y)(x-y)$ è la stessa funzione scritta in due modi diversi, ma se usiamo la seconda espressione su un calcolatore possiamo ottenere un risultato $\tilde{\gamma}$ diverso:

$$\tilde{\gamma} = \tilde{f}(\tilde{x}, \tilde{y}) = (\tilde{x} \oplus \tilde{y}) \odot (\tilde{x} \ominus \tilde{y}).$$

non è per forza uguale alla quantità che avevamo chiamato $\tilde{\gamma}$ in precedenza: abbiamo visto nell’Esercizio 3.5 che anche solo facendo operazioni di macchina in ordine diverso possiamo ottenere risultati diversi. Lavorando nello stesso modo, possiamo ottenere in questo caso la disuguaglianza sull’errore

$$\begin{aligned} \frac{|\tilde{\gamma} - \gamma|}{|\gamma|} &\leq \frac{2x^2}{|x^2 - y^2|} |\varepsilon_x| + \frac{2y^2}{|x^2 - y^2|} |\varepsilon_y| + |\varepsilon_1| + |\varepsilon_2| + |\varepsilon_3| \\ &\leq \frac{2x^2}{|x^2 - y^2|} u + \frac{2y^2}{|x^2 - y^2|} u + 3u. \end{aligned} \quad (4.4)$$

Abbiamo ottenuto un risultato diverso da quello in (4.2): i due algoritmi non eseguono le stesse operazioni sul calcolatore, e quindi sono soggetti a errori diversi. Tuttavia, una parte del risultato è la stessa: i coefficienti di fronte a ε_x e ε_y . Come abbiamo visto sopra, questi coefficienti sono uguali al numero di condizionamento della $f(x, y)$ rispetto alla x e rispetto alla y . Questi numeri di condizionamento dipendono solo dalla funzione che vogliamo calcolare, non dalla sequenza di operazioni (*algoritmo*) che stiamo usando per farlo. Quindi

all'interno dell'errore di macchina ci sono degli addendi che sono inevitabili, anche se usiamo l'algoritmo migliore del mondo per calcolare il risultato: sono quelli dovuti all'errore sull'approssimazione con numeri di macchina dei dati in ingresso. Questo errore si chiama *errore inerente*, e si può stimare come il numero di condizionamento della funzione che stiamo calcolando moltiplicato per u . Se abbiamo una funzione che dipende da più di un valore in ingresso, come la nostra $f(x, y)$ che dipende sia dalla x che dalla y , abbiamo un numero di condizionamento diverso rispetto a ognuna delle variabili, e l'errore inerente è la somma di questi errori $(\kappa_{f,x} + \kappa_{f,y})u$.

I termini rimanenti, che dipendono dall'algoritmo che stiamo utilizzando, compongono l'*errore algoritmico*. Nella (4.4), l'errore algoritmico è minore o uguale di $3u$, quindi è sempre dello stesso ordine di grandezza della precisione di macchina, a parte un piccolo fattore moltiplicativo; questo è quanto di meglio possiamo sperare per un algoritmo. Nella (4.2) invece l'errore algoritmico $\frac{x^2}{|x^2-y^2|}|\varepsilon_1| + \frac{y^2}{|x^2-y^2|}|\varepsilon_2| + |\varepsilon_3|$ può essere molto più grande della precisione di macchina. Comunque, anche in questo caso però l'errore algoritmico non può mai essere molto più grande dell'errore inerente: la quantità $\frac{x^2}{|x^2-y^2|}u$ è sempre minore di $\frac{2x^2}{|x^2-y^2|}u$, e la quantità $\frac{y^2}{|x^2-y^2|}u$ è sempre minore di $\frac{2y^2}{|x^2-y^2|}u$. Un algoritmo in cui l'errore algoritmico è sempre al più dello stesso ordine di grandezza dell'errore inerente si dice *stabile*. Quindi entrambi gli algoritmi che abbiamo visto per calcolare $f(x, y) = x^2 - y^2$ sono stabili. La stabilità è una proprietà che cerchiamo sempre di avere nei nostri algoritmi: ci dice che l'algoritmo non fa mai un errore molto più grande di quello inerente che è inevitabile. Quindi un algoritmo stabile (informalmente) restituisce il minore errore possibile, come ordine di grandezza.

Esempio di un algoritmo instabile (*) Consideriamo la funzione $f(x) = 2x + x^2 = (1+x)^2 - 1$. Queste due espressioni equivalenti portano a due algoritmi per calcolare il valore della funzione:

$$\tilde{f}_A(\tilde{x}) = 2 \odot \tilde{x} \oplus \tilde{x} \odot \tilde{x}, \quad \tilde{f}_B(\tilde{x}) = (1 \oplus \tilde{x}) \odot (1 \oplus \tilde{x}) \ominus 1.$$

Studiamo l'errore di macchina del secondo algoritmo, che scriviamo per comodità con delle quantità intermedie:

$$A = 1 + x \quad B = A \cdot A, \quad \gamma = B - 1.$$

Abbiamo innanzitutto

$$\begin{aligned} \tilde{A} &= 1 \oplus \tilde{x} = (1 + x(1 + \varepsilon_x))(1 + \varepsilon_1) \\ &= (1 + x) \left(1 + \frac{x}{1+x} \varepsilon_x \right) (1 + \varepsilon_1) \\ &= (1 + x) \left(1 + \underbrace{\frac{x}{1+x} \varepsilon_x + \varepsilon_1 + \mathcal{O}(u^2)}_{=\varepsilon_A} \right) \end{aligned}$$

poiché il termine $x\varepsilon_x\varepsilon_1$ contiene due fattori minori di u . Poi continuiamo con

$$\begin{aligned}\tilde{B} &= \tilde{A} \odot \tilde{A} = A^2(1 + \varepsilon_A)^2(1 + \varepsilon_2) \\ &= B(1 + 2\varepsilon_A + \varepsilon_2 + \mathcal{O}(u^2)) \\ &= B \left(1 + \underbrace{2\frac{x}{1+x}\varepsilon_x + 2\varepsilon_1 + \varepsilon_2 + \mathcal{O}(u^2)}_{\varepsilon_B} \right).\end{aligned}$$

Per la sottrazione finale, notiamo che il termine 1 si rappresenta esattamente con un numero di macchina, quindi non commettiamo nessun errore nel rappresentarlo. Consideriamo quindi

$$\begin{aligned}\tilde{\gamma} &= \tilde{B} \ominus 1 \\ &= (B(1 + \varepsilon_B) - 1)(1 + \varepsilon_3) \\ &= (B - 1) \left(1 + \frac{B}{B - 1}\varepsilon_B \right) (1 + \varepsilon_3) \\ &= (B - 1) \left(1 + \frac{B}{B - 1}\varepsilon_B + \varepsilon_3 + \mathcal{O}(u^2) \right) \\ &= \gamma \left(1 + \frac{(1+x)^2}{2x+x^2} \left(2\frac{x}{1+x}\varepsilon_x + 2\varepsilon_1 + \varepsilon_2 \right) + \varepsilon_3 + \mathcal{O}(u^2) \right) \\ &= \gamma \left(1 + \frac{2(1+x)}{2+x}\varepsilon_x + \frac{(1+x)^2}{2x+x^2}(2\varepsilon_1 + \varepsilon_2) + \varepsilon_3 + \mathcal{O}(u^2) \right)\end{aligned}$$

Quindi

$$\begin{aligned}e_{mac} &= \left| \frac{2(1+x)}{2+x}\varepsilon_x + \frac{(1+x)^2}{2x+x^2}(2\varepsilon_1 + \varepsilon_2) + \varepsilon_3 + \mathcal{O}(u^2) \right| \\ &\leq \left| \frac{2(1+x)}{2+x} \right| u + \left| 3\frac{(1+x)^2}{2x+x^2} \right| u + u + \mathcal{O}(u^2)\end{aligned}$$

Come già sappiamo, il coefficiente davanti a ε_x è uguale al numero di condizionamento della funzione:

$$\kappa_{f,x} = \left| \frac{f'(x)x}{f(x)} \right| = \left| \frac{(2+2x)x}{2x+x^2} \right| = \left| \frac{2+2x}{2+x} \right|;$$

questo numero di condizionamento diventa molto grande quando $x \approx -2$; quindi la funzione è mal condizionata intorno a questo punto. Però l'errore di macchina diventa molto grande anche quando $x \approx 0$, per colpa del denominatore $2x+x^2$; mentre il numero di condizionamento resta limitato per $x \approx 0$. Quindi quando $x \approx 0$ l'errore algoritmico è molto più grande dell'errore inerente, e perciò l'algoritmo dato da $\tilde{f}_B$ è instabile. (Si noti che quando abbiamo esattamente $x = 0$ si ha $f(x) = 0$, quindi non è ben definito il concetto di errore relativo perché ci richiederebbe di dividere per 0.)

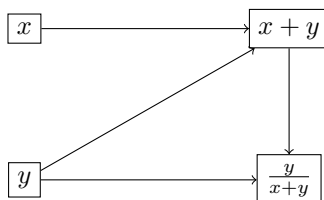
Esempio: attacchi avversari Alcune reti neurali sono usate in informatica per classificare immagini, ad esempio di segnali stradali: il modello restituisce come output dei numeri reali che sono probabilità (secondo il modello) che

l'immagine contenga un determinato oggetto: per esempio, un certo segnale può essere riconosciuto con una probabilità dell'87% di essere un divieto di transito, 7% di essere divieto di sosta, 6% di essere un limite di velocità. Alcuni ricercatori hanno osservato che queste probabilità possono essere funzioni mal condizionate dei pixel dell'immagine, e specialmente di alcuni pixel in posizione chiave. Facendo piccole modifiche mirate ai segnali, come quelle rappresentate nell'immagine qui sotto (fonte), è possibile fare sì che l'immagine venga riconosciuta dalla rete neurale come un segnale stradale diverso, per esempio un limite di velocità. Questo mal condizionamento può avere conseguenze catastrofiche, per esempio, per i sistemi di guida automatica.

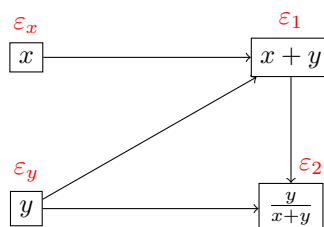


4.1 Grafi computazionali per l'analisi dell'errore

Mostriamo ora una tecnica per calcolare l'approssimazione al primo ordine dell'errore algoritmico basata sulla rappresentazione delle operazioni eseguite come un grafo. Rappresentiamo innanzitutto le operazioni da fare come un grafo in cui le frecce collegano gli *input* di un'operazione ai suoi *output*; per esempio qui sotto vediamo il grafo che rappresenta il calcolo della quantità $\frac{y}{x+y}$ a partire dai due input x, y .



Quando memorizziamo i due input x, y come numeri di macchina, commettiamo errori $\varepsilon_x, \varepsilon_y$, come visto sopra. Inoltre, i risultati delle due operazioni (somma e divisione) sono affetti da errori $\varepsilon_1, \varepsilon_2$. Disegniamo di nuovo il grafo, ma questa volta mettiamo in ogni nodo gli errori in cui incorriamo.



Il punto chiave ora è capire come gli errori si propagano da un nodo all'altro del grafo. Concentriamoci sulla prima addizione. Essa ha come input x e y , e

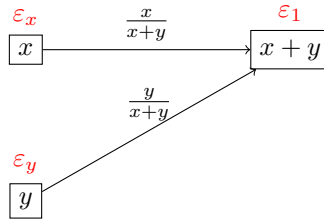
come output $x + y$. La formula (4.3) ci dice che l'errore sull'input x ε_x causa un errore sul risultato della somma di

$$\frac{\partial(x+y)}{\partial x} \frac{x}{x+y} \varepsilon_x = 1 \cdot \frac{x}{x+y} \varepsilon_x.$$

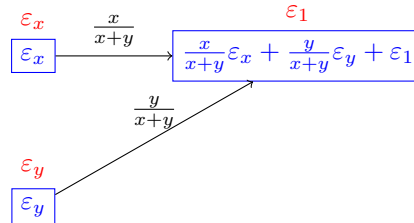
Similmente,

$$\frac{\partial(x+y)}{\partial y} \frac{y}{x+y} \varepsilon_y = 1 \cdot \frac{y}{x+y} \varepsilon_y.$$

Scriviamo i due coefficienti di amplificazione dell'errore $\frac{x}{x+y}$ e $\frac{y}{x+y}$ sui lati del grafo che collegano il risultato della somma (output) ai suoi due input x e y :



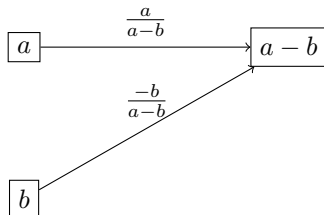
Questi due coefficienti rappresentano come l'errore si propaga lungo l'operazione da fare, in questo caso una somma. L'errore relativo totale sulla quantità calcolata $x + y$ quindi è dato dalla somma dell'errore effettuato sull'operazione stessa, ε_1 , e dei due errori propagati dai due input, $\frac{x}{x+y} \varepsilon_x + \frac{y}{x+y} \varepsilon_y$. Scriviamo questi errori nel grafo, rimpiazzando i risultati delle operazioni in modo da non avere troppe formule.



Procediamo in modo simile per ogni operazione contenuta nel grafo. Tutte le volte che nel grafo computazionale c'è una somma tra due quantità a e b , sui lati corrispondenti scriveremo $\frac{a}{a+b}$ e $\frac{b}{a+b}$. Analogamente, quando abbiamo una sottrazione $g(a, b) = a - b$ (tra due numeri a e b) i due coefficienti di amplificazione sono

$$\frac{\partial g(a, b)}{\partial a} \frac{a}{g(a, b)} = 1 \frac{a}{a - b} = \frac{a}{a - b}, \quad \frac{\partial g(a, b)}{\partial b} \frac{b}{g(a, b)} = -1 \frac{b}{a - b} = \frac{-b}{a - b},$$

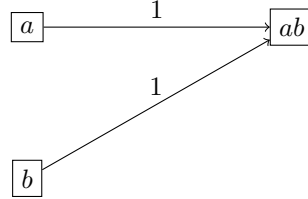
e quindi andremo a scrivere



Per un prodotto $g(a, b) = ab$ invece si ha

$$\frac{\partial g(a, b)}{\partial a} \frac{a}{g(a, b)} = b \frac{a}{ab} = 1, \quad \frac{\partial g(a, b)}{\partial b} \frac{b}{g(a, b)} = a \frac{b}{ab} = 1$$

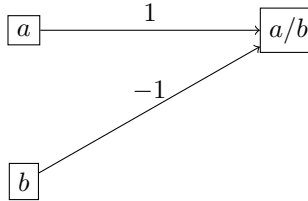
e quindi



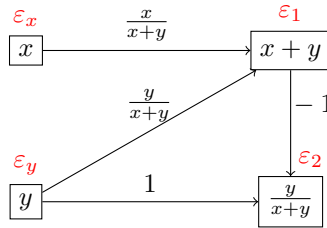
e infine per una divisione $g(a, b) = a/b$

$$\frac{\partial g(a, b)}{\partial a} \frac{a}{g(a, b)} = \frac{1}{b} \frac{a}{a/b} = 1, \quad \frac{\partial g(a, b)}{\partial b} \frac{b}{g(a, b)} = -\frac{a}{b^2} \frac{b}{a/b} = -1$$

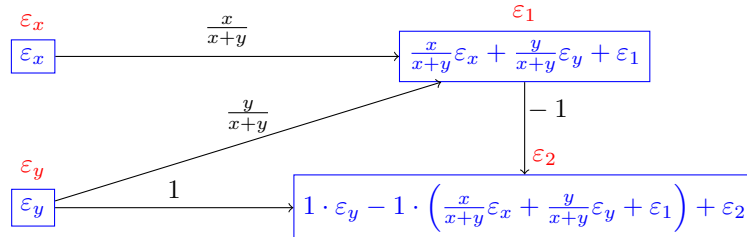
quindi



Questi calcoli ci dicono che coefficienti scrivere su ogni arco del nostro grafo, a seconda di a quale operazione elementare esso corrisponde. Se sugli archi del nostro grafo ci fossero avere operazioni più complesse che il computer è in grado di eseguire, per esempio un elevamento a potenza o un logaritmo, a ognuna di esse possiamo associare dei coefficienti di amplificazione in base alla formula (4.3). Completiamo quindi il calcolo per il nostro esempio: i coefficienti di amplificazione lungo i due lati rimanenti sono 1 e -1 , quindi



Quindi scrivendo in ogni nodo l'errore totale corrispondente si ha



Facciamo attenzione a moltiplicare i coefficienti di propagazione per l'errore “totale” (in blu) e non quello “parziale” (in rosso).

Possiamo ottenere lo stesso risultato anche eseguendo le operazioni in un ordine diverso. Per determinare il coefficiente davanti a ε_x nel risultato finale, moltiplichiamo tra loro tutti i coefficienti di amplificazione che compaiono lungo i lati del percorso che lo collega al risultato: $-1 \cdot \frac{x}{x+y} \cdot \varepsilon_x$, andando a ritroso. Analogamente per ε_1 : il prodotto (questa volta con un solo coefficiente di amplificazione) è $-1 \cdot \varepsilon_1$. Nel caso di ε_y , i percorsi sono due, e dobbiamo sommare i loro contributi:

$$1 \cdot \varepsilon_y + (-1) \cdot \frac{y}{x+y} \cdot \varepsilon_y.$$

In altre parole, l'algoritmo che possiamo seguire per calcolare l'errore di macchina lungo una sequenza di operazioni è questo:

- Rappresentiamo in un grafo la sequenza di operazioni da svolgere, collegando ogni input con il suo output;
- Scriviamo sopra ogni nodo un errore ε_k che rappresenta l'errore “parziale” in cui si incorre nell'operazione del nodo.
- Scriviamo sopra ogni arco del grafo il coefficiente di propagazione corrispondente, che dice come il valore nel nodo di arrivo dipende da quello nel nodo di partenza.
- Per ogni errore parziale, calcoliamo il prodotto dei coefficienti lungo il percorso che lo collega al risultato finale (o lungo ognuno dei percorsi, se sono più di uno); esso rappresenta un addendo dell'errore di macchina $\frac{\tilde{\gamma} - \gamma}{\gamma}$.

Osservazione 4.2. Con un algoritmo simile a questo possiamo calcolare gli errori *assoluti*, anziché relativi, cioè essenzialmente le derivate. Questa è una variante del famoso algoritmo di *backpropagation* che viene utilizzato per calcolare derivate quando si allenano modelli di intelligenza artificiale.

Capitolo 5

Equazioni lineari e autovalori

5.1 Richiami di algebra lineare

Ricordiamo che il prodotto matrice-vettore $A\mathbf{x}$ è definito come $(Ax)_i = \sum_{j=1}^n A_{ij}x_j$ (riga per colonna). Più geometricamente, crea una combinazione lineare $\mathbf{v}_1x_1 + \mathbf{v}_2x_2 + \dots + \mathbf{v}_nx_n$ delle colonne $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_n$ di A .

Un *sistema lineare* è il problema inverso del prodotto: data $A \in \mathbb{R}^{n \times n}$ e $\mathbf{b} \in \mathbb{R}^n$, trovare il vettore di coefficienti $\mathbf{x}$ che servono per scrivere $\mathbf{b}$ come combinazione lineare delle colonne di A .

Partiamo studiando sistemi quadrati, cioè $A \in \mathbb{R}^{n \times n}$ (o $\mathbb{C}^{n \times n}$). Un sistema ha *una e una sola* soluzione quando le righe/colonne di A formano una base di $\mathbb{R}^n$ (in particolare, quando sono linearmente indipendenti). Ci concentriamo su sistemi che soddisfano questa ipotesi. Dall'algebra lineare, sappiamo che se A è invertibile esiste una matrice $A^{-1} \in \mathbb{R}^{n \times n}$ tale che la soluzione si scrive come prodotto $\mathbf{x} = A^{-1}\mathbf{b}$. Questa matrice è unica e soddisfa $A^{-1}A = AA^{-1} = I$, la matrice con uni sulla diagonale e zeri altrove.

Non possiamo scrivere $\mathbf{x} = \mathbf{b}A^{-1}$ (dimensioni non compatibili per il prodotto; l'ordine dei fattori conta!), né $\mathbf{x} = \frac{\mathbf{b}}{A}$ (non vuol dire nulla, e non mi specifica l'ordine!).

In Matlab, esiste una funzione `inv(A)` che calcola la matrice inversa, quindi potremmo scrivere `inv(A) * b`; però questo metodo di risolvere un sistema lineare è più costoso (e spesso anche più inaccurato) di altri algoritmi. C'è una notazione diversa per risolvere un sistema, $\mathbf{x} = \mathbf{A} \setminus \mathbf{b}$. Occhio: la barra è la “backslash” `\`, non `/`. Per non confondere le barre, pensate a questa operazione come a una sorta di “divisione da un lato”: c'è una barra di frazione, e la $\mathbf{A}$ sta al di sotto.

Detto informalmente, per calcolare le colonne della matrice inversa dobbiamo risolvere gli n sistemi lineari $A^{-1}\mathbf{e}_1, A^{-1}\mathbf{e}_2, \dots, A^{-1}\mathbf{e}_n$; quando poi calcoliamo $\mathbf{x} = \text{inv}(\mathbf{A}) * \mathbf{b}$ facciamo una combinazione lineare di queste soluzioni. Per questo l'errore rischia di essere maggiore di quello fatto risolvendo un solo sistema lineare.

Determinanti Altri algoritmi da evitare per risolvere sistemi lineari su un computer sono quelli basati su determinanti, come il *metodo di Cramer*. Tipi-

camente i determinanti sono più lenti da calcolare, e per matrici grandi vanno spesso in overflow/underflow; per esempio, $\det(0.1 I_{400 \times 400})$ restituisce 0 su Matlab. Quindi i determinanti sono inaffidabili anche solo per dire se una matrice è singolare o no.

Nelle prossime sezioni vedremo invece gli algoritmi che Matlab utilizza per risolvere sistemi lineari. Ma prima di farlo, studiamo il condizionamento di questo problema.

5.2 Condizionamento della soluzione di sistemi lineari

Partiamo studiando il condizionamento della soluzione di sistemi lineari. Per farlo, introduciamo alcuni strumenti teorici.

Norme vettoriali L'analisi dell'errore che abbiamo fatto nei capitoli precedenti assumeva di avere funzioni $f : \mathbb{R} \rightarrow \mathbb{R}$, funzioni $x = f(a)$ di un reale a . La soluzione di un sistema lineare invece è una funzione $\mathbf{x} = f(A, \mathbf{b})$ che prende una matrice e un vettore e restituisce un vettore. Potremmo studiare separatamente il condizionamento di ogni componente rispetto a ogni componente dell'input, ma si preferisce un approccio diverso che passa attraverso il misurare "distanze" tra vettori e matrici.

Lo strumento teorico che ci serve è una *norma vettoriale*, cioè una funzione che "assomiglia al valore assoluto" per vettori.

Si definisce *norma vettoriale* una funzione $f : \mathbb{C}^n \rightarrow \mathbb{R}$ che ha queste proprietà.

1. $f(\mathbf{v}) \geq 0$ per ogni vettore $\mathbf{v} \in \mathbb{C}^n$, e l'uguaglianza vale solo per il vettore zero.
2. $f(\alpha \mathbf{v}) = |\alpha| f(\mathbf{v})$ per ogni vettore $\mathbf{v} \in \mathbb{C}^n$ e scalare $\alpha \in \mathbb{C}$.
3. $f(\mathbf{v} + \mathbf{w}) \leq f(\mathbf{v}) + f(\mathbf{w})$ per ogni $\mathbf{v}, \mathbf{w} \in \mathbb{C}^n$.

Notate che queste proprietà sono analoghe a quelle del valore assoluto; per esempio l'ultima è la disuguaglianza triangolare. Difatti non è complicato verificare che il valore assoluto è una norma per $n = 1$.

Una norma di solito non si indica con $f(\mathbf{v})$, ma con $\|\mathbf{v}\|$ (due stanghette).

Le norme più usate sono le seguenti.

- Norma-1: $\|\mathbf{v}\|_1 = \sum_{i=1}^n |v_i|$.
- Norma-2 (o Euclidea): $\|\mathbf{v}\|_2 = \sqrt{\sum_{i=1}^n |v_i|^2} = \sqrt{\mathbf{v}^* \mathbf{v}}$.
- Norma infinito: $\|\mathbf{v}\|_\infty = \max_{i \in \{1, 2, \dots, n\}} |v_i|$.

Si dimostra che tutte e tre soddisfano le proprietà qui sopra. L'unica un po' più difficile è la disuguaglianza triangolare per la norma-2; le altre potete provare a farle come esercizio.

Esempio: calcola le tre norme di $\begin{bmatrix} 2 \\ -3 \\ -1 \end{bmatrix}$.

Data una norma vettoriale, $\|\mathbf{v} - \mathbf{w}\|$ fornisce un modo di misurare la distanza tra $\mathbf{v}$ e $\mathbf{w}$. Ogni norma fornisce valori diversi, che danno modi leggermente diversi di definire questa distanza. In generale, si dimostra (noi non lo vediamo) che date due norme qualunque queste non possono restituire valori troppo diversi l'una dall'altra: per ogni coppia di norme $\|\cdot\|_p, \|\cdot\|_q$ esistono due reali $c_1, c_2 > 0$ tali che

$$c_1 \|\mathbf{v}\|_p \leq \|\mathbf{v}\|_q \leq c_2 \|\mathbf{v}\|_p \quad \text{per ogni } \mathbf{v} \in \mathbb{C}^n.$$

Queste costanti spesso sono una funzione della dimensione; per esempio, per ogni $\mathbf{v} \in \mathbb{C}^n$ si ha

$$\|\mathbf{v}\|_0 \leq \|\mathbf{v}\|_2 \leq \sqrt{n} \|\mathbf{v}\|_0.$$

(esercizio: dimostrarlo.)

Esempio: disegnare le “sfere” $\|\mathbf{v}\| = 1$ in $\mathbb{R}^2$ nelle tre norme viste.

Norme matriciali Similmente ai vettori, possiamo definire norme su matrici. Si dice *norma matriciale* una funzione $f : \mathbb{C}^{n \times n} \rightarrow \mathbb{R}$ che soddisfa queste proprietà:

1. $f(A) \geq 0$ per ogni matrice $A \in \mathbb{C}^{n \times n}$, e l'uguaglianza vale solo per la matrice zero.
2. $f(\alpha A) = |\alpha| f(A)$ per ogni $A \in \mathbb{C}^{n \times n}$ e scalare $\alpha \in \mathbb{C}$.
3. $f(A + B) \leq f(A) + f(B)$ per ogni $A, B \in \mathbb{C}^{n \times n}$.
4. $f(AB) \leq f(A)f(B)$ per ogni $A, B \in \mathbb{C}^{n \times n}$.

Rispetto al caso dei vettori, abbiamo aggiunto una proprietà che lega la norma al prodotto di matrici. Notate stavolta una differenza rispetto al valore assoluto; non abbiamo $\|AB\| = \|A\|\|B\|$. Sarebbe impossibile ottenere norme con questa proprietà più forte.

Norme matriciali indotte È possibile costruire una norma matriciale a partire da ogni norma vettoriale in questo modo. Fissata una norma vettoriale $\|\cdot\|_p$ (per esempio quelle con $p = 1, 2, \infty$ viste sopra) definiamo

$$\|A\|_p = \max_{\substack{\mathbf{u} \in \mathbb{R}^n \\ \|\mathbf{u}\|_p = 1}} \|A\mathbf{u}\|_p. \quad (5.1)$$

In generale la matrice A manderà gli infiniti vettori con norma uguale a 1 in vettori di lunghezza diversa; prendiamo il più lungo, e definiamolo come la norma. Si può dimostrare (non lo faremo) che questa definizione soddisfa tutte le proprietà di una norma matriciale.

La definizione fatta in questo modo serve per assicurare un'ulteriore proprietà.

Teorema 5.1 (compatibilità della norma matriciale indotta). *La norma matriciale $\|A\|_p$ definita qui sopra soddisfa $\|A\mathbf{v}\|_p \leq \|A\|_p \|\mathbf{v}\|_p$ per ogni matrice $A \in \mathbb{C}^{n \times n}$ e vettore $\mathbf{v} \in \mathbb{C}^n$.*

Notare che mi serve usare *la stessa* norma: per esempio se misuro i vettori in una norma p con $p \in \{1, 2, \infty\}$, dovrò usare la norma matriciale costruita usando la norma p nella (5.1).

Dimostrazione. Prima un caso particolare: se $\mathbf{v} = 0$, allora anche $A\mathbf{v} = 0$ e sia il membro di sinistra che quello di destra si annullano. Possiamo quindi proseguire considerando il caso $\mathbf{v} \neq 0$, e quindi $\|\mathbf{v}\|_p \neq 0$.

Mi basta considerare il vettore $\mathbf{u} = \mathbf{v} \frac{1}{\|\mathbf{v}\|_p}$. Questo vettore ha norma uguale a 1, per le proprietà delle norme vettoriali (considerandolo come il prodotto dello scalare $\frac{1}{\|\mathbf{v}\|_p}$ e del vettore $\mathbf{v}$); quindi

$$\frac{1}{\|\mathbf{v}\|_p} \|A\mathbf{v}\|_p = \|A\mathbf{u}\|_p \leq \|A\|_p,$$

ed eliminando il denominatore otteniamo la tesi. $\square$

Norma di Frobenius Non tutte le norme matriciali si ottengono da questa costruzione. Un altro esempio è la *norma di Frobenius*,

$$\|A\|_F = \sqrt{\sum_{i,j=1}^n |A_{ij}|^2}.$$

Questa funzione soddisfa tutte le proprietà di una norma matriciale, ma non è una norma matriciale *indotta*. Un modo veloce di vederlo è considerando la norma della matrice identità: $\|I\|_F = \sqrt{n}$, ma per una norma matriciale indotta segue dalla definizione che $\|I\|_p = 1$.

Norme, autovalori e raggio spettrale Data una matrice quadrata A , ricordiamo che quando $A\mathbf{v} = \mathbf{v}\lambda$ per un qualche vettore $\mathbf{v} \in \mathbb{C}^n$ (diverso dal vettore nullo!) e scalare $\lambda \in \mathbb{C}$ si dice che λ è un *autovalore* e $\mathbf{v}$ è un *autovettore* di A . Avete visto ad algebra lineare diverse proprietà degli autovalori. Prendendo norme, abbiamo che

$$\|\mathbf{v}\|_p |\lambda| = \|\mathbf{v}\lambda\|_p = \|A\mathbf{v}\|_p \leq \|A\|_p \|\mathbf{v}\|_p.$$

Possiamo semplificare $\|\mathbf{v}\|_p \neq 0$, quindi $|\lambda| \leq \|A\|_p$ per ogni autovalore.

Data una matrice $A \in \mathbb{C}^{n \times n}$ (quindi gli autovalori esistono sempre), si chiama *spettro* l'insieme dei suoi autovalori, e *raggio spettrale* (e si indica $\rho(A)$) il valore assoluto più grande degli autovalori, $\rho(A) = \max_{\lambda \text{ autoval.}} |\lambda|$. Notate che il raggio spettrale non è per forza un autovalore: per esempio potremmo avere A con autovalori $\{-2, i, -i\}$, e quindi $\rho(A) = 2$ non è un autovalore.

In ogni caso, dalla formula qui sopra segue

$$\rho(A) \leq \|A\|_p$$

per ogni norma matriciale indotta.

Formule per le norme matriciali 1, 2, ∞ È abbastanza complicato calcolare le norme matriciali indotte usando la loro definizione: c'è da fare un massimo su un insieme infinito di vettori. . . . Per le norme 1, 2, ∞ ci sono delle formule più

semplici. Le enunciamo senza dimostrazione.

$$\begin{aligned}\|A\|_\infty &= \max_{i=1}^n \sum_{j=1}^n |A_{ij}|, \\ \|A\|_1 &= \max_{j=1}^n \sum_{i=1}^n |A_{ij}|, \\ \|A\|_2 &= \rho(A^T A)^{1/2}.\end{aligned}$$

(Qui $\rho(\cdot)$ è di nuovo il raggio spettrale.)

Esempio: calcolare queste tre norme (e anche la norma di Frobenius) sulla matrice

$$A = \begin{bmatrix} -2 & -1 \\ -2 & 1 \end{bmatrix},$$

e verificare (aiutandosi eventualmente con Matlab) che $\rho(A) \leq \|A\|_p$ per $p = 1, 2, \infty$. (La disuguaglianza è vera anche per $\|A\|_F$, ma la dimostrazione che abbiamo fatto funziona solo per norme matriciali indotte.)

Condizionamento della soluzione di sistemi lineari Prima ancora di parlare di algoritmi risolutivi, andiamo a studiare il condizionamento della soluzione di sistemi lineari, che ci dice quanto vale l'errore inerente. La soluzione di sistemi lineari è un problema che ha come “input” $A, \mathbf{b}$ e come “output” $\mathbf{x}$. Ha senso chiederci come cambia $\mathbf{x}$ se perturbiamo A oppure $\mathbf{b}$, o anche tutti e due insieme. Possiamo definire degli errori relativi su queste quantità vettoriali e matriciali rimpiazzando i valori assoluti con delle norme: $\frac{\|\tilde{\mathbf{x}} - \mathbf{x}\|}{\|\mathbf{x}\|}$, $\frac{\|\tilde{A} - A\|}{\|A\|}$, $\frac{\|\tilde{\mathbf{b}} - \mathbf{b}\|}{\|\mathbf{b}\|}$ (occhio che $\frac{\|\tilde{\mathbf{b}} - \mathbf{b}\|}{\|\mathbf{b}\|}$ non vuol dire niente, non possiamo dividere per vettori!)

Qui vediamo cosa succede quando perturbiamo il vettore dei termini noti $\mathbf{b}$.

Teorema 5.2. *Sia $A \in \mathbb{C}^{n \times n}$ una matrice invertibile, e $\mathbf{b}, \tilde{\mathbf{b}} \in \mathbb{C}^n$ due vettori, con $\mathbf{b} \neq \mathbf{0}$. Siano $\mathbf{x}$ e $\tilde{\mathbf{x}}$ le soluzioni dei due sistemi lineari $A\mathbf{x} = \mathbf{b}$, $A\tilde{\mathbf{x}} = \tilde{\mathbf{b}}$. Allora, per una qualunque norma vettoriale $\|\cdot\|_p$ si ha*

$$\frac{\|\tilde{\mathbf{x}} - \mathbf{x}\|_p}{\|\mathbf{x}\|_p} \leq \|A\|_p \|A^{-1}\|_p \frac{\|\tilde{\mathbf{b}} - \mathbf{b}\|_p}{\|\mathbf{b}\|_p}. \quad (5.2)$$

In questa disuguaglianza, nei termini $\|A\|_p$, $\|A^{-1}\|_p$ si intende che utilizzano la norma matriciale indotta dalla norma vettoriale $\|\cdot\|_p$.

Dimostrazione. Possiamo calcolare

$$\|\tilde{\mathbf{x}} - \mathbf{x}\|_p = \|A^{-1}\tilde{\mathbf{b}} - A^{-1}\mathbf{b}\|_p = \|A^{-1}(\tilde{\mathbf{b}} - \mathbf{b})\|_p \leq \|A^{-1}\|_p \|\tilde{\mathbf{b}} - \mathbf{b}\|_p,$$

utilizzando la proprietà di compatibilità della norma matriciale indotta (Teorema 5.1). Per la stessa proprietà abbiamo

$$\|\mathbf{b}\|_p = \|A\mathbf{x}\|_p \leq \|A\|_p \|\mathbf{x}\|_p.$$

Possiamo dividere membro a membro le due disuguaglianze (i versi sono quelli corretti per farlo!), ottenendo la (5.2). $\square$

Questa disuguaglianza è valida non solo al prim'ordine, ma per tutti gli $A, \mathbf{x}, \mathbf{b}$. La quantità $\kappa_p(A) = \|A\|_p \|A^{-1}\|_p$ si definisce “numero di condizionamento” (in norma- p) della matrice A (con un piccolo abuso di notazione, visto che finora abbiamo definito il condizionamento di un *problema* o di una *funzione*).

La quantità $\kappa(A)$ è sempre maggiore di 1, perché per ogni norma matriciale indotta $1 = \|I\|_p = \|AA^{-1}\|_p \leq \|A\|_p \|A^{-1}\|_p$. Una matrice si dice “ben condizionata” se questa quantità è vicina a 1, e “mal condizionata” se è molto maggiore di 1 (qualche migliaio almeno, di solito; non c'è una soglia precisa).

Non vediamo la dimostrazione, ma si può dimostrare che $\kappa(A)$ è anche il numero di condizionamento rispetto a perturbazioni della matrice A : se $\tilde{\mathbf{x}}$ e $\mathbf{x}$ sono le soluzioni rispettivamente di $\tilde{A}\tilde{\mathbf{x}} = \mathbf{b}$ e $A\mathbf{x} = \mathbf{b}$, allora

$$\frac{\|\tilde{\mathbf{x}} - \mathbf{x}\|_p}{\|\mathbf{x}\|_p} \leq \|A\|_p \|A^{-1}\|_p \frac{\|\tilde{A} - A\|_p}{\|A\|_p}.$$

Occhio al punto sopra il $\leq$: a differenza del precedente, in questo caso si tratta di un risultato che vale solo a meno di termini dell'ordine del quadrato di $\frac{\|\tilde{A} - A\|_p}{\|A\|_p}$: se $\frac{\|\tilde{A} - A\|_p}{\|A\|_p}$ è grande, questa disuguaglianza può essere ben lontana dall'essere verificata.

Esempio Con Matlab, è problematico mostrare gli errori facendo esempi di perturbazioni dell'ordine della precisione di macchina $u \approx 2.2 \times 10^{-16}$, visto che i calcoli stessi che facciamo sono affetti da un errore dello stesso ordine di grandezza. Possiamo però vedere cosa succede con perturbazioni molto più grandi.

```

1 >> A = [1 2; 3 4]
2 A =
3         1         2
4         3         4
5 >> b = [3;7];
6 >> x = A \ b
7 x =
8     1.0000e+00
9     1.0000e+00
10 >> btilde = [3.0001; 6.9999];
11 >> xtilde = A \ btilde
12 xtilde =
13     9.9970e-01
14     1.0002e+00
15 >> norm(btilde - b, inf) / norm(b, inf) % errore relativo
16 ans =
17     1.4286e-05
18 >> norm(xtilde - x, inf) / norm(x, inf)
19 ans =
20     3.0000e-04
21 >> norm(A, inf) * norm(inv(A), inf) % numero di
    condizionamento
22 ans =
23     2.1000e+01
24 >> cond(A, inf) % funzione Matlab per calcolarlo
    direttamente

```

```

25 ans =
26     2.1000e+01

```

La matrice A ha numero di condizionamento 21. Quindi l'errore relativo sulla soluzione $\mathbf{x}$ calcolata è pari ad al più 21 volte l'errore relativo sul dato in ingresso $\mathbf{b}$. Visto che questo fattore 21 è tutto sommato piccolo, diciamo che la matrice A è *ben condizionata*.

```

1  >> A = [1 2; 2.0001 4]
2  A =
3      1.0000e+00    2.0000e+00
4      2.0001e+00    4.0000e+00
5  >> b = A*[1;1]
6  b =
7      3.0000e+00
8      6.0001e+00
9  >> btilde = [2.9999; 6.0002]
10 btilde =
11     2.9999e+00
12     6.0002e+00
13 >> x = A \ b
14 x =
15     1.0000e+00
16     1.0000e+00
17 >> xtilde = A \ btilde
18 xtilde =
19     4.0000e+00
20    -5.0005e-01
21 >> norm(btilde - b, inf) / norm(b, inf)
22 ans =
23     1.6666e-05
24 >> norm(xtilde - x, inf) / norm(x, inf)
25 ans =
26     3.0000e+00
27 >> norm(A, inf) * norm(inv(A), inf)
28 ans =
29     1.8000e+05
30 >> cond(A, inf)
31 ans =
32     1.8000e+05

```

Questa nuova matrice A ha un numero di condizionamento molto più grande, 1.8×10^{-5} . Diciamo che è *mal condizionata*. L'errore relativo sul termine noto b pari a 1.66×10^{-5} produce un errore relativo sulla soluzione x pari a 3; cioè la soluzione ottenuta è completamente sbagliata. La disuguaglianza (5.2) comunque è rispettata anche in questo caso.

5.3 Condizionamento del calcolo di autovalori (**)

Un altro problema classico dell'algebra lineare è il calcolo di autovalori e autovettori. Non vediamo nel dettaglio algoritmi per farlo, perché sono molto

più complicati; ci fideremo di $\text{eig}(A)$ di Matlab. Però, almeno per studiare il problema teoricamente, è importante vedere quale è il condizionamento di questa operazione, in modo da sapere quando anche Matlab rischia di calcolare un risultato errato.

Ci limitiamo a un caso più facile, quello di *autovalori semplici*: un autovalore si dice *semplice* se la sua molteplicità geometrica e algebrica è uguale a 1, cioè, se è uno zero semplice del polinomio caratteristico $\det(\lambda I - A)$. In questo caso, esistono e sono unici (a meno di multipli) un autovalore destro $\mathbf{x}$ e uno sinistro $\mathbf{y}^*$ associati a λ ; cioè, $A\mathbf{x} = \mathbf{x}\lambda$ e $\mathbf{y}^*A = \lambda\mathbf{y}^*$. Si può dimostrare che per un autovalore semplice $\mathbf{x}$ e $\mathbf{y}^*$ non sono mai ortogonali. Il risultato di perturbazione che enunciamo dipende dal coseno dell'angolo che essi formano, cioè

$$\cos \theta = \frac{|\mathbf{y}^*\mathbf{x}|}{\|\mathbf{y}\|_2\|\mathbf{x}\|_2}.$$

Teorema 5.3 (Perturbazione di autovalori (*)). *Sia λ un autovalore semplice della matrice $A \in \mathbb{C}^{n \times n}$. Sia $\tilde{A}$ una perturbazione di A ; allora esiste un autovalore $\tilde{\lambda}$ di $\tilde{A}$ tale che*

$$|\tilde{\lambda} - \lambda| \leq \frac{1}{\cos \theta} \|\tilde{A} - A\|_2,$$

dove θ è l'angolo tra l'autovettore destro $\mathbf{x}$ e quello sinistro $\mathbf{y}^*$ associati a λ .

Occhio al punto sopra il $\leq$: anche in questo caso il risultato è valido solo al prim'ordine, cioè ignorando termini dell'ordine di $\|\tilde{A} - A\|_2^2$.

In particolare, quando A è una matrice simmetrica gli autovalori destri e sinistri coincidono, cioè $\mathbf{x} = \mathbf{y}$; quindi il coseno è uguale a 1 e il calcolo degli autovalori di una matrice simmetrica è sempre un'operazione ben condizionata.

Dimostrazione. Scriviamo $\tilde{A} - A = \varepsilon E$, dove $\|E\|_2 = 1$ e $\varepsilon = \|\tilde{A} - A\|_2$. Consideriamo la funzione $A(t) = A + tE$. È possibile dimostrare usando il teorema della funzione implicita che esistono funzioni differenziabili $\lambda(t)$, $\mathbf{x}(t)$ tali che $\lambda(t)$ è un autovalore di $A(t)$ e $\mathbf{x}(t)$ è il suo autovettore, e tali che $\lambda(0) = \lambda$, $\mathbf{x}(0) = \mathbf{x}$. Quindi, in particolare, vale la relazione

$$A(t)\mathbf{x}(t) = \mathbf{x}(t)\lambda(t).$$

Possiamo derivare entrambi i termini rispetto a t , per ottenere

$$\underbrace{\dot{A}(t)}_{=E}\mathbf{x}(t) + A(t)\dot{\mathbf{x}}(t) = \dot{\mathbf{x}}(t)\lambda(t) + \mathbf{x}(t)\dot{\lambda}(t).$$

Valutiamo in $t = 0$, e otteniamo

$$E\mathbf{x} + A\dot{\mathbf{x}}(0) = \dot{\mathbf{x}}(0)\lambda + \mathbf{x}\dot{\lambda}(0)$$

Moltiplichiamo a sinistra per $\mathbf{y}^*$, e usiamo il fatto che $\mathbf{y}^*A = \lambda\mathbf{y}^*$ per semplificare due termini della relazione

$$\mathbf{y}^*E\mathbf{x} + \mathbf{y}^*A\dot{\mathbf{x}}(0) = \lambda\mathbf{y}^*\dot{\mathbf{x}}(0) + \mathbf{y}^*\mathbf{x}\dot{\lambda}(0).$$

Quindi otteniamo

$$\dot{\lambda}(0) = \frac{1}{\mathbf{y}^*\mathbf{x}}\mathbf{y}^*E\mathbf{x}.$$

Possiamo scrivere, a meno di termini di ordine superiore in ε , lo sviluppo di Taylor

$$\tilde{\lambda} = \lambda(\varepsilon) \doteq \lambda + \varepsilon \dot{\lambda}(0) = \lambda + \varepsilon \frac{1}{\mathbf{y}^* \mathbf{x}} \mathbf{y}^* E \mathbf{x} \quad (5.3)$$

da cui

$$|\tilde{\lambda} - \lambda| \doteq \varepsilon \frac{1}{\mathbf{y}^* \mathbf{x}} \mathbf{y}^* E \mathbf{x}$$

Per concludere dimostriamo che $|\mathbf{y}^* E \mathbf{x}| \leq \|\mathbf{y}\|_2 \|\mathbf{x}\|_2$. Poiché $\mathbf{y}^* E \mathbf{x}$ è il prodotto scalare tra $\mathbf{y}$ e $E \mathbf{x}$, chiamando α l'angolo che formano abbiamo

$$|\mathbf{y}^* E \mathbf{x}| = \|\mathbf{y}\|_2 \|E \mathbf{x}\|_2 \cos \alpha \leq \|\mathbf{y}\|_2 \|E \mathbf{x}\|_2$$

Infine, poiché $\|E\|_2 = 1$, dalle proprietà delle norme matriciali abbiamo $\|E \mathbf{x}\|_2 \leq \|E\|_2 \|\mathbf{x}\|_2$. Combinando le disuguaglianze e ricordando che $\varepsilon = \|\tilde{A} - A\|_2$, otteniamo la tesi. $\square$

Nel caso di autovalori non semplici, le perturbazioni agli autovalori possono essere molto più grandi. Per esempio, consideriamo la matrice $n \times n$ che ha uni sulla sopradiagonale, ε in posizione $(n, 1)$, e zero in tutte le altre posizioni

$$A(\varepsilon) = \begin{bmatrix} 0 & 1 & & \\ & 0 & \ddots & \\ & & \ddots & 1 \\ \varepsilon & & & 0 \end{bmatrix}.$$

Se $\varepsilon = 0$, chiaramente tutti gli autovalori sono zero, perché la matrice è triangolare superiore. Per $\varepsilon \neq 0$, è possibile dimostrare che gli autovalori di $A(\varepsilon)$ sono le radici n -esime complesse di ε , che hanno tutte modulo $\varepsilon^{1/n}$. Quindi

$$|\tilde{\lambda} - \lambda| = \varepsilon^{1/n},$$

che è molto più grande di ε . Per esempio, se $\varepsilon = 10^{-16}$, $n = 8$, abbiamo $\varepsilon^{1/n} = 10^{-2}$: una perturbazione alla matrice $A(0)$ di norma 10^{-16} fa spostare gli autovalori di una distanza 10^{-2} .

5.4 Teorema dei cerchi di Gershgorin (*)

Vediamo ora un risultato teorico che ci dice in quali regioni del piano complesso si possono trovare gli autovalori di una matrice. Per introdurlo, definiamo i *cerchi di Gershgorin* di una matrice $A \in \mathbb{C}^{n \times n}$ come gli insiemi

$$K_i = \left\{ z \in \mathbb{C} : |z - A_{ii}| \leq \sum_{\substack{j=1 \\ j \neq i}}^n |A_{ij}| \right\}, \quad i = 1, 2, \dots, n.$$

Le equazioni definiscono l'interno di n cerchi, che hanno centro negli elementi diagonali di A e raggio uguale alla somma dei moduli degli elementi al di fuori della riga.

Teorema 5.4 (Teorema dei cerchi di Gershgorin). *Sia $A \in \mathbb{C}^{n \times n}$, e λ un suo autovalore. Allora λ appartiene all'unione dei cerchi K_i definiti sopra (al variare di $i = 1, 2, \dots, n$).*

Dimostrazione. Prendiamo un autovettore $\mathbf{x}$ associato a λ . Scrivendo componente per componente la relazione $A\mathbf{x} = \mathbf{x}\lambda$ otteniamo

$$\sum_{j=1}^n A_{ij}x_j = x_i\lambda, \quad i = 1, 2, \dots, n.$$

Riarrangiando termini e prendendo i moduli otteniamo

$$|\lambda - A_{ii}||x_i| = |x_i\lambda - A_{ii}x_i| = \left| \sum_{\substack{j=1 \\ j \neq i}}^n A_{ij}x_j \right| \leq \sum_{\substack{j=1 \\ j \neq i}}^n |A_{ij}||x_j|.$$

Questa uguaglianza è valida per ogni $i = 1, 2, \dots, n$. Scegliamo ora come i l'indice p (o uno degli indici) tale che il modulo $|x_p|$ sia massimo. Poiché $\mathbf{x} \neq 0$, avremo $|x_p| \neq 0$. Possiamo quindi riscrivere l'uguaglianza precedente per $i = p$ e dividere per $|x_p|$ entrambi i lati, ottenendo

$$|\lambda - A_{pp}| \leq \sum_{\substack{j=1 \\ j \neq p}}^n |A_{pj}| \frac{|x_j|}{|x_p|} \leq \sum_{\substack{j=1 \\ j \neq p}}^n |A_{pj}|.$$

L'ultima disuguaglianza segue dal fatto che $|x_p|$ è massimo. Questa catena di disuguaglianze dimostra che λ sta all'interno del cerchio K_p . Poiché λ sta all'interno di almeno un cerchio, sta anche all'interno della loro unione. $\square$

5.5 Soluzione di equazioni lineari: casi speciali

Sistemi diagonali Il caso più semplice è quello di A *diagonale*, vale a dire

$$A = \begin{bmatrix} A_{11} & & & \\ & A_{22} & & \\ & & \ddots & \\ & & & A_{nn} \end{bmatrix} \in \mathbb{R}^{n \times n}$$

(ricordiamo la convenzione che gli elementi che non scriviamo sono zeri).

In questo caso i prodotti sono semplici da fare, e $Ax = b$ implica

$$x_i = \frac{b_i}{A_{ii}}, \quad i = 1, 2, \dots, n.$$

Il costo chiaramente è di n operazioni aritmetiche.

Sistemi triangolari Un caso particolare è quello in cui la matrice è *triangolare inferiore*, cioè contiene zeri al di sopra della diagonale ($A_{ij} = 0$ se $i < j$), oppure è *triangolare superiore* ($A_{ij} = 0$ se $i > j$).

(Una matrice può essere al tempo stesso *triangolare* e *quadrata*!)

Ricordiamo che una matrice triangolare è invertibile se gli elementi sulla sua diagonale sono tutti diversi da zero. Per dimostrarlo, per esempio notiamo che una matrice è invertibile se tutti i suoi autovalori sono diversi da zero, e che gli autovalori di una matrice triangolare sono uguali agli elementi sulla diagonale.

Se la matrice è triangolare inferiore, possiamo risolvere il sistema per *sostituzione in avanti* partendo dalla prima equazione, che contiene solo la prima incognita:

$$\begin{aligned}x_1 &= \frac{b_1}{A_{11}}, \\x_2 &= \frac{b_2 - A_{21}x_1}{A_{22}}, \\&\vdots \\x_n &= \frac{b_n - A_{n1}x_1 - A_{n2}x_2 - \cdots - A_{n,n-1}x_{n-1}}{A_{nn}}.\end{aligned}$$

Notare che ad ogni passo compaiono a destra dell'uguale solo valori x_i che abbiamo già calcolato nei passi precedenti. Possiamo contare il numero di operazioni in ogni riga, ottenendo

$$1 + 3 + 5 + \cdots + (2n - 1) = n^2$$

(questa ultima uguaglianza si può dimostrare per induzione).

Diciamo che la complessità del metodo è di n^2 operazioni aritmetiche. Notare che in questo caso non ci sono funzioni sconosciute da valutare, quindi questa è davvero tutta la complessità. Inoltre queste operazioni calcolano esattamente la soluzione, quindi quello che abbiamo chiamato “errore analitico” è zero.

Spesso quando si parla di complessità ci interessa solo l'ordine di grandezza, e quindi si scrive $\mathcal{O}(n^2)$. Come in analisi, questa è una notazione che include complessità come $2n^2$, $\frac{1}{3}n^2 + 2n - 5$, ecc. Si intende che lavoriamo nel limite $n \rightarrow \infty$, quindi ignoriamo potenze *inferiori* della n che crescono più lentamente di n^2 .

Per una matrice triangolare superiore, la tecnica è analoga, ma dobbiamo partire dall'ultima equazione e risolvere “andando al contrario” (*sostituzione all'indietro*):

$$\begin{aligned}x_n &= \frac{b_n}{A_{nn}}, \\x_{n-1} &= \frac{b_{n-1} - A_{n-1,n}x_n}{A_{n-1,n-1}}, \\&\vdots \\x_1 &= \frac{b_1 - A_{1,n}x_n - A_{1,n-1}x_{n-1} - \cdots - A_{1,2}x_2}{A_{11}}.\end{aligned}$$

La complessità è di nuovo n^2 operazioni.

Se una matrice triangolare ha molti elementi uguali a zero (si dice che è *sparsa*), e sappiamo da principio questo fatto, possiamo scrivere del codice

specializzato che salta le operazioni che coinvolgono degli elementi A_{ij} che sappiamo essere zero. In questo modo si utilizzano $\mathcal{O}(2\mathbf{nnz})$ operazioni, dove $\mathbf{nnz}$ è il *numero di nonzeri* della matrice.

Stabilità della soluzione di sistemi lineari speciali Abbiamo visto che l'errore commesso nella soluzione numerica di un problema è $e_{tot} \doteq e_{an} + e_{mac}$. In questi algoritmi, la quantità γ che calcoliamo è *esattamente* uguale alla soluzione α del problema: non abbiamo processi di limite, o successioni che convergono alla soluzione α e che dobbiamo troncare. Quindi $e_{an} = 0$. Abbiamo visto ulteriormente che $e_{mac} \doteq e_{in} + e_{alg}$, e che $e_{alg} \leq \kappa_{\mathbf{x},A}\mathbf{u} + \kappa_{\mathbf{x},\mathbf{b}}\mathbf{u} \doteq 2\kappa(A)\mathbf{u}$. È possibile dimostrare (qui non lo vediamo) che gli algoritmi di soluzione di sistemi diagonali e sostituzione in avanti e all'indietro sono *stabili*, cioè e_{alg} non è mai molto più grande di e_{in} .

5.6 Eliminazione di Gauss e fattorizzazione LU

Si chiama *fattorizzazione LU* di A una decomposizione di una matrice $A \in \mathbb{C}^{n \times n}$ come prodotto $A = LU$, dove L è una matrice triangolare inferiore con elementi tutti uguali a 1 sulla diagonale, e U è una matrice triangolare superiore. Vediamo ora un metodo che permette di calcolare una fattorizzazione LU di una matrice $A \in \mathbb{C}^{n \times n}$; questo algoritmo è sostanzialmente una variante dell'eliminazione di Gauss che avete già visto ad algebra lineare.

Ripasso: l'eliminazione di Gauss Ricordiamo come funziona l'eliminazione di Gauss. Ce ne sono diverse varianti che si vedono ad algebra lineare, qui in particolare ci concentriamo su una di esse:

- Lavoriamo solo sulla matrice quadrata $A \in \mathbb{C}^{n \times n}$, *non* sulla cosiddetta “matrice estesa” $[A \quad \mathbf{b}]$;
- Puntiamo a fare combinazioni lineari delle *righe* (non colonne) per trasformare A in forma *triangolare superiore*; non vogliamo eliminare anche gli elementi sopra il pivot per trasformare A in una matrice diagonale, come talvolta si fa (eliminazione di Gauss–Jordan).
- Non facciamo (per ora) scambi di righe, e assumiamo (per ora) che tutti i pivot siano diversi da zero; in questo modo otterremo una matrice triangolare superiore con elementi diversi da zero sulla triangolare principale.

Partiamo definendo $U_1 = A$, e vogliamo applicare ripetutamente delle trasformazioni in modo da generare una sequenza di matrici $U_2, U_3, \dots, U_n$, dove l'ultima matrice U_n è triangolare superiore. Supponiamo di aver già fatto $k - 1$ passi ed essere al k -esimo. Abbiamo generato a partire da A una matrice $A_k \in \mathbb{C}^{n \times n}$ che è parzialmente in forma triangolare superiore: in ogni colonna $j < k$, gli

elementi sotto la diagonale sono nulli.

$$U_k = \begin{bmatrix} * & \dots & * & & * & & * & \dots & * \\ 0 & \ddots & * & & * & & * & \dots & * \\ \vdots & \ddots & & & & & & & \\ 0 & 0 & 0 & (U_k)_{kk} & * & \dots & * \\ 0 & 0 & 0 & (U_k)_{k+1,k} & * & \dots & * \\ 0 & 0 & 0 & \vdots & * & \dots & * \\ 0 & 0 & 0 & (U_k)_{n,k} & * & \dots & * \end{bmatrix}$$

(abbiamo chiamato $(A_k)_{ij}$ gli elementi della matrice A_k). Nel k -esimo passo, andiamo a introdurre zeri anche nella k -esima colonna, costruendo una nuova matrice

$$U_{k+1} = \begin{bmatrix} * & \dots & * & & * & & * & \dots & * \\ 0 & \ddots & * & & * & & * & \dots & * \\ \vdots & \ddots & & & & & & & \\ 0 & 0 & 0 & (U_k)_{kk} & * & \dots & * \\ 0 & 0 & 0 & 0 & * & \dots & * \\ 0 & 0 & 0 & \vdots & * & \dots & * \\ 0 & 0 & 0 & 0 & * & \dots & * \end{bmatrix}.$$

Per ottenere U_{k+1} , sottraiamo da ogni riga $i > k$ di U_k un opportuno multiplo (*moltiplicatore*) della k -esima riga; visto che vogliamo eliminare l'elemento in posizione (i, k) , questo multiplo dev'essere scelto pari a $L_{ik} = \frac{(U_k)_{ik}}{(U_k)_{kk}}$. Le prime k righe di U_k rimangono invariate.

Moltiplicatori In aggiunta all'eliminazione di Gauss, vogliamo tenere traccia dei moltiplicatori usati ad ogni passo, andando a scriverli separatamente in una seconda matrice. Al primo passo, partiamo da una matrice $L_1 = I$ (vedremo poi perché partiamo dalla matrice identità); poi, ad ogni passo di eliminazione di Gauss, andiamo a scrivere il moltiplicatore $L_{ik} = \frac{(U_k)_{ik}}{(U_k)_{kk}}$ nella posizione i, k di questa matrice. In questo modo, per esempio per una matrice 4×4 , ad ogni passo la struttura delle matrici ottenute è (indicando con $*$ gli elementi diversi

da zero)

$$\begin{aligned}
 L_1 &= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} & U_1 &= \begin{bmatrix} * & * & * & * \\ * & * & * & * \\ * & * & * & * \\ * & * & * & * \end{bmatrix}, \\
 L_2 &= \begin{bmatrix} 1 & 0 & 0 & 0 \\ * & 1 & 0 & 0 \\ * & 0 & 1 & 0 \\ * & 0 & 0 & 1 \end{bmatrix} & U_2 &= \begin{bmatrix} * & * & * & * \\ 0 & * & * & * \\ 0 & * & * & * \\ 0 & * & * & * \end{bmatrix}, \\
 L_3 &= \begin{bmatrix} 1 & 0 & 0 & 0 \\ * & 1 & 0 & 0 \\ * & * & 1 & 0 \\ * & * & 0 & 1 \end{bmatrix} & U_3 &= \begin{bmatrix} * & * & * & * \\ 0 & * & * & * \\ 0 & 0 & * & * \\ 0 & 0 & * & * \end{bmatrix}, \\
 L_4 &= \begin{bmatrix} 1 & 0 & 0 & 0 \\ * & 1 & 0 & 0 \\ * & * & 1 & 0 \\ * & * & * & 1 \end{bmatrix} & U_4 &= \begin{bmatrix} * & * & * & * \\ 0 & * & * & * \\ 0 & 0 & * & * \\ 0 & 0 & 0 & * \end{bmatrix}.
 \end{aligned}$$

Dimostreremo più avanti che ad ogni passo si ha $L_k U_k = A$, cioè il prodotto di queste due matrici è uguale alla matrice di partenza A . Questo è il motivo per cui siamo partiti con $L_1 = I$, anziché, per esempio, da $L_1 = 0$: gli uni sulla diagonale sono necessari per far valere l'uguaglianza $L_k U_k = A$. Notiamo in particolare due casi di questa uguaglianza: quando $k = 1$, abbiamo l'uguaglianza (ovvia) $I \cdot A = A$; e all'ultimo passo abbiamo $L_n U_n = A$, che è esattamente la fattorizzazione LU che stiamo cercando: $L_n = L$ è triangolare inferiore con 1 sulla diagonale, $U_n = U$ è triangolare superiore.

Esempio Vediamo innanzitutto un esempio.

```

1 >> A = rand(4, 4)
2 A =
3     0.8147     0.6324     0.9575     0.9572
4     0.9058     0.0975     0.9649     0.4854
5     0.1270     0.2785     0.1576     0.8003
6     0.9134     0.5469     0.9706     0.1419
7 >> U1 = A;
8 >> E1 = eye(4); E1(2:4, 1) = -U1(2:4, 1) / U1(1, 1)
9 E1 =
10     1.0000         0         0         0
11    -1.1118     1.0000         0         0
12    -0.1559         0     1.0000         0
13    -1.1211         0         0     1.0000
14 >> U2 = E1 * U1
15 U2 =
16     0.8147     0.6324     0.9575     0.9572
17         0    -0.6055    -0.0996    -0.5788
18         0     0.1799     0.0084     0.6511
19         0    -0.1620    -0.1029    -0.9312
20 >> E2 = eye(4); E2(3:4, 2) = -U2(3:4, 2) / U2(2, 2)
21 E2 =

```

```

22      1.0000      0      0      0
23      0      1.0000      0      0
24      0      0.2972      1.0000      0
25      0      -0.2676      0      1.0000
26 >> U3 = E2 * U2
27 U3 =
28      0.8147      0.6324      0.9575      0.9572
29      0      -0.6055      -0.0996      -0.5788
30      0      0      -0.0212      0.4791
31      0      0      -0.0762      -0.7763
32 >> E3 = eye(4); E3(4:4, 3) = -U3(4:4, 3) / U3(3, 3)
33 E3 =
34      1.0000      0      0      0
35      0      1.0000      0      0
36      0      0      1.0000      0
37      0      0      -3.5869      1.0000
38 >> U4 = E3 * U3
39 U4 =
40      0.8147      0.6324      0.9575      0.9572
41      0      -0.6055      -0.0996      -0.5788
42      0      0      -0.0212      0.4791
43      0      0      0      -2.4948

```

Basandoci sull'esempio, andiamo ora a scrivere in una funzione una versione generale dell'algoritmo. Vogliamo utilizzare due sole variabili: una che chiamiamo U e che contiene U_k ad ogni passo, e una che chiamiamo L , che inizialmente è uguale alla matrice identità, e in cui scriviamo di volta in volta i moltiplicatori usati.

```

1  function [L, U] = fattorizzazioneLU(A)
2
3  [m, n] = size(A);
4  if not(m == n)
5      error('A dev''essere quadrata');
6  end
7
8  L = eye(n); % qui scriveremo i moltiplicatori
9  U = A;      % qui scriveremo U1, U2, ... Un
10 for k = 1:n-1
11     % ad ogni passo di questo ciclo vale L*U = A
12
13     if U(k,k) == 0
14         error('Pivot nullo');
15     end
16     L(k+1:n, k) = U(k+1:n, k) / U(k, k);
17     % sappiamo che questi elementi devono diventare zero
18     U(k+1:n, k) = 0;
19     % aggiorniamo le righe di U una ad una
20     for i = k+1:n
21         U(i, k+1:n) = U(i, k+1:n) - L(i,k)*U(k,k+1:n);
22     end
23 end

```

Costo computazionale dell'eliminazione di Gauss Il grosso del costo è dato dall'aggiornamento del blocco $U_{k+1:n, k+1:n}$ ad ogni passo del ciclo for; questo richiede $2(n-1)^2$ operazioni al primo passo, $2(n-2)^2$ al secondo, $\dots 2 \cdot 1^2$ al $n-2$ -esimo passo, $2 \cdot 0^2$ al $n-1$ -esimo passo. Un'identità algebrica (che possiamo dimostrare per induzione) ci dice che

$$0^2 + 1^2 + 2^2 + \dots + (n-1)^2 = \frac{1}{6}(n-1)n(2n-1) = \frac{1}{3}n^3 + \mathcal{O}(n^2).$$

I termini che abbiamo ommesso e indicato con $\mathcal{O}(n^2)$ sono *di ordine inferiore* a $\frac{1}{3}n^3$, cioè contengono potenze più basse della n . Se teniamo traccia delle operazioni nelle altre righe (ad esempio, il calcolo degli L_{ij}) otteniamo termini di ordine inferiore. Quindi il costo della fattorizzazione LU (così come quello dell'eliminazione di Gauss) è $\frac{2}{3}n^3 + \mathcal{O}(n^2)$ operazioni aritmetiche.

Notiamo che nel codice non abbiamo mai scritto esplicitamente moltiplicazioni di matrici $E_k \cdot U_k$, ma abbiamo invece usato del codice che ne calcola il risultato usando la struttura particolare di queste matrici E_k . Se invece lo avessimo scritto come prodotto, per esempio $U_2 = E_1 \cdot U_1$, Matlab avrebbe usato ad ogni passo l'algoritmo generale per calcolare un prodotto matrice-matrice, che è molto più costoso. Solo effettuare il prodotto $U_2 = E_1 U_1$ al primo passo avrebbe richiesto $2n^3 + \mathcal{O}(n^2)$ operazioni in aritmetiche: più di quello che abbiamo speso per tutto l'algoritmo!

Soluzione di sistemi lineari tramite la fattorizzazione LU Una volta calcolata $A = LU$, abbiamo decomposto A in un prodotto di matrici più semplici, e possiamo usare questa decomposizione per risolvere sistemi lineari.

Notiamo inoltre che la matrice L è sempre invertibile (perché ha 1 sulla diagonale), e in più abbiamo $\det(A) = \det(LU) = \det(L)\det(U)$, quindi U è invertibile se e solo se A lo è. Quindi possiamo risolvere sistemi lineari con matrici L e U . Abbiamo $\mathbf{b} = A\mathbf{x} = LU\mathbf{x}$. L'algoritmo è composto di tre passi:

1. Calcolo la fattorizzazione $A = LU$.
2. Calcolo il vettore $\mathbf{y} = U\mathbf{x}$ risolvendo il sistema lineare $\mathbf{b} = L\mathbf{y}$ (sostituzione in avanti).
3. Calcolo il vettore $\mathbf{x}$ risolvendo il sistema lineare $\mathbf{y} = U\mathbf{x}$ (sostituzione all'indietro).

Il costo del primo passo è di $\frac{2}{3}n^3 + \mathcal{O}(n^2)$ operazioni aritmetiche. Il secondo e il terzo costano n^2 operazioni aritmetiche, quindi sono molto più economici e possono venire inclusi dentro il termine $\mathcal{O}(n^2)$. In totale quindi la soluzione di un sistema lineare tramite la fattorizzazione LU richiede $\frac{2}{3}n^3 + \mathcal{O}(n^2)$ operazioni aritmetiche.

Un'altra osservazione interessante è che se dobbiamo risolvere molti sistemi lineari con vettori $\mathbf{b}$ diversi ma la stessa matrice A , possiamo riutilizzare la stessa fattorizzazione LU più volte, e quindi effettuare il passo più costoso una volta sola.

Matrici elementari di Gauss Ci concentriamo ora sul dimostrare la proprietà che $L_k U_k = A$ in ogni passo $k = 1, 2, \dots, n$ della fattorizzazione. Per farlo, ci

serve introdurre una famiglia di matrici che permette di vedere l'eliminazione di Gauss come una sequenza di moltiplicazioni.

Possiamo notare che effettuare un passo di eliminazione di Gauss si può vedere come una moltiplicazione a sinistra per una matrice: $U_{k+1} = E_k U_k$. La matrice E_k è

$$E_k = \begin{bmatrix} 1 & & & & & & \\ & 1 & & & & & \\ & & \ddots & & & & \\ & & & 1 & & & \\ & & & -L_{k+1,k} & 1 & & \\ & & & -L_{k+2,k} & & 1 & \\ & & & \vdots & & & \ddots \\ & & & -L_{n,k} & & & & 1 \end{bmatrix} \quad (5.4)$$

(ricordiamo la convenzione: gli elementi che non scriviamo sono zero). Una matrice della forma (5.4) è detta *matrice elementare di Gauss*. Difatti, se calcoliamo il prodotto di E_k per un vettore generico $\mathbf{x}$, otteniamo

$$E_k \mathbf{x} = \begin{bmatrix} 1 & & & & & & \\ & 1 & & & & & \\ & & \ddots & & & & \\ & & & 1 & & & \\ & & & -L_{k+1,k} & 1 & & \\ & & & -L_{k+2,k} & & 1 & \\ & & & \vdots & & & \ddots \\ & & & -L_{n,k} & & & & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_k \\ x_{k+1} \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_k \\ x_{k+1} - L_{k+1,k}x_k \\ \vdots \\ x_n - L_{n,k}x_k \end{bmatrix}.$$

Quindi in ogni riga $i > k$ sottraiamo all'elemento presente sulla riga un multiplo dell'elemento sulla k -esima riga, con moltiplicatore $L_{i,k}$. Se immaginiamo di farlo su tutte le colonne di una matrice, questo è esattamente un passo di eliminazione di Gauss.

Possiamo anche scrivere E_k come

$$E_k = I - \ell_k \mathbf{e}_k^T,$$

dove ℓ_k è il vettore tale che

$$(\ell_k)_i = L_{i,k} = \begin{cases} 0 & i \leq k, \\ \frac{(U_k)_{ik}}{(U_k)_{kk}} & i > k, \end{cases}$$

e $\mathbf{e}_k$ è il k -esimo vettore della base canonica.

Difatti, è semplice verificare che per ogni vettore $\mathbf{x} \in \mathbb{R}^n$

$$E_k \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_k \\ x_{k+1} - L_{k+1,k}x_k \\ \vdots \\ x_n - L_{n,k}x_k \end{bmatrix},$$

quindi l'effetto della matrice E_k è proprio di sommare a ogni riga dalla $k+1$ -esima in poi un multiplo della k -esima, per la precisione alla i -esima riga viene sommata $L_{i,k}$ volte la k -esima riga, per ogni $i = k+1, \dots, n$.

Dimostriamo innanzitutto alcune proprietà di queste matrici.

Lemma 5.5 (Proprietà delle matrici elementari di Gauss). *Valgono le seguenti due proprietà:*

1. $E_k^{-1} = I + \ell_k \mathbf{e}_k^T$.
2. $E_1^{-1} E_2^{-1} \dots E_k^{-1} = I + \ell_1 \mathbf{e}_1^T + \ell_2 \mathbf{e}_2^T + \dots + \ell_k \mathbf{e}_k^T$.

Dimostrazione. 1. Basta verificare che

$$E_k(I + \ell_k \mathbf{e}_k^T) = (I - \ell_k \mathbf{e}_k^T)(I + \ell_k \mathbf{e}_k^T) = I - \ell_k \mathbf{e}_k^T + \ell_k \mathbf{e}_k^T - \underbrace{\ell_k \mathbf{e}_k^T \ell_k \mathbf{e}_k^T}_{=0} = I.$$

Difatti il prodotto scalare $\mathbf{e}_k^T \ell_k$ fa zero perché ℓ_k ha uno zero in posizione k .

2. Possiamo verificarlo per induzione: il caso $k=1$ l'abbiamo dimostrato al passo precedente. Supponendo che sia vero per un certo $k-1$, si ha

$$E_1^{-1} E_2^{-1} \dots E_k^{-1} = (I + \ell_1 \mathbf{e}_1^T + \ell_2 \mathbf{e}_2^T + \dots + \ell_{k-1} \mathbf{e}_{k-1}^T)(I + \ell_k \mathbf{e}_k^T).$$

Ora espandiamo le parentesi e utilizziamo il fatto che $\mathbf{e}_i^T \ell_k = 0$ per $i < k$, vero perché ℓ_k ha le prime componenti uguali a zero.

□

Notiamo che la matrice $E_1^{-1} E_2^{-1} \dots E_k^{-1} = I + \ell_1 \mathbf{e}_1^T + \ell_2 \mathbf{e}_2^T + \dots + \ell_k \mathbf{e}_k^T$ è esattamente la matrice che abbiamo chiamato L_{k+1} più sopra:

$$L_{k+1} = \begin{bmatrix} 1 & & & & & & & \\ & \ddots & & & & & & \\ & & \ddots & & & & & \\ & & & 1 & & & & \\ & & & & L_{k+1,k} & & & \\ & & & & & L_{k+2,k} & 0 & 1 \\ & & & & & & \ddots & \ddots \\ & & & & & & & \ddots \\ L_{n1} & \dots & & L_{n,k} & 0 & \dots & 0 & 1 \end{bmatrix}.$$

Possiamo ora enunciare il risultato promesso.

Teorema 5.6. *Sia data una matrice $A \in \mathbb{R}^{n \times n}$, supponiamo di non incontrare mai pivot $(U_k)_{kk}$ nulli durante l'eliminazione di Gauss. Allora, dopo ogni passo dell'eliminazione si ha $L_k U_k = A$. In particolare, dopo $n-1$ passi, otteniamo la fattorizzazione LU $A = LU = L_n U_n$, in cui L è una matrice triangolare inferiore con tutti 1 sulla diagonale, e U è una matrice triangolare superiore.*

Dimostrazione. All'inizio, abbiamo $L_1 U_1 = I \cdot A = A$, quindi la proprietà è vera per $k=1$. Otteniamo la matrice U_{k+1} moltiplicando A successivamente per le matrici elementari $E_1, E_2, \dots, E_k$, quindi (attenzione all'ordine dei fattori!)

$$U_{k+1} = E_k E_{k-1} \dots E_2 E_1 A.$$

Abbiamo già osservato al termine della dimostrazione del lemma che $L_{k+1} = E_1^{-1} E_2^{-1} \dots E_{k-1}^{-1} E_k^{-1}$, quindi il prodotto di queste due matrici fa A . □

Esistenza della fattorizzazione LU Cosa può andare storto durante la fattorizzazione LU? Una sola cosa: quando dobbiamo dividere per $(U_k)_{kk}$ (elemento *pivot*), questo potrebbe essere zero. Possiamo dimostrare un criterio che mostra quando questo succede.

Definiamo le *sottomatrici principali di testa* di una matrice A come $A_{1:k,1:k}$. Questa notazione (che ci è familiare da Matlab) significa prendere gli elementi della matrice A che stanno nelle righe dalla 1 alla k e nelle colonne dalla 1 alla k , ottenendo una sottomatrice quadrata $k \times k$.

Teorema 5.7 (Esistenza della fattorizzazione LU). *Data una matrice $A \in \mathbb{C}^{n \times n}$, supponiamo che le sottomatrici principali di testa $A_{1:k,1:k}$ di A , per $k = 1, \dots, n-1$, siano tutte invertibili. Allora, è possibile portare a termine l'algoritmo di fattorizzazione LU senza mai incontrare pivot nulli, e quindi esiste una fattorizzazione LU di A .*

Dimostrazione. Partiamo dall'equazione

$$A = \underbrace{\begin{bmatrix} 1 & & & & & \\ \ell_{21} & \ddots & & & & \\ \vdots & \vdots & 1 & & & \\ \vdots & \vdots & \ell_{k+1,k} & 1 & & \\ \vdots & \vdots & \ell_{k+2,k} & 0 & 1 & \\ \vdots & \vdots & \vdots & 0 & 0 & \ddots \\ \ell_{n1} & \dots & \ell_{n,k} & 0 & \dots & \dots & 1 \end{bmatrix}}_{L_{k+1} = E_1^{-1} E_2^{-1} \dots E_k^{-1}} \underbrace{\begin{bmatrix} U_{11} & U_{12} & \dots & & & & U_{1,n} \\ 0 & \ddots & \dots & \dots & \dots & \dots & \vdots \\ 0 & & U_{kk} & \dots & \dots & \dots & \vdots \\ 0 & \dots & 0 & U_{k+1,k+1} & \dots & \dots & U_{k+1,n} \\ 0 & \dots & 0 & U_{k+2,k+1} & & & \vdots \\ 0 & \dots & 0 & \vdots & & & \vdots \\ 0 & \dots & 0 & U_{n,k+1} & \dots & \dots & U_{n,n} \end{bmatrix}}_{U_{k+1}}. \quad (5.5)$$

che mostra le matrici ottenute dopo i primi k passi della fattorizzazione. Siamo quindi pronti per fare il passo $k+1$, e vogliamo controllare se il pivot $(U_{k+1})_{k+1,k+1}$ è zero.

La matrice $A_{1:k+1,1:k+1}$ è uguale al prodotto tra le prime $k+1$ righe di L_{k+1} e le prime $k+1$ colonne di U_{k+1} . In realtà, per come sono messi gli zeri nella matrice di destra, ci basta considerare le due matrici $(k+1) \times (k+1)$ ottenute prendendo le prime $k+1$ righe e $k+1$ colonne delle matrici al membro di destra, che chiameremo $\hat{L}$ e $\hat{U}$. Difatti tutti gli elementi a destra di $\hat{L}$ sono zeri, quindi quando facciamo il prodotto con gli elementi sotto $\hat{U}$ (che pure *non* sono tutti zeri!) non contano.

Per questo abbiamo

$$A_{1:k+1,1:k+1} = \hat{L}\hat{U}.$$

Questa è una fattorizzazione LU della matrice $A_{1:k+1,1:k+1}$. In particolare, abbiamo

$$\det A_{1:k+1,1:k+1} = (\det \hat{L})(\det \hat{U}) = (1 \cdot 1 \cdot \dots \cdot 1)(U_{11}U_{22} \dots U_{k+1,k+1}).$$

Quindi se $\det A_{1:k+1,1:k+1} \neq 0$, allora anche l'elemento $U_{k+1,k+1}$ è diverso da zero e possiamo effettuare il passo k -esimo dell'eliminazione di Gauss. Visto che ci sono $n-1$ passi nel metodo, mi basta guardare fino a $A_{1:n-1,1:n-1}$: la matrice A stessa potrebbe essere singolare, e quindi potremmo avere $U_{nn} = 0$, ma non importa: non ci sono passi successivi in cui dobbiamo dividere per esso. $\square$

Notiamo che quello che abbiamo dimostrato qui sopra in realtà è un “se e solo se”: se $A_{1:k+1,1:k+1}$ è la *prima* sottomatrice principale di testa non invertibile, allora possiamo effettuare i primi k passi, e $U_{11}, \dots, U_{k,k}$ sono diversi da zero, ma arrivati al passo $k+1$ si ha $U_{k+1,k+1} = 0$ e quindi l’eliminazione di Gauss fallisce.

Matrici dominanti diagonali Ci sono alcune famiglie di matrici particolari per cui si può dimostrare che esiste sempre la fattorizzazione LU. Una di esse sono le matrici *dominanti diagonali*. Una matrice $A \in \mathbb{C}^{n \times n}$ si dice *dominante diagonale per righe* se

$$|A_{ii}| > \sum_{j \neq i} |A_{ij}|, \quad i = 1, 2, \dots, n$$

vale a dire, in ogni riga il valore assoluto dell’elemento sulla diagonale è maggiore della somma di tutti gli altri.

Teorema 5.8. *Una matrice dominante diagonale è invertibile.*

Dimostrazione. Supponiamo invece (per assurdo) che $A\mathbf{x} = 0$ per un qualche vettore $\mathbf{x} \neq 0$. Allora, $\mathbf{x}$ è un autovettore con autovalore $\lambda = 0$. Per il teorema dei cerchi di Gershgorin, dovrà esistere un indice $p \in \{1, 2, \dots, n\}$ tale che $0 \in K_p$. Ma questa relazione vuol dire che

$$|A_{pp}| = |0 - A_{pp}| \leq \sum_{j \neq p} |A_{pj}|,$$

e quindi contraddice la dominanza diagonale. $\square$

Osserviamo che se A è dominante diagonale, allora lo sono anche tutte le sue sottomatrici principali di testa: le disuguaglianze diventano più forti perché stiamo omettendo dei termini. Quindi se A è dominante diagonale allora ammette fattorizzazione LU.

Analogamente, possiamo definire matrici dominanti diagonali per colonne, quando

$$|A_{jj}| > \sum_{i \neq j} |A_{ij}|, \quad j = 1, 2, \dots, n.$$

Una matrice A è dominante diagonale per colonne se A^T è dominante diagonale per righe, quindi valgono gli stessi risultati.

Esempio: la matrice

$$\begin{bmatrix} -5 & 2 & 2 \\ 1 & 3 & -1 \\ 1 & 1 & 3 \end{bmatrix}$$

è dominante diagonale per righe, ma non per colonne (valgono uguaglianze). Le sue sottomatrici principali di testa sono di nuovo pred. diag. per righe (e questa volta anche per colonne).

Stabilità e necessità del pivoting Su un computer, difficilmente gli “zeri” vengono calcolati esattamente come zero! L’eliminazione di Gauss tecnicamente fallisce solo se c’è un pivot *esattamente* uguale a zero, ma un pivot molto vicino a zero è comunque problematico. Non facciamo un’analisi dettagliata dell’errore algoritmico, ma vediamo direttamente un esempio in cui le cose vanno storte. Supponiamo di avere (per un $\varepsilon > 0$ molto piccolo) la matrice

$$A = \begin{bmatrix} \varepsilon & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & -1 \end{bmatrix}.$$

Questa matrice è ben condizionata (in tutte e tre le norme che abbiamo visto, il suo condizionamento è minore di 10), ma dopo il primo passo di eliminazione di Gauss otteniamo

$$U_2 = \begin{bmatrix} \varepsilon & 1 & 1 \\ 0 & 1 - \frac{1}{\varepsilon} & 1 - \frac{1}{\varepsilon} \\ 0 & 1 - \frac{1}{\varepsilon} & -1 - \frac{1}{\varepsilon} \end{bmatrix}.$$

Se ε è molto piccolo, $\frac{1}{\varepsilon}$ è molto grande. In particolare, se per esempio $\varepsilon = 2^{-60}$, il numero di macchina più vicino sia a $1 - \frac{1}{\varepsilon}$ che a $-1 - \frac{1}{\varepsilon}$ è $\frac{1}{\varepsilon}$, e quindi quando andiamo a scrivere quei numeri su un calcolatore la matrice che viene memorizzata è

$$\tilde{U}_2 = \begin{bmatrix} \varepsilon & 1 & 1 \\ 0 & -\frac{1}{\varepsilon} & -\frac{1}{\varepsilon} \\ 0 & -\frac{1}{\varepsilon} & -\frac{1}{\varepsilon} \end{bmatrix}.$$

In particolare, le ultime due righe sono identiche. Pertanto al passo successivo l’algoritmo di fattorizzazione LU si blocca affermando (erroneamente) che la matrice è singolare. Ma in realtà la matrice di partenza A è molto lontana dall’essere singolare, anzi, è una matrice ben condizionata e quindi non dovremmo avere problemi numerici con un algoritmo stabile. Anche in un esempio senza valori così estremi, possiamo vedere che quando A_{11} è molto più piccolo degli altri valori presenti nella matrice dobbiamo sommare un numero molto grande a tutti gli elementi della sottomatrice $A_{2:3,2:3}$, e poi successivamente dobbiamo fare una sottrazione tra due valori molto grandi e molto vicini tra loro, al passo successivo dell’eliminazione. Questo causa perdita di precisione e problemi numerici.

La soluzione per eliminare questi problemi è introdurre la possibilità di scambiare tra loro le righe durante la fattorizzazione. In particolare, ad ogni passo scambieremo le righe in modo da portare, ad ogni passo k , in posizione pivot (k, k) il numero *più grande* in valore assoluto tra quelli della sottomatrice $(U_k)_{k:n,k}$. Questo scambio lo faremo sempre, non solo quando $(U_k)_{k,k}$ è molto piccolo.

Notare che questo è il contrario rispetto a quello che facevate ad algebra lineare facendo i conti a mano: se per esempio i numeri della prima colonna erano 2, -4, -3, 1, tipicamente volevate 1 come pivot in modo da non avere denominatori. Questa volta invece l’obiettivo è diverso: vogliamo pivot più grandi possibili (in valore assoluto), quindi scegliamo -4 come pivot.

Scambiando righe, possiamo completare l’eliminazione di Gauss, esattamente come abbiamo visto nel corso di algebra lineare. Tutte le volte che la matrice A è invertibile, otterremo n pivot, quindi una matrice U triangolare superiore, con elementi non nulli sulla diagonale. Però per continuare il nostro algoritmo vogliamo interpretare questo processo di nuovo come una fattorizzazione; nelle prossime sezioni vediamo come farlo.

Matrici di permutazione La soluzione che vedremo per questo problema corrisponde essenzialmente a calcolare la fattorizzazione LU di una matrice ottenuta dalla A permutando le righe in un ordine opportuno: quindi per esempio

$$A = \begin{bmatrix} A_{1,:} \\ A_{2,:} \\ A_{3,:} \\ A_{4,:} \end{bmatrix}, \quad PA = \begin{bmatrix} A_{4,:} \\ A_{2,:} \\ A_{1,:} \\ A_{3,:} \end{bmatrix}.$$

Abbiamo indicato la matrice di destra con PA perché effettivamente si verifica che si può ottenere moltiplicando la matrice A a sinistra per una matrice $P \in \mathbb{R}^{n \times n}$: la matrice ottenuta dalla matrice identità applicando alcuni scambi di righe (gli stessi applicati in A); ad esempio,

$$P = \begin{bmatrix} \mathbf{e}_4^T \\ \mathbf{e}_2^T \\ \mathbf{e}_1^T \\ \mathbf{e}_3^T \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}.$$

Una matrice di questo tipo è detta *matrice di permutazione*, perché il suo effetto su vettori e matrici è quello di permutarne le entrate:

$$P\mathbf{x} = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} x_4 \\ x_2 \\ x_1 \\ x_3 \end{bmatrix}.$$

La stessa cosa succede quando effettuiamo il prodotto PA ; la stessa permutazione viene applicata calcolando ogni colonna separatamente, quindi l'effetto netto è quello di permutare le righe di A .

L'algoritmo che vedremo non suppone di conoscere a priori la matrice P , ma la costruisce passo per passo scambiando le righe di A .

Eliminazione di Gauss con pivoting (parziale) Partiamo da una matrice $P_1 = I$. Al primo passo dell'eliminazione di Gauss, una volta individuata la riga p dove sta l'elemento con $|A_{p1}| = \max |A_{i1}|$, scambiamo la riga p e la riga 1 in A . Questo corrisponde a rimpiazzare la matrice A con una matrice P_2A , dove P_2 è la matrice che differisce dall'identità solo per lo scambio delle righe 1 e p .

Similmente, prima di ogni passo k , individuiamo la riga dove sta il pivot $|(U_k)_{pk}| = \max |(U_k)_{k:n,k}|$, e scambiamo la riga k con la riga p nella matrice P_kA . Quella che otteniamo quindi è un'altra matrice ottenuta permutando le righe della matrice A , che possiamo quindi scrivere come $P_{k+1}A$, dove P_k è un'opportuna matrice di permutazione.

Notiamo che fino a questo punto l'algoritmo di fattorizzazione LU ha lavorato indipendentemente sulle righe da $k : n$: se queste righe fossero state in un altro ordine nella matrice A , avremmo ottenuto le stesse matrici L_k e U_k , solo con gli elementi calcolati nelle ultime righe in ordine diverso. Quindi per ottenere il risultato dei primi k passi di fattorizzazione LU su questa matrice $P_{k+1}A$, dobbiamo semplicemente prendere le matrici L_k e U_k che abbiamo già calcolato, e scambiare gli elementi calcolati nelle righe p e k . (Occhio: sulla matrice L_k , questo scambio si limita alle prime $k-1$ colonne, quelle contenenti gli elementi

che abbiamo calcolato: gli elementi uguali a 1 sulla diagonale restano al loro posto.)

A questo punto, effettuiamo normalmente un passo di eliminazione di Gauss sulle matrici ottenute, ottenendo due nuove matrici L_{k+1} e U_{k+1} . Per come abbiamo lavorato, vale ad ogni passo la relazione $P_k A = L_k U_k$.

Vediamo insieme il codice Matlab di questo algoritmo.

```

1 function [L, U, P] = fattorizzazioneLU_pivoting(A)
2 % fattorizzazione LU con pivoting
3 % restituisce matrici tali che L*U = P*A
4 [m, n] = size(A);
5 if not(m == n)
6     error('A deve essere quadrata');
7 end
8
9 L = eye(n);
10 U = A;
11 P = eye(n);
12 for k = 1:n-1
13     % ad ogni iterazione, abbiamo L*U == P*A
14
15     % calcola riga p del pivot
16     [valore, posizione] = max(abs(U(k:n,k)));
17     % converte una posizione in k:n in un indice di riga
18     p = posizione + k-1;
19     if valore == 0
20         error('matrice esattamente singolare');
21     end
22
23     % esegue gli scambi
24     L([p k], 1:k-1) = L([k p], 1:k-1);
25     U([p k], k:n) = U([k p], k:n);
26     P([p k], 1:n) = P([k p], 1:n);
27
28     % prosegue con l'eliminazione di Gauss
29     L(k+1:n, k) = U(k+1:n, k) / U(k, k);
30     U(k+1:n, k) = 0;
31     for i = k+1:n
32         U(i, k+1:n) = U(i, k+1:n) - L(i,k)*U(k,k+1:n);
33     end
34 end

```

Il secondo argomento di output della funzione `max`, che abbiamo salvato nella variabile `posizione`, restituisce un indice all'interno del vettore di $n - k + 1$ elementi $U_{k:n,n}$: se per esempio l'elemento di modulo massimo fosse il primo, `posizione = 1`, però questa posizione 1 corrisponde alla riga k della matrice U . Per convertire questa posizione in un indice di riga, dobbiamo sommare $k - 1$.

Qual è il costo computazionale di questo algoritmo? Le uniche operazioni che abbiamo aggiunto sono la ricerca del massimo e gli scambi di righe. Queste sono operazioni che richiedono tempo per essere effettuate sul computer, però non sono operazioni aritmetiche, strettamente parlando. Se misuriamo il costo in termini di numero di operazioni aritmetiche necessarie, come abbiamo fatto finora, quindi

il costo dell'eliminazione di Gauss con pivoting è sempre $\frac{2}{3}n^3 + \mathcal{O}(n^2)$ operazioni aritmetiche, come nel caso senza pivoting.

In Matlab esiste già una funzione che calcola la fattorizzazione LU con pivoting, come la nostra `fattLU`, con un nome diverso: `[L, U, P] = lu(A)`.

Soluzione di sistemi lineari tramite fattorizzazione LU con pivoting

La fattorizzazione LU con pivoting restituisce quindi tre matrici L, U, P (le prime due triangolari, la terza matrice di permutazione) tali che $LU = PA$. Possiamo usare questa fattorizzazione per risolvere un sistema lineare $A\mathbf{x} = \mathbf{b}$, in questo modo.

Il sistema $A\mathbf{x} = \mathbf{b}$ ha la stessa soluzione di $PA\mathbf{x} = P\mathbf{b}$; difatti, moltiplicare per P ha il solo effetto di riordinare le equazioni del sistema. Possiamo quindi risolvere $LU\mathbf{x} = P\mathbf{b}$, e per farlo procediamo come nel caso della fattorizzazione LU: chiamiamo $\mathbf{y} = U\mathbf{x}$, e risolviamo nell'ordine

$$L\mathbf{y} = P\mathbf{b}, \quad U\mathbf{x} = \mathbf{y}$$

per sostituzione in avanti e all'indietro rispettivamente.

La fattorizzazione LU con pivoting è l'algoritmo normalmente usato per risolvere sistemi lineari su un computer; in Matlab anche `A \ b` utilizza questo algoritmo. Questo algoritmo è più stabile ed efficiente di tutti gli altri che avete visto ad algebra lineare: quelli basati sull'inversa A^{-1} , quelli basati su varianti diverse dell'eliminazione di Gauss che comportano di eliminare elementi sopra i pivot, quelli basati su determinanti come il metodo di Cramer, ...

Errore algoritmico nella soluzione di sistemi lineari Non vediamo un'analisi completa dell'errore algoritmico della soluzione di sistemi lineari con la fattorizzazione LU, poiché è abbastanza complessa; ma si può dimostrare che l'errore algoritmico (sia con che senza pivoting) è dell'ordine di $\|L\|\|U\|\|A^{-1}\|\mathbf{u}$. In particolare, visto che l'errore inerente è invece dell'ordine di $\kappa(A)\mathbf{u} = \|A\|\|A^{-1}\|\mathbf{u}$ l'algoritmo è stabile a patto che l'algoritmo produca matrici L e U tali che $\|L\|\|U\|$ non sia molto più grande di $\|A\|$.

Il pivoting assicura che gli elementi della L siano tutti minori o uguali a 1 in valore assoluto, quindi la norma di L non è mai troppo grande. Però, anche con il pivoting, esistono matrici particolarmente sfortunate in cui $\|U\|/\|A\|$ cresce come 2^n , dove n è il numero di righe e colonne della matrice A . Nella maggior parte dei casi, però, questo non succede e l'eliminazione di Gauss con pivoting è un algoritmo stabile: cioè l'errore algoritmico e_{alg} è dello stesso ordine di grandezza di quello inerente e_{in} . L'errore analitico, di nuovo, è 0, visto che l'algoritmo calcola la soluzione esatta di $A\mathbf{x} = \mathbf{b}$, non una successione convergente ad essa.

Calcolo del determinante mediante fattorizzazione LU con pivoting

Abbiamo visto nell'introduzione del corso che usare le formule classiche dell'algebra lineare (espansione di Laplace, formule ricorsive sui minori, ...) non è un modo efficiente di calcolare il determinante: il loro costo computazionale cresce troppo velocemente. Vediamo qui invece un metodo basato sulla fattorizzazione LU per calcolarlo.

Teorema 5.9. Sia $PA = LU$ una fattorizzazione LU con pivoting della matrice $A \in \mathbb{C}^{n \times n}$. Allora,

$$\det(A) = (-1)^s U_{11}U_{22} \dots U_{nn},$$

dove s è il numero di scambi di riga effettuati durante il calcolo della fattorizzazione.

Dimostrazione. Usando il teorema di Binet, abbiamo

$$\det P \det A = \det PA = \det LU = \det L \det U. \quad (5.6)$$

Poiché L e U sono triangolari, i loro determinanti sono i prodotti degli elementi lungo la diagonale:

$$\det L = 1 \cdot 1 \cdot 1 \cdots 1 = 1; \quad \det U = U_{11}U_{22} \cdots U_{nn}.$$

La matrice di permutazione P è ottenuta applicando alla matrice identità gli scambi di riga effettuati lungo la fattorizzazione; sappiamo dall'algebra lineare che ogni scambio di riga fa cambiare il segno del determinante, quindi abbiamo

$$\det P = (-1)^s \det I = (-1)^s.$$

Possiamo quindi ricavare $\det A$ dalla (5.6), ottenendo la tesi. $\square$

Esercizio Dato un parametro $\alpha \in \mathbb{C}$, consideriamo la matrice

$$A_\alpha = \begin{bmatrix} 1 & & & & -\alpha \\ -\alpha & 1 & & & \\ & -\alpha & 1 & & \\ & & \ddots & \ddots & \\ & & & -\alpha & 1 \end{bmatrix} \in \mathbb{C}^{n \times n}.$$

1. Per quali valori di α il teorema dei cerchi di Gershgorin ci permette di concludere che la matrice è invertibile?
2. Per quali valori di α è soddisfatta la condizione di esistenza della fattorizzazione LU?
3. Sapreste trovare la fattorizzazione LU di A ?
4. Per quali valori di α la matrice è invertibile?

Soluzione:

1. I cerchi di Gershgorin hanno tutti centro 1 e raggio $|\alpha|$. Se $|\alpha| < 1$, allora 0 non appartiene all'unione dei cerchi di Gershgorin, quindi non è un autovalore della matrice A_α ; pertanto A_α è invertibile. Se invece $|\alpha| \geq 1$, allora 1 sta all'interno dell'unione dei cerchi, quindi il teorema non ci permette di concludere nulla: 0 potrebbe essere autovalore oppure no.
2. Le sottomatrici di testa fino all'ordine $n - 1$ sono tutte triangolare inferiori con 1 sulla diagonale; quindi sono invertibili per ogni valore di $\alpha \in \mathbb{C}$.
3. Eseguiamo l'eliminazione di Gauss: al primo passo, l'unico elemento da eliminare è quello in posizione $(2, 1)$, e per farlo dobbiamo sottrarre $-\alpha$

volte la riga 1 dalla riga 2 (occhio al doppio segno $-$).

$$U_1 = A; U_2 = \begin{bmatrix} 1 & 0 & 0 & \dots & 0 & -\alpha \\ 0 & 1 & 0 & \dots & 0 & -\alpha^2 \\ 0 & -\alpha & 1 & \dots & 0 & 0 \\ & & \alpha & 1 & & \\ & & & \ddots & \ddots & \\ & & & & -\alpha & 1 \end{bmatrix}$$

Continuando nello stesso modo, al passo successivo dobbiamo eliminare l'elemento in posizione $(3,2)$, di nuovo con moltiplicatore $-\alpha$, ottenendo

$$U_3 = \begin{bmatrix} 1 & 0 & 0 & \dots & 0 & -\alpha \\ 0 & 1 & 0 & \dots & 0 & -\alpha^2 \\ 0 & 0 & 1 & \dots & 0 & 0 \\ & & \alpha & 1 & & \\ & & & \ddots & \ddots & \\ & & & & -\alpha & 1 \end{bmatrix}.$$

L'unico passo leggermente diverso è l'ultimo, in cui abbiamo

$$U_{n-1} = \begin{bmatrix} 1 & 0 & 0 & \dots & 0 & -\alpha \\ 0 & 1 & 0 & \dots & 0 & -\alpha^2 \\ 0 & 0 & 1 & \dots & 0 & 0 \\ & & \ddots & \ddots & & \\ & & & 0 & 1 & -\alpha^{n-1} \\ & & & & -\alpha & 1 \end{bmatrix}, \quad U_n = \begin{bmatrix} 1 & 0 & 0 & \dots & 0 & -\alpha \\ 0 & 1 & 0 & \dots & 0 & -\alpha^2 \\ 0 & 0 & 1 & \dots & 0 & 0 \\ & & \ddots & \ddots & & \\ & & & -\alpha & 1 & -\alpha^{n-1} \\ & & & & 0 & 1 - \alpha_n \end{bmatrix}.$$

Quindi otteniamo la fattorizzazione $A = LU$ con

$$L = \begin{bmatrix} 1 & 0 & 0 & \dots & 0 & 0 \\ -\alpha & 1 & 0 & \dots & 0 & 0 \\ 0 & -\alpha & 1 & \dots & 0 & 0 \\ & & \alpha & 1 & & \\ & & & \ddots & \ddots & \\ & & & & -\alpha & 1 \end{bmatrix}, \quad U = \begin{bmatrix} 1 & 0 & 0 & \dots & 0 & -\alpha \\ 0 & 1 & 0 & \dots & 0 & -\alpha^2 \\ 0 & 0 & 1 & \dots & 0 & 0 \\ & & \ddots & \ddots & & \\ & & & 0 & 1 & -\alpha^{n-1} \\ & & & & 0 & 1 - \alpha_n \end{bmatrix}.$$

Ricordiamo che la matrice L è ottenuta a partire dalla matrice identità inserendo sotto la diagonale tutti i moltiplicatori utilizzati nell'eliminazione.

4. Dalla fattorizzazione LU calcolata, abbiamo $\det A_\alpha = \det U = 1 \cdot 1 \cdot \dots \cdot 1 \cdot (1 - \alpha^n) = 1 - \alpha^n$. Quindi il determinante si annulla se e solo se α è una radice n -esima complessa dell'unità. Notiamo che questo risultato è coerente con quanto visto sopra con il teorema dei cerchi di Gershgorin.

5.7 Metodi iterativi per sistemi lineari

Matrici sparse Spesso nelle applicazioni compaiono matrici che hanno molti elementi uguali a zero. Per esempio, alcuni metodi di soluzione di equazioni

differenziali conducono a matrici *tridiagonali*, oppure a matrici che hanno elementi diversi da zero solo lungo alcune diagonali (non per forza vicine a quella principale). In una matrice $n \times n$ di questo tipo, il numero di elementi diversi da zero (“non-zeri”) è proporzionale a n , (possiamo scriverlo come $\mathcal{O}(n)$), quindi molti meno degli n^2 elementi.

Alcune operazioni su matrici sparse si possono effettuare più velocemente. Per esempio, per calcolare il prodotto con un vettore, Av , mi basta considerare nella somma gli elementi di A diversi da zero. Il costo computazionale di questo prodotto quindi è di $2nnz$ operazioni, dove con nnz indichiamo il “numero di non-zeri” della matrice A .

In Matlab, abbiamo alcuni comandi per gestire matrici sparse. Per esempio, `sparse(A)` restituisce una copia della matrice A , memorizzata in un formato diverso che elenca i soli “non-zeri”. Similmente, `sparse(m, n)` e `speye(m)` creano, in questo formato, rispettivamente una matrice nulla e l’identità. Lavorare con questo formato comincia a diventare conveniente solo quando la A ha una densità di non-zeri molto bassa, 10% o meno: se una matrice triangolare ha semplicemente zeri nella parte triangolare inferiore, o superiore, ma a parte questo è densa, allora i non-zeri sono circa il 50%, e i costi collegati alla gestione delle matrici sparse sono maggiori dei risparmi ottenuti usando questo formato. Ve lo dico solo per conoscenza, ma non vediamo dettagli su come gestire matrici sparse in questo corso.

I metodi che abbiamo visto finora per risolvere sistemi lineari non sono l’ideale per matrici sparse, perché spesso l’eliminazione di Gauss elimina anche la sparsità, introducendo molti elementi diversi da zero. Per esempio, data una matrice della forma

$$\begin{bmatrix} * & * & * & * & * & * \\ * & * & & & & \\ * & & * & & & \\ * & & & * & & \\ * & & & & * & \\ * & & & & & * \end{bmatrix}$$

già dopo il primo passaggio otteniamo una matrice con molti più non-zeri di quella di partenza:

$$\begin{bmatrix} * & * & * & * & * & * \\ 0 & * & * & * & * & * \\ 0 & * & * & * & * & * \\ 0 & * & * & * & * & * \\ 0 & * & * & * & * & * \\ 0 & * & * & * & * & * \end{bmatrix}.$$

Esistono metodi per risolvere sistemi lineari che riescono a sfruttare il fatto che A sia una matrice sparsa, ottenendo un minore costo computazionale rispetto a $\mathcal{O}(n^3)$. Questi metodi sono molto simili a quelli che abbiamo visto nella prima sezione: producono, passo dopo passo, una successione di vettori, $\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \mathbf{x}^{(3)}, \dots, \mathbf{x}^{(3)}, \dots$ (ogni $\mathbf{x}^{(i)}$ è un vettore; mettiamo l’indice in alto per non confonderci con gli elementi). Sotto condizioni opportune (che vedremo), questa successione converge alla soluzione $\mathbf{x}$ di $A\mathbf{x} = \mathbf{b}$.

Metodi iterativi basati su splitting Descriviamo ora un modo per ottenere un metodo iterativo a partire da uno *splitting* della matrice A , cioè una scrittura

del tipo $A = M - N$, dove $M \in \mathbb{C}^{n \times n}$ è una matrice invertibile. La soluzione $\mathbf{x} \in \mathbb{C}^{n \times n}$ del sistema lineare soddisfa

$$A\mathbf{x} = \mathbf{b} \iff (M - N)\mathbf{x} = \mathbf{b} \iff M\mathbf{x} = \mathbf{b} + N\mathbf{x} \iff \mathbf{x} = M^{-1}(\mathbf{b} + N\mathbf{x}) \quad (5.7)$$

Questo suggerisce di impostare un'iterazione di punto fisso

$$\begin{cases} \mathbf{x}^{(0)} \in \mathbb{C}^n & \text{fissato;} \\ \mathbf{x}^{(k+1)} = M^{-1}(N\mathbf{x}^{(k)} + \mathbf{b}) & k = 0, 1, 2, \dots \end{cases} \quad (5.8)$$

Notiamo che questa iterazione si può implementare in modo efficiente se abbiamo un modo efficiente di risolvere il sistema lineare $M\mathbf{x}^{(k+1)} = \mathbf{b} + N\mathbf{x}^{(k)}$: per esempio, se M è triangolare possiamo usare un processo di sostituzione. Tipicamente *non* vogliamo calcolare M^{-1} , né tantomeno $M^{-1}N$, perché questo sarebbe troppo costoso.

La (5.8) è una versione “multidimensionale” del metodo di punto fisso $x_{k+1} = \Phi(x_k)$ che abbiamo già visto per equazioni non lineari. Parlando di quel metodo, avevamo visto che spesso la convergenza avviene solo quando $\mathbf{x}^{(1)}$ è sufficientemente vicino alla soluzione esatta $\mathbf{x}$. Per questi metodi, invece, la convergenza solitamente *non* dipende dal punto iniziale, come vedremo. Questa è una conseguenza della linearità.

Metodi di Jacobi e Gauss–Seidel Due metodi iterativi sono particolarmente comuni per la loro semplicità. Il *metodo di Jacobi* si ottiene scegliendo come M una matrice diagonale che ha elementi diagonali uguali a quelli di A , cioè

$$M = \begin{bmatrix} A_{11} & & & \\ & A_{22} & & \\ & & \ddots & \\ & & & A_{nn} \end{bmatrix}, \quad N = - \begin{bmatrix} 0 & A_{12} & \dots & A_{1n} \\ A_{21} & 0 & \dots & A_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ A_{n1} & A_{n2} & \dots & 0 \end{bmatrix}.$$

(Notiamo che, una volta scelta la M , la matrice $N = M - A$ è determinata univocamente.) Con questa scelta, la soluzione di sistemi lineari con M è molto veloce perché M è diagonale. Questo metodo è applicabile tutte le volte che gli elementi sulla diagonale di A sono diversi da zero, visto che ci serve che M sia invertibile.

Il *metodo di Gauss–Seidel* si ottiene scegliendo come M una matrice triangolare inferiore che ha entrate uguali a quelle di A nella parte triangolare inferiore, cioè

$$M = \begin{bmatrix} A_{11} & & & \\ A_{21} & A_{22} & & \\ A_{31} & \ddots & \ddots & \\ A_{n1} & A_{n2} & \dots & A_{nn} \end{bmatrix}, \quad N = - \begin{bmatrix} 0 & A_{12} & \dots & A_{1n} \\ & 0 & \ddots & \vdots \\ & & \ddots & A_{n-1,n} \\ & & & 0 \end{bmatrix}.$$

La soluzione di sistemi lineari con M è veloce perché M è triangolare. Anche questo metodo è applicabile se e solo se $A_{ii} \neq 0$ per ogni i .

Implementazione del metodo di Jacobi Per ottenere una formula più esplicita, partiamo scrivendo la relazione $M\mathbf{x}^{(k+1)} = \mathbf{b} + N\mathbf{x}^{(k)}$:

$$\begin{bmatrix} A_{11} & & & \\ & A_{22} & & \\ & & \ddots & \\ & & & A_{nn} \end{bmatrix} \begin{bmatrix} x_1^{(k+1)} \\ x_2^{(k+1)} \\ \vdots \\ x_n^{(k+1)} \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix} - \begin{bmatrix} 0 & A_{12} & \dots & A_{1n} \\ A_{21} & 0 & \dots & A_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ A_{n1} & A_{n2} & \dots & 0 \end{bmatrix} \begin{bmatrix} x_1^{(k)} \\ x_2^{(k)} \\ \vdots \\ x_n^{(k)} \end{bmatrix}$$

La i -esima riga corrisponde a

$$A_{ii}x_i^{(k+1)} = b_i - \sum_{\substack{j=1 \\ j \neq i}}^n A_{ij}x_j^{(k)}$$

da cui possiamo ricavare $x_i^{(k+1)}$ ottenendo

$$x_i^{(k+1)} = \frac{b_i - \sum_{\substack{j=1 \\ j \neq i}}^n A_{ij}x_j^{(k)}}{A_{ii}} = \frac{b_i - \sum_{j=1}^{i-1} A_{ij}x_j^{(k)} - \sum_{j=i+1}^n A_{ij}x_j^{(k)}}{A_{ii}}, \quad i = 1, 2, \dots, n.$$

Con queste formule possiamo calcolare uno ad uno gli elementi di $\mathbf{x}^{(k+1)}$ a partire da quelli $\mathbf{x}^{(k)}$. Questo costituisce un'iterazione del metodo.

Il costo computazionale è di $2n$ operazioni aritmetiche per ogni i , quindi in totale $2n^2$ per ogni iterazione (il numero di iterazioni che servono per ottenere convergenza non è noto a priori, anzi, il metodo potrebbe non convergere del tutto). Inoltre, se sappiamo a priori che alcuni elementi A_{ij} sono zero, possiamo restringere la somma agli elementi non-nulli; questo modifica il costo totale in $2nmz(A)$.

Implementazione del metodo di Gauss-Seidel Di nuovo, partiamo scrivendo la relazione $M\mathbf{x}^{(k+1)} = \mathbf{b} + N\mathbf{x}^{(k)}$:

$$\begin{bmatrix} A_{11} & & & \\ A_{21} & A_{22} & & \\ A_{31} & \ddots & \ddots & \\ A_{n1} & A_{n2} & \dots & A_{nn} \end{bmatrix} \begin{bmatrix} x_1^{(k+1)} \\ x_2^{(k+1)} \\ \vdots \\ x_n^{(k+1)} \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix} - \begin{bmatrix} 0 & A_{12} & \dots & A_{1n} \\ & 0 & \ddots & \vdots \\ & & \ddots & A_{n-1,n} \\ & & & 0 \end{bmatrix} \begin{bmatrix} x_1^{(k)} \\ x_2^{(k)} \\ \vdots \\ x_n^{(k)} \end{bmatrix}.$$

L'equazione corrispondente alla riga i -esima è

$$\sum_{j=1}^i A_{ij}x_j^{(k+1)} = b_i - \sum_{j=i+1}^n A_{ij}x_j^{(k)}.$$

Risolvendola per $x_i^{(k+1)}$ otteniamo

$$x_i^{(k+1)} = \frac{b_i - \sum_{j=1}^{i-1} A_{ij}x_j^{(k+1)} - \sum_{j=i+1}^n A_{ij}x_j^{(k)}}{A_{ii}}, \quad i = 1, 2, \dots, n. \quad (5.9)$$

Possiamo risolvere queste equazioni una dopo l'altra per $i = 1, 2, \dots, n$ (in quest'ordine!) per calcolare tutti gli elementi di $\mathbf{x}^{(k+1)}$. Questo corrisponde a

risolvere il sistema lineare $M\mathbf{x}^{(k+1)} = \mathbf{b} + N\mathbf{x}^{(k)}$ per sostituzione in avanti; il sistema è triangolare inferiore, quindi non ci stupisce che questo sia possibile!

La (5.9) differisce dalla corrispondente formula per il metodo di Jacobi solo per il fatto che abbiamo $x_j^{(k+1)}$ anziché $x_j^{(k)}$ nella prima sommatoria. Questo corrisponde ad usare immediatamente gli elementi più nuovi $x_j^{(k+1)}$ appena li abbiamo calcolati, invece di aspettare l'iterazione successiva del metodo. Intuitivamente, questo metodo fornisce un'approssimazione migliore della soluzione, perché usiamo approssimazioni più accurate ad ogni passo. Questa intuizione non sempre corrisponde a realtà: esistono matrici per cui il metodo di Jacobi converge più velocemente di quello di Gauss-Seidel. (Lo vedremo in laboratorio.)

Un'osservazione interessante è che il metodo di Gauss-Seidel può essere implementato in Matlab usando *una sola* variabile (vettore) $\mathbf{x}$ che contiene ad ogni passo le approssimazioni più recenti degli elementi di $\mathbf{x}$; ad ogni passo $i = 1, 2, \dots, n$ rimpiazziamo un elemento $x_i^{(k)}$ con $x_i^{(k+1)}$:

$$\mathbf{x}^{(k)} = \begin{bmatrix} x_1^{(k)} \\ x_2^{(k)} \\ \vdots \\ x_{n-1}^{(k)} \\ x_n^{(k)} \end{bmatrix} \xrightarrow{i=1} \begin{bmatrix} x_1^{(k+1)} \\ x_2^{(k)} \\ \vdots \\ x_{n-1}^{(k)} \\ x_n^{(k)} \end{bmatrix} \xrightarrow{i=2} \begin{bmatrix} x_1^{(k+1)} \\ x_2^{(k+1)} \\ \vdots \\ x_{n-1}^{(k)} \\ x_n^{(k)} \end{bmatrix} \xrightarrow{i=3} \dots \xrightarrow{i=n-1} \begin{bmatrix} x_1^{(k+1)} \\ x_2^{(k)} \\ x_3^{(k)} \\ \vdots \\ x_{n-1}^{(k+1)} \\ x_n^{(k)} \end{bmatrix} \xrightarrow{i=n} \begin{bmatrix} x_1^{(k+1)} \\ x_2^{(k)} \\ x_3^{(k)} \\ \vdots \\ x_{n-1}^{(k+1)} \\ x_n^{(k+1)} \end{bmatrix} = \mathbf{x}^{(k+1)}.$$

Lavorando in questo modo, in ogni passo abbiamo a disposizione tutti gli elementi che ci servono per calcolare l'elemento successivo.

Lo stesso trucco non funziona per il metodo di Jacobi: non possiamo cancellare $x_1^{(k)}$ rimpiazzandolo con $x_1^{(k+1)}$, perché quell'elemento ci serve anche successivamente quando calcoliamo $x_2^{(k+1)}, \dots, x_n^{(k+1)}$.

Convergenza dei metodi iterativi Dimostriamo ora dei risultati che ci dicono quando un metodo iterativo converge. Partiamo introducendo una definizione. Diciamo che il metodo iterativo è *convergente* se per ogni scelta del punto iniziale $\mathbf{x}^{(0)} \in \mathbb{C}^n$ la successione generata dal metodo converge alla soluzione $\mathbf{x}$ del sistema lineare.

Per studiare la convergenza, definiamo $\mathbf{e}^{(k)} = \mathbf{x}^{(k)} - \mathbf{x}$ l'errore (assoluto) al passo k . Usando la (5.7) e la (5.8) abbiamo per ogni $k = 1, 2, \dots$

$$\mathbf{e}^{(k+1)} = \mathbf{x}^{(k+1)} - \mathbf{x} = M^{-1}(N\mathbf{x}^{(k)} + \mathbf{b}) - M^{-1}(N\mathbf{x} + \mathbf{b}) = M^{-1}(N\mathbf{x}^{(k)} - N\mathbf{x}) = M^{-1}N\mathbf{e}^{(k)}. \quad (5.10)$$

Definiamo la *matrice di iterazione* del metodo come $H = M^{-1}N$.

Teorema 5.10. *Il metodo iterativo (5.8) è convergente se e solo se $\rho(H) < 1$.*

Dimostrazione. In questo corso dimostriamo il teorema solo nel caso particolare in cui H è diagonalizzabile (ma in realtà è vero sempre).

Usando ripetutamente la (5.10) si ha

$$\begin{aligned}\mathbf{e}^{(1)} &= H\mathbf{e}^{(0)}, \\ \mathbf{e}^{(2)} &= H\mathbf{e}^{(1)} = H^2\mathbf{e}^{(0)}, \\ &\vdots \\ \mathbf{e}^{(k)} &= H^k\mathbf{e}^{(0)}.\end{aligned}$$

Supponiamo che la matrice H sia diagonalizzabile, cioè che esistano una matrice V invertibile e D diagonale tali che $H = VDV^{-1}$; sappiamo dall'algebra lineare che gli elementi diagonali di D sono proprio gli autovalori di H . Possiamo scrivere

$$H^k = \underbrace{(VDV^{-1})(VDV^{-1}) \dots (VDV^{-1})}_{k \text{ volte}} = VD^kV^{-1},$$

semplificando le coppie $V^{-1}V = I$ che compaiono. Se $\rho(H) < 1$, allora $|D_{ii}| < 1$ per ogni $i = 1, 2, \dots, n$, e quindi $\lim_{k \rightarrow \infty} D^k = 0$. Allora anche

$$\lim_{k \rightarrow \infty} \mathbf{e}^{(k)} = \lim_{k \rightarrow \infty} H^k \mathbf{e}^{(0)} = \lim_{k \rightarrow \infty} VD^kV^{-1}\mathbf{e}^{(0)} = 0.$$

Questo conclude (con l'ipotesi aggiuntiva che k sia diagonalizzabile) la dimostrazione che se $\rho(H) < 1$ allora il metodo è convergente. Visto che questo è un “se e solo se”, vogliamo però dimostrare anche l'implicazione opposta: se $\rho(H) \geq 1$, allora il metodo non converge (per ogni scelta del punto iniziale $\mathbf{x}^{(0)}$). Per fare questo, ci basta dimostrare che se H ha un autovalore e un autovettore $H\mathbf{v} = \mathbf{v}\lambda$, con $|\lambda| \geq 1$, allora esiste *almeno una* scelta di $\mathbf{x}^{(0)}$ per cui l'errore $\mathbf{e}^{(k)}$ non converge a zero. Prendiamo $\mathbf{x}^{(0)} = \mathbf{x} + \mathbf{v}$. Allora abbiamo $\mathbf{e}^{(0)} = \mathbf{x}^{(0)} - \mathbf{x} = \mathbf{v}$, e

$$\begin{aligned}\mathbf{e}^{(1)} &= H\mathbf{e}^{(0)} = H\mathbf{v} = \mathbf{v}\lambda, \\ \mathbf{e}^{(2)} &= H\mathbf{e}^{(1)} = H\mathbf{v}\lambda = \mathbf{v}\lambda^2, \\ &\vdots \\ \mathbf{e}^{(k)} &= H\mathbf{e}^{(k-1)} = \mathbf{v}\lambda^k.\end{aligned}$$

(Ogni volta che facciamo un prodotto $H\mathbf{v}$ otteniamo il vettore $\mathbf{v}\lambda$.)

Se $|\lambda| \geq 1$ (e $\mathbf{v} \neq 0$, visto che $\mathbf{v}$ è un autovettore), allora la quantità $\mathbf{v}\lambda^k$ non converge a zero, che era quello che volevamo dimostrare. $\square$

Visto che $\rho(H) \leq \|H\|_p$ per ogni norma matriciale indotta, abbiamo anche il seguente risultato.

Corollario 5.11. *Se $\|H\|_p \leq 1$ per una qualunque norma matriciale indotta, allora il metodo iterativo (5.8) è convergente.*

Questo *non* è un “se e solo se”! In particolare, possiamo anche avere situazioni in cui alcune norme matriciali indotte sono minori di 1 e altre maggiori di 1.

È possibile dimostrare anche che per ogni norma vettoriale vale

$$\lim_{k \rightarrow \infty} \frac{\|\mathbf{e}^{(k+1)}\|}{\|\mathbf{e}^{(k)}\|} \leq \rho(H),$$

con uguaglianza per quasi tutti i vettori di partenza $\mathbf{x}^{(0)}$. Pertanto il metodo converge linearmente. La convergenza è tanto più veloce (pendenza della retta in scala logaritmica più vicina a $-\infty$) quanto più $\rho(H)$ è piccolo.

Criteri di arresto per metodi iterativi per sistemi lineari Abbiamo due scelte naturali come criteri di arresto per il metodo:

1. Ci fermiamo quando $\|\mathbf{x}^{(k)} - \mathbf{x}^{(k+1)}\| \leq \varepsilon$, per una certa norma e un valore $\varepsilon > 0$ fissato.
2. Ci fermiamo quando $\|A\mathbf{x}^{(k+1)} - \mathbf{b}\| \leq \varepsilon$.

Il primo criterio è facile da implementare, ma (esattamente come abbiamo visto per il metodo del punto fisso nel caso scalare), quando la convergenza è lenta, $\|\mathbf{x}^{(k)} - \mathbf{x}^{(k+1)}\|$ può essere molto più piccolo di $\|\mathbf{x}^{(k+1)} - \mathbf{x}\|$, quindi rischiamo di sottostimare l'errore.

Residuo e stabilità all'indietro (*) Se $\mathbf{x}$ è la soluzione esatta del sistema lineare $A\mathbf{x} = \mathbf{b}$, allora chiaramente $\|A\mathbf{x} - \mathbf{b}\| = 0$. Ma se per un vettore $\tilde{\mathbf{x}} \in \mathbb{C}^n$ il residuo $\mathbf{r} = A\tilde{\mathbf{x}} - \mathbf{b}$ è piccolo, questo implica che $\tilde{\mathbf{x}}$ e $\mathbf{x}$ sono vicini? Intuitivamente sì, ma vediamo un risultato quantitativo che dice cosa succede.

Teorema 5.12 (Stima del residuo). *Siano $A \in \mathbb{C}^{n \times n}$, $\mathbf{b} \in \mathbb{C}^n$. Sia $\mathbf{x}$ la soluzione del sistema lineare $A\mathbf{x} = \mathbf{b}$, sia $\tilde{\mathbf{x}} \in \mathbb{R}^n$ un altro vettore, e sia $\mathbf{r} = A\tilde{\mathbf{x}} - \mathbf{b}$. Allora,*

$$\frac{\|\tilde{\mathbf{x}} - \mathbf{x}\|}{\|\mathbf{x}\|} \leq \kappa(A) \frac{\|\mathbf{r}\|}{\|\mathbf{b}\|}.$$

Dimostrazione. Abbiamo $A\tilde{\mathbf{x}} = \mathbf{b} + \mathbf{r}$. Allora, $\tilde{\mathbf{x}}$ è la soluzione del sistema lineare $A\tilde{\mathbf{x}} = \hat{\mathbf{b}}$, in cui abbiamo perturbato il termine noto in $\hat{\mathbf{b}} = \mathbf{b} + \mathbf{r}$. Pertanto possiamo applicare la disuguaglianza (5.2), che ci dà la tesi. $\square$

Questo risultato non si applica solo all'iterata $\mathbf{x}^{(k+1)}$ prodotta da un metodo iterativo, ma a una qualunque soluzione approssimata di un sistema lineare, non importa come l'abbiamo ottenuta. È quindi particolarmente interessante perché ci permette di ottenere una stima esplicita sull'errore relativo nella $\tilde{\mathbf{x}}$ calcolata, sulla base di quantità calcolabili al computer.

Convergenza di Jacobi e Gauss-Seidel per matrici A dominanti diagonali

Teorema 5.13. *Se A è dominante diagonale, i metodi di Jacobi e Gauss-Seidel sono applicabili e convergenti.*

Dimostrazione. Innanzitutto notiamo che i metodi sono applicabili; difatti $A_{ii} \neq 0$:

$$|A_{ii}| > \sum_{\substack{j=1 \\ j \neq i}}^n |A_{ij}| \geq 0, \quad i = 1, 2, \dots, n.$$

Vogliamo dimostrare che $\rho(H) < 1$, in modo da avere la convergenza. Scriviamo il polinomio caratteristico

$$\det(H - \lambda I) = \det(M^{-1}(N - \lambda M)) = \det M^{-1} \det(N - \lambda M).$$

Poiché $\det M^{-1} \neq 0$, il polinomio caratteristico si annulla solo quando $\det(N - \lambda M) = 0$, cioè $N - \lambda M$ è singolare. Quindi ci basta mostrare che $N - \lambda M$ è invertibile quando $|\lambda| \geq 1$.

La matrice $N - \lambda M$ ha elementi

$$N - \lambda M = - \begin{bmatrix} \lambda A_{11} & A_{12} & \dots & A_{1n} \\ A_{21} & \lambda A_{22} & \dots & A_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ A_{n1} & A_{n2} & \dots & \lambda A_{nn} \end{bmatrix}$$

nel caso del metodo di Jacobi (rispetto a $-A$, la diagonale è moltiplicata per λ), e

$$N - \lambda M = - \begin{bmatrix} \lambda A_{11} & A_{12} & \dots & A_{1n} \\ \lambda A_{21} & \lambda A_{22} & \dots & A_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ \lambda A_{n1} & \lambda A_{n2} & \dots & \lambda A_{nn} \end{bmatrix}$$

nel caso del metodo di Gauss-Seidel (la parte triangolare inferiore è moltiplicata per λ). In entrambi i casi, questa matrice è predominante diagonale: difatti, la predominanza diagonale di A assicura che

$$|\lambda A_{ii}| = |\lambda| |A_{ii}| > \sum_{j \neq i} |\lambda| |A_{ij}|$$

e il membro di destra (quando $|\lambda| \geq 1$) è più grande del termine corrispondente agli elementi fuori dalla diagonale di $N - \lambda M$. $\square$

5.8 Sistemi di equazioni non lineari

Introduzione Facciamo qualche accenno anche a metodi per risolvere sistemi di equazioni *non* lineari: data una $F : D \subseteq \mathbb{R}^n \rightarrow \mathbb{R}^n$, come possiamo trovare una soluzione del sistema?

Ad esempio, supponiamo di voler risolvere $F(\mathbf{x}) = 0$, dove

$$F \left(\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \right) = \begin{bmatrix} 2x_1 + \cos(x_2) \\ \sin(x_1) + 2x_2 - \pi \end{bmatrix}. \quad (5.11)$$

Questo è un sistema (*non* lineare) di due equazioni in due incognite; una soluzione del sistema per esempio è $(x_1, x_2) = (0, \pi/2)$.

Un sistema di questo tipo è un problema molto più complicato che non un sistema lineare; tanto per cominciare, in generale si possono avere zero soluzioni, una, più di una, o anche infinite, e non c'è un criterio generale per capire quando questi casi si verificano.

Non c'è un modo facile di estendere a sistemi di più equazioni il metodo di bisezione, più che altro perché non abbiamo un analogo facile del teorema di Weierstrass in più dimensioni: anche se $F(\mathbf{a})$ ha tutte le componenti < 0 e $F(\mathbf{b})$ ha tutte le componenti > 0 , non è detto che ci sia una soluzione simultanea di tutte le equazioni né nella retta che congiunge (a_1, a_2) e (b_1, b_2) né nel “rettangolo” che li comprende. I metodi di punto fisso invece si riescono a generalizzare facilmente, come abbiamo già visto: possiamo calcolare la successione $\mathbf{x}_{k+1} = \Phi(\mathbf{x}_k)$ anche per una funzione in più variabili $\Phi : \mathbb{R}^n \rightarrow \mathbb{R}^n$. In particolare ci concentriamo su un metodo del punto fisso particolarmente efficiente, il metodo di Newton.

Metodo di Newton multivariato Possiamo definire, generalizzando la derivata prima di una funzione, la *matrice Jacobiana*

$$J_F(\mathbf{x}) = \begin{bmatrix} \frac{\partial F_1}{\partial x_1}(x) & \frac{\partial F_1}{\partial x_2}(x) & \dots & \frac{\partial F_1}{\partial x_n}(x) \\ \frac{\partial F_2}{\partial x_1}(x) & \frac{\partial F_2}{\partial x_2}(x) & \dots & \frac{\partial F_2}{\partial x_n}(x) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial F_n}{\partial x_1}(x) & \frac{\partial F_n}{\partial x_2}(x) & \dots & \frac{\partial F_n}{\partial x_n}(x) \end{bmatrix} \in \mathbb{R}^{n \times n}$$

Per evitare di confondere J_F con la sua trasposta, possiamo usare questa tecnica mnemonica per ricordarci la posizione degli indici: le diverse componenti di $F(\mathbf{x})$ sono elementi di un vettore colonna, e anche nello Jacobiano vediamo che le diverse componenti di ogni derivata parziale stanno nella stessa colonna.

Per esempio per la (5.11) abbiamo

$$J_F(\mathbf{x}) = \begin{bmatrix} 2 & -\sin(x_2) \\ \cos(x_1) & 2 \end{bmatrix}.$$

Per funzioni sufficientemente regolari vale una generalizzazione dello sviluppo di Taylor:

$$F(\mathbf{x} + \mathbf{h}) = F(\mathbf{x}) + J_F(\mathbf{x})\mathbf{h} + \mathcal{O}(\|\mathbf{h}\|^2), \quad (5.12)$$

con $\mathbf{x}, \mathbf{h} \in \mathbb{R}^n$, e un termine di resto che abbiamo indicato con $\mathcal{O}(\|\mathbf{h}\|^2)$ e che va a zero ogni volta che il vettore $\mathbf{h}$ tende a zero. Notiamo in particolare che nel membro di destra compare un prodotto matrice-vettore $J_F(\mathbf{x})\mathbf{h}$. Ragionando come nel metodo di Newton in una sola variabile, possiamo trascurare il termine di resto $\mathcal{O}(\|\mathbf{h}\|^2)$ e cercare una nuova approssimazione $\mathbf{x}_{k+1} = \mathbf{x}_k + \mathbf{h}$ imponendo che

$$\mathbf{0} = F(\mathbf{x}_k + \mathbf{h}) = F(\mathbf{x}_k) + J_F(\mathbf{x}_k)\mathbf{h}.$$

Questo produce il metodo

$$\begin{cases} \mathbf{x}^{(0)} \in \mathbb{R}^n & \text{assegnato} \\ \mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - (J_F(\mathbf{x}^{(k)}))^{-1} F(\mathbf{x}^{(k)}), & k = 0, 1, 2, \dots \end{cases}$$

Si possono dimostrare (noi qui non lo facciamo) risultati analoghi a quelli del caso scalare: su funzioni sufficientemente regolari, il metodo converge a patto di prendere $\mathbf{x}^{(0)}$ sufficientemente vicino a una soluzione $\mathbf{x}$, e la convergenza è quadratica se la matrice $J_F(\mathbf{x})$ è invertibile.

Ad ogni passo, dobbiamo risolvere un sistema lineare con matrice $J_F(\mathbf{x}^{(k)})$. Nella pratica, possiamo evitare di calcolare esplicitamente la matrice inversa: calcoliamo la matrice $J_F(\mathbf{x}^{(k)})$, e poi risolviamo il sistema lineare

$$J_F(\mathbf{x}^{(k)})\mathbf{h} = F(\mathbf{x}^{(k)}), \quad \mathbf{h} = \mathbf{x}^{(k)} - \mathbf{x}^{(k+1)}$$

con una fattorizzazione LU o altri metodi. Abbiamo visto, difatti, che quando dobbiamo effettuare il calcolo $\mathbf{x} = A^{-1}\mathbf{b}$, calcolare esplicitamente l'inversa A^{-1} non è una buona idea, visto che costa di più e ha proprietà di stabilità peggiori rispetto a risolvere il sistema lineare con la fattorizzazione LU.

I passi del metodo di Newton multivariato quindi sono, nell'ordine

1. calcolare $F(\mathbf{x}^{(k)})$;

2. calcolare $J_F(\mathbf{x}^{(k)})$;
3. risolvere il sistema lineare $J_F(\mathbf{x}^{(k)})\mathbf{h} = F(\mathbf{x}^{(k)})$ per trovare $\mathbf{h}$;
4. calcolare $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \mathbf{h}$.

È impossibile dire qualcosa in generale su come si confrontano i costi dei vari passi senza sapere quanto costa calcolare F e J_F . In molti casi, però, calcolare gli n elementi di F e gli n^2 elementi di J_F costa in tutto $\mathcal{O}(n^2)$ operazioni aritmetiche. Se per risolvere il sistema lineare utilizziamo una fattorizzazione LU, il costo del terzo passo è invece $\mathcal{O}(n^3)$: quindi spesso questo passo risulta essere quello più costoso. Per questo, spesso vengono utilizzate varianti del metodo di Newton che cercano di ridurre il costo di questo passo. Una di esse è il metodo delle corde, in cui al posto della matrice $J_F(\mathbf{x}^{(k)})$ viene utilizzata una matrice fissata A , per esempio lo Jacobiano calcolato al primo passo $A = J_F(\mathbf{x}^{(0)})$. In questo modo, è possibile calcolare una volta sola la fattorizzazione LU della matrice A (con il costo di $\mathcal{O}(n^3)$ da “pagare” una volta all’inizio del metodo), e poi riutilizzarla ad ogni passo per risolvere il sistema lineare (con il costo di $\mathcal{O}(n^2)$ per ogni passo).

Una strategia ibrida Un altro metodo utilizzato abbastanza spesso cerca di combinare i benefici del metodo delle corde e di quello di Newton. L’idea è che la matrice J_F viene ricalcolata solo in alcune iterazioni del metodo; in altre, viene riutilizzata la matrice calcolata ai passi precedenti.

Nel dettaglio, per ogni $k = 0, 1, 2, \dots$:

1. Calcolo $F(\mathbf{x}^{(k)})$;
2. Faccio una di queste due cose:
 - (a) valuto $A = J_F(\mathbf{x}^{(k)})$ e calcolo una sua fattorizzazione $PA = LU$; oppure
 - (b) prendo le matrici A, L, U, P calcolate in un passo precedente del metodo.
3. Utilizzo la fattorizzazione per risolvere il sistema lineare $A\mathbf{h} = F(\mathbf{x}^{(k)})$;
4. Calcolo $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \mathbf{h}$.

Per esempio, posso fissare un intero $m > 1$ e scegliere di fare il passo (2a) ogni m passi: al passo 1, $m + 1$, $2m + 1$, $\dots$.

Con questa strategia, riduciamo il costo computazionale per passo: possiamo vedere che il costo è $\mathcal{O}(n^3)$ nei passi in cui ricalcoliamo lo Jacobiano, ma solo $\mathcal{O}(n^2)$ nei passi in cui riutilizziamo quello precedente.

Come nel caso scalare, la convergenza è garantita solo se partiamo all’interno di un certo intorno della soluzione (che può essere anche molto piccolo!).

La convergenza di questo metodo è quadratica, sotto le stesse condizioni del metodo di Newton. Nella pratica, il grafico dell’errore si comporta come una spezzata: nelle iterazioni in cui teniamo la matrice A costante osserviamo dei segmenti di retta, visto che se A è costante si ha convergenza lineare; ma nelle iterazioni in cui aggiorniamo la matrice A la pendenza di questa retta cambia e diventa più ripida, come ci aspettiamo da un metodo a convergenza quadratica. Un esempio di questo comportamento è raffigurato in Figura 5.1.

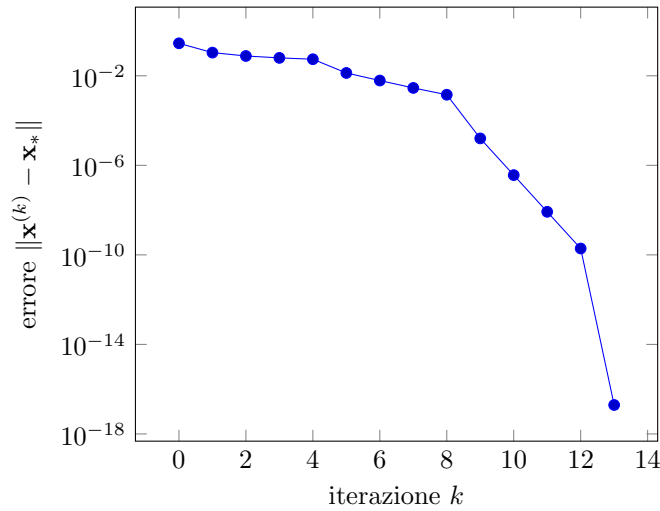


Figura 5.1: Convergenza della variante del metodo di Newton in un esempio con $m = 4$.

5.9 Sistemi lineari sovradeterminati (minimi quadrati)

Supponiamo di avere un sistema del tipo

$$A\mathbf{x} = \mathbf{b}, \quad A \in \mathbb{R}^{m \times n}, \mathbf{b} \in \mathbb{R}^m, \mathbf{x} \in \mathbb{R}^n$$

in cui A non è quadrata, bensì alta e stretta ($m > n$).

In un sistema di questo tipo, la soluzione spesso non esiste: abbiamo più equazioni che incognite, ed esse tipicamente non sono soddisfatte contemporaneamente.

Esempio Consideriamo il sistema di tre equazioni

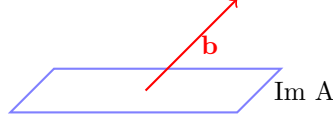
$$\begin{cases} x_1 + x_2 = 2, \\ x_1 - x_2 = 1, \\ x_2 = 1, \end{cases} \quad \text{quindi } A = \begin{bmatrix} 1 & 1 \\ 1 & -1 \\ 0 & 1 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 2 \\ 1 \\ 1 \end{bmatrix}. \quad (5.13)$$

Visto che ho più equazioni che incognite, non è garantito che esista un vettore $\mathbf{x}$ che soddisfa tutte e tre le equazioni. Per esempio, il vettore che soddisfa le prime due equazioni è $\mathbf{x}_1 = \begin{bmatrix} 1.5 \\ 0.5 \end{bmatrix}$, ma la terza equazione non è soddisfatta, e si ha $A\mathbf{x}_1 - \mathbf{b} = \begin{bmatrix} 0 \\ 0 \\ -0.5 \end{bmatrix}$.

Similmente, se prendiamo il vettore $\mathbf{x}_2$ che soddisfa la seconda e la terza equazione, abbiamo $\mathbf{x}_2 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$; questa volta la prima equazione non è soddisfatta e si ha $A\mathbf{x}_2 - \mathbf{b} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$.

In un certo senso però $\mathbf{x}_1$ è “meno sbagliato” di $\mathbf{x}_2$, visto che $A\mathbf{x}_1$ è più vicino a $\mathbf{b}$ che non $A\mathbf{x}_2$: difatti $\|A\mathbf{x}_1 - \mathbf{b}\| = 0.5$, $\|A\mathbf{x}_2 - \mathbf{b}\| = 1$, per tutte e tre le norme che abbiamo visto. Da questo punto di vista, ha senso chiedersi qual è il vettore $\mathbf{x}$ per cui $A\mathbf{x}$ è più vicino a $\mathbf{b}$.

Interpretazione geometrica Dall'algebra lineare, sappiamo che il vettore $A\mathbf{x}$ è una combinazione lineare delle colonne di A , e al variare del vettore $\mathbf{x}$ queste combinazioni lineari stanno tutte all'interno dell'immagine $\text{Im } A$, che è un sottospazio di $\mathbb{R}^m$.



Problema dei minimi quadrati Possiamo quindi risolvere questo problema: di calcolare la combinazione lineare delle colonne di A , cioè il vettore $A\mathbf{x}$, che va più vicino al vettore $\mathbf{b}$:

$$\min_{\mathbf{x} \in \mathbb{R}^n} \|A\mathbf{x} - \mathbf{b}\|. \quad (5.14)$$

In altre parole, stiamo cercando il vettore $\mathbf{x}$ che rende più piccola possibile la norma del *residuo* $\mathbf{r}(\mathbf{x}) = A\mathbf{x} - \mathbf{b}$, cioè la differenza tra il termine di sinistra e quello di destra delle equazioni che stiamo considerando. Attenzione: in questo problema, anche quando lo calcoliamo nel punto di minimo $\mathbf{x}$, il residuo non vale $\mathbf{0}$, perché le equazioni non ammettono una soluzione comune!

Vogliamo concentrarci sul risolvere il problema (5.14) nel caso della norma-2; difatti, vedremo che in questo caso ha una soluzione particolarmente semplice; non succede altrettanto per le altre norme. Minimizzare la norma-2 di $\mathbf{r}(\mathbf{x})$ equivale a minimizzare il suo quadrato $\sum r_i^2 = \mathbf{r}(\mathbf{x})^T \mathbf{r}(\mathbf{x})$, ovvero

$$\min_{\mathbf{x} \in \mathbb{R}^n} \|\mathbf{r}(\mathbf{x})\|_2^2 = \min_{\mathbf{x} \in \mathbb{R}^n} \|A\mathbf{x} - \mathbf{b}\|_2^2. \quad (5.15)$$

Il problema (5.15) è detto *problema dei minimi quadrati*, visto che se espandiamo $\|\mathbf{r}(\mathbf{x})\|_2^2 = \mathbf{r}(\mathbf{x})^T \mathbf{r}(\mathbf{x}) = \sum r_i^2$ otteniamo la somma dei quadrati delle componenti di $\mathbf{r}(\mathbf{x})$. Per esempio, nell'esempio (5.13) si ha la somma di quadrati

$$\min_{\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \in \mathbb{R}^2} (x_1 + x_2 - 2)^2 + (x_1 - x_2 - 1)^2 + (x_2 - 1)^2.$$

Equazioni normali Dimostriamo il seguente risultato.

Teorema 5.14. Sia $\mathbf{x} \in \mathbb{R}^n$ un vettore tale che $A^T \mathbf{r}(\mathbf{x}) = A^T (A\mathbf{x} - \mathbf{b}) = \mathbf{0}$. Allora, $\mathbf{x}$ risolve il problema di minimo (5.15).

Dimostrazione. Prendiamo un qualunque vettore diverso da $\mathbf{x}$: possiamo scriverlo come $\mathbf{x} + \mathbf{z}$, con $\mathbf{z} \in \mathbb{R}^n$, $\mathbf{z} \neq \mathbf{0}$; il suo residuo vale

$$\mathbf{r}(\mathbf{x} + \mathbf{z}) = A(\mathbf{x} + \mathbf{z}) - \mathbf{b} = \underbrace{A\mathbf{x} - \mathbf{b}}_{=\mathbf{r}} + \underbrace{A\mathbf{z}}_{=\mathbf{s}} = \mathbf{r} + \mathbf{s}.$$

(Abbiamo usato la sola lettera $\mathbf{r}$ per indicare il residuo $\mathbf{r}(\mathbf{x})$, per avere una notazione più semplice.)

Vogliamo dimostrare che la norma di questo residuo è maggiore o uguale a quella di $\mathbf{r}$; questo ci permette di concludere che $\mathbf{x}$ è la soluzione del problema. Il quadrato di questa norma vale

$$\|\mathbf{r}(\mathbf{x} + \mathbf{z})\|_2^2 = \|A(\mathbf{x} + \mathbf{z}) - \mathbf{b}\|_2^2 = (\mathbf{r} + \mathbf{s})^T (\mathbf{r} + \mathbf{s}) = \mathbf{s}^T \mathbf{s} + \mathbf{s}^T \mathbf{r} + \mathbf{r}^T \mathbf{s} + \mathbf{r}^T \mathbf{r}.$$

Notiamo però che il prodotto scalare $\mathbf{r}^T \mathbf{s} = \mathbf{s}^T \mathbf{r}$ è uguale a zero: difatti,

$$\mathbf{s}^T \mathbf{r} = (\mathbf{A}\mathbf{z})^T \mathbf{r} = \mathbf{z}^T \mathbf{A}^T \mathbf{r} = 0,$$

visto che $\mathbf{A}^T \mathbf{r} = \mathbf{A}^T (\mathbf{A}\mathbf{x} - \mathbf{b}) = \mathbf{0}$ per ipotesi.

Allora si ha

$$\|\mathbf{A}(\mathbf{x} + \mathbf{z}) - \mathbf{b}\|_2^2 = \mathbf{s}^T \mathbf{s} + \mathbf{r}^T \mathbf{r} \geq \mathbf{r}^T \mathbf{r}.$$

Difatti $\mathbf{s}^T \mathbf{s} = s_1^2 + s_2^2 + \dots + s_m^2 \geq 0$ è positivo. Questo dimostra che $\mathbf{r}(\mathbf{x} + \mathbf{z})$ ha norma maggiore o uguale a quella di $\mathbf{r}(\mathbf{x})$ per ogni scelta del vettore $\mathbf{z}$. $\square$

Quindi per trovare la soluzione $\mathbf{x}$ di questo problema ci basta risolvere il sistema di equazioni

$$\mathbf{A}^T \mathbf{A} \mathbf{x} = \mathbf{A}^T \mathbf{b}, \quad (5.16)$$

che si ottiene moltiplicando per $\mathbf{A}^T$ il sistema originale (che era rettangolare e quindi potenzialmente senza soluzione) $\mathbf{A}\mathbf{x} = \mathbf{b}$. La (5.16) si chiama *metodo delle equazioni normali*, perché le equazioni corrispondenti dicono (in termini geometrici) che il residuo $\mathbf{r} = \mathbf{A}\mathbf{x} - \mathbf{b}$ è ortogonale (prodotto scalare nullo) alle colonne di $\mathbf{A}$.

È semplice vedere che $\mathbf{A}^T \mathbf{A}$ è quadrata e simmetrica; inoltre, si può dimostrare che è positiva definita (quindi invertibile!) tutte le volte che le colonne di $\mathbf{A}$ sono linearmente indipendenti. Possiamo allora risolvere il sistema lineare (5.16) usando la fattorizzazione di Cholesky.

Per risolvere questo problema quindi possiamo usare il seguente algoritmo.

1. Calcoliamo la matrice $\mathbf{A}^T \mathbf{A}$ (costo computazionale: $O(mn^2)$);
2. Calcoliamo il vettore $\mathbf{A}^T \mathbf{b}$ (costo computazionale: $O(mn)$);
3. Risolviamo il sistema lineare $(\mathbf{A}^T \mathbf{A})\mathbf{x} = \mathbf{A}^T \mathbf{b}$, utilizzando la fattorizzazione di Cholesky (costo computazionale: $O(n^3)$).

Poiché $m > n$, il maggiore di questi costi è $O(mn^2)$.

Questo metodo basato sul sistema (5.16) però in alcuni casi risulta instabile: il condizionamento del sistema lineare (5.16) $\kappa(\mathbf{A}^T \mathbf{A})$ può essere molto grande, e in particolare essere più alto di quello del problema di minimo originale. Esiste un altro metodo più costoso ma più stabile.

Fattorizzazione QR Si può dimostrare (noi qui non lo facciamo) il seguente risultato.

Teorema 5.15. Per ogni $\mathbf{A} \in \mathbb{R}^{m \times n}$, esistono una matrice $\mathbf{Q} \in \mathbb{R}^{m \times m}$ ortogonale (cioè che soddisfa $\mathbf{Q}^T \mathbf{Q} = \mathbf{I}$) e una matrice $\mathbf{R} \in \mathbb{R}^{m \times n}$ triangolare superiore tali che $\mathbf{A} = \mathbf{Q}\mathbf{R}$.

La matrice $\mathbf{R}$ è rettangolare: quando scriviamo “triangolare” intendiamo che

$$\mathbf{R} = \begin{bmatrix} \mathbf{R}_1 \\ \mathbf{O} \end{bmatrix},$$

dove $\mathbf{R}_1 \in \mathbb{R}^{n \times n}$ è una normale matrice quadrata triangolare, e $\mathbf{O} \in \mathbb{R}^{(m-n) \times n}$ è un blocco di zeri. Se suddividiamo in blocchi anche

$$\mathbf{Q} = \begin{bmatrix} \mathbf{Q}_1 & \mathbf{Q}_2 \end{bmatrix}, \quad \mathbf{Q}_1 \in \mathbb{R}^{m \times n}, \mathbf{Q}_2 \in \mathbb{R}^{m \times (m-n)},$$

vediamo che gli elementi di Q_2 si “scontrano” con il blocco di zeri quando facciamo il prodotto, quindi $A = QR = Q_1 R_1$.

Con alcune manipolazioni algebriche (che non vediamo nel dettaglio) si può vedere che la soluzione x delle (5.16) è una soluzione anche del sistema lineare triangolare

$$R_1 x = Q_1^T b. \quad (5.17)$$

Questo ci suggerisce un altro algoritmo per la soluzione del problema dei minimi quadrati:

- Calcoliamo la fattorizzazione $A = QR$ (o anche solo i blocchi $A = Q_1 R_1$).
- Risolviamo per sostituzione all'indietro il sistema (5.17).

Questo metodo è più costoso del precedente: facendo un'analisi più accurata, è possibile dimostrare che quando $m \gg n$ richiede $2mn^2$ operazioni più termini di ordine inferiore, contro mn^2 per il metodo delle equazioni normali. Però, il metodo è più stabile in alcuni problemi in cui $A^T A$ è mal condizionata.

Soluzione tramite Matlab In Matlab, risolvere un problema ai minimi quadrati (5.15) è semplice: basta il comando `A \ b`, lo stesso che avreste usato per risolvere il sistema lineare se A fosse quadrata. Matlab usa il metodo più stabile basato sulla fattorizzazione QR.

Capitolo 6

Interpolazione e approssimazione

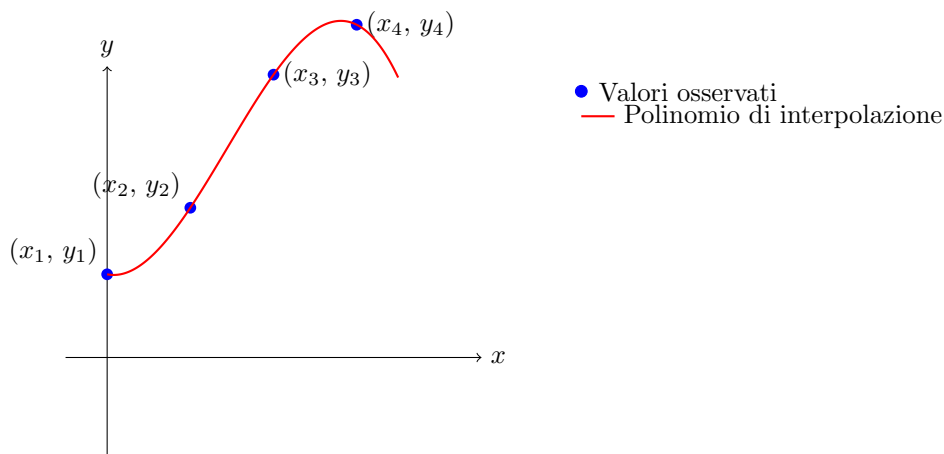
6.1 Approssimazione e interpolazione

In diversi problemi applicativi, abbiamo una serie di misurazioni che corrispondono a punti del piano $(x_1, y_1), \dots, (x_n, y_n) \in \mathbb{R}^2$, e siamo interessati a determinare una funzione che passa (esattamente o approssimativamente) per questi punti.

Ad esempio: supponiamo di avere un impianto industriale in cui stiamo eseguendo una reazione chimica, e di averne misurato la temperatura in 4 tempi distinti, alle 0:00, alle 1:00, alle 2:00, e alle 3:00. In che modo possiamo trovare una stima della temperatura a un'altra ora, ad esempio alle 2:45? O anche in un'ora al di fuori dell'intervallo di misurazione, per esempio alle 4:00? (In questo caso a volte si usa il termine *estrapolazione*.)

Magari abbiamo dei modelli che ci dicono che la temperatura decresce linearmente o esponenzialmente, ma questi modelli dipendono da parametri che dobbiamo calcolare. Oppure qualche volta non abbiamo proprio un modello e stiamo genericamente cercando una funzione 'semplice' che approssima i valori trovati. Studieremo due problemi:

- Si chiama *interpolazione* il problema di trovare una funzione ϕ (all'interno di una certa classe) tale che $\phi(x_i) = y_i$ per ogni $i = 1, 2, \dots, n$. In altre parole, il grafico della funzione nel piano cartesiano deve passare esattamente per i punti (x_i, y_i) .
- Si chiama *approssimazione* (o, dall'inglese, *fit*) il problema di trovare una funzione ϕ tale che $\phi(x_i) \approx y_i$, minimizzando un qualche tipo di errore (vedremo poi esattamente quale).



6.2 Interpolazione polinomiale

Vediamo più nel dettaglio il problema dell'*interpolazione polinomiale*. Fissiamo un grado massimo d ; si usa questa lettera dall'inglese *degree*. Supponiamo di avere $d + 1$ punti dati nel piano $(x_0, y_0), (x_1, y_1), \dots, (x_d, y_d)$ (attenzione: per comodità con gli indici partiamo da 0 questa volta) e di voler trovare un polinomio di grado minore o uguale a d il cui grafico passa esattamente per questi punti; cioè, stiamo cercando coefficienti incogniti $\alpha_0, \alpha_1, \dots, \alpha_d$ tali che il polinomio

$$p(x) = \alpha_0 + \alpha_1 x + \alpha_2 x^2 + \dots + \alpha_d x^d$$

soddisfa le relazioni

$$p(x_i) = y_i, \quad i = 0, 1, \dots, d. \quad (6.1)$$

I punti x_i qui sono detti *nodi* dell'interpolazione, e i punti y_i *valori* da interpolare. Possiamo immaginare che questi punti y_i siano calcolati come $y_i = f(x_i)$ a partire da una funzione f più complicata da calcolare (perché è data da una formula più complicata, o perché è il risultato di misurazioni che non possiamo ripetere); in questo modo costruiamo un polinomio che fa da 'modello' più semplice di questa funzione.

Possiamo osservare che le relazioni (6.1) sono equivalenti a un sistema lineare di $d + 1$ equazioni nelle $d + 1$ incognite $\alpha_0, \dots, \alpha_d$:

$$\underbrace{\begin{bmatrix} 1 & x_0 & x_0^2 & \dots & x_0^d \\ 1 & x_1 & x_1^2 & \dots & x_1^d \\ 1 & x_2 & x_2^2 & \dots & x_2^d \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_d & x_d^2 & \dots & x_d^d \end{bmatrix}}_{=X} \begin{bmatrix} \alpha_0 \\ \alpha_1 \\ \alpha_2 \\ \vdots \\ \alpha_d \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ \vdots \\ y_d \end{bmatrix}.$$

Attenzione: non ci facciamo ingannare dalle lettere usate: le x_i e le y_i non sono incognite: qui sono note, e sono i dati del problema.

La matrice associata a questo sistema, che abbiamo indicato con X , si chiama *matrice di Vandermonde*.

Risolubilità È possibile dimostrare il seguente risultato.

Teorema 6.1. *Per ogni scelta di $x_0, x_1, \dots, x_d$ distinti, la matrice di Vandermonde è invertibile.*

Dimostrazione. (*) Sia

$$\mathbf{a} = \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_d \end{bmatrix}$$

un vettore nel kernel della matrice X , cioè tale che $X\mathbf{a} = \mathbf{0}$. Vogliamo dimostrare che dev'essere $\mathbf{a} = \mathbf{0}$; questo implica che la matrice X è invertibile.

Se scriviamo l'uguaglianza $X\mathbf{a} = \mathbf{0}$ riga per riga, vediamo che essa corrisponde a

$$1 \cdot a_0 + x_i \cdot a_1 + x_i^2 \cdot a_2 + \dots + x_i^d \cdot a_d = 0, \quad i = 0, 1, \dots, d;$$

Queste uguaglianze ci dicono che il polinomio $a(x) = a_0 + a_1x + a_2x^2 + \dots + a_dx^d$ si annulla nei punti $x_0, x_1, \dots, x_d$. Ma dalle proprietà dei polinomi sappiamo che un polinomio di grado al più d che si annulla in $d+1$ punti dev'essere necessariamente il polinomio nullo¹. Questo quindi dimostra che $\mathbf{a} = \mathbf{0}$ e quindi che X è invertibile. $\square$

Dall'invertibilità di questa matrice segue che il problema dell'interpolazione polinomiale è sempre risolubile, cioè vale questo risultato.

Teorema 6.2 (esistenza e unicità del polinomio di interpolazione). *Date $n+1$ coppie $(x_0, y_0), \dots, (x_d, y_d)$, con gli x_i tutti diversi tra loro, esiste uno e un solo polinomio $p(x)$ di grado minore o uguale a d tale che $p(x_i) = y_i$ per ogni $i = 0, 1, \dots, d$.*

Esempio Cerchiamo il polinomio di grado minore o uguale a $d = 2$ che passa per i tre punti $(x_0, y_0) = (1, 2)$, $(x_1, y_1) = (2, 5)$, $(x_2, y_2) = (3, 7)$. Questo equivale a trovare il polinomio $p(x) = \alpha_0 + \alpha_1x + \alpha_2x^2$ che risolve il sistema $X\alpha = \mathbf{y}$

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 4 \\ 1 & 3 & 9 \end{bmatrix} \begin{bmatrix} \alpha_0 \\ \alpha_1 \\ \alpha_2 \end{bmatrix} = \begin{bmatrix} 2 \\ 5 \\ 7 \end{bmatrix}.$$

Risolvendo il sistema (per esempio con `x \ \ y` in Matlab, dopo aver definito X e $\mathbf{y}$) si ottengono i coefficienti del polinomio $p(x) = -2 + 4.5x = 0.5x^2$, che effettivamente soddisfa le tre condizioni cercate. Questo è l'unico polinomio di grado minore o uguale a due che soddisfa le tre condizioni; ne esistono altri di grado maggiore: si può mostrare (qui non lo vediamo nel dettaglio) che sono tutti i polinomi della forma $p(x) = -2 + 4.5x = 0.5x^2 + (x-1)(x-2)(x-3)q(x)$, dove $q(x)$ è un qualunque altro polinomio; difatti vediamo che l'ultimo addendo si annulla per $x = 1, x = 2$, e $x = 3$.

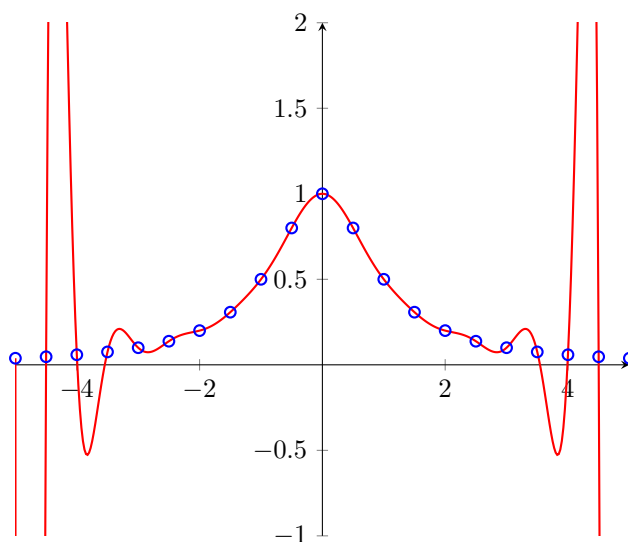
Notare che sul grado abbiamo solo una disuguaglianza: nulla vieta che il coefficiente α_d (o anche quelli precedenti) sia uguale a zero per una particolare scelta

¹Per il teorema di Ruffini, un polinomio che si annulla in $x_0, x_1, \dots, x_d$ è un multiplo del polinomio $(x-x_0)(x-x_1)\dots(x-x_d)$; quindi o è il polinomio nullo, oppure ha grado almeno $d+1$.

dei punti (x_i, y_i) . Per esempio, se modifichiamo i dati in $(1, 3), (2, 5), (3, 7)$ (che sono tre punti allineati), allora il polinomio di grado ≤ 2 che passa esattamente per questi punti è $p(x) = 1 + 2x + 0x^2 = 1 + 2x$, che ha grado 1 perché $\alpha_2 = 0$.

Solitamente l'interpolazione polinomiale si usa con pochi nodi; $d \leq 10$ per esempio. Questo perché per valori grandi di d in molti casi la matrice di Vandermonde, seppure invertibile, risulta mal condizionata. Ad esempio, la matrice di Vandermonde costruita con i 10 nodi $x_0 = 0, x_1 = 1, \dots, x_9 = 9$ ha condizionamento in norma-2 $\kappa(V) \approx 9 \cdot 10^{10}$, un numero altissimo! Ricordiamo il significato del numero di condizionamento: questo significa che a piccolissime variazioni dei dati x_i o y_i possono corrispondere grandi variazioni nei coefficienti del polinomio.

Un problema collegato è che spesso quando d è grande succede che il polinomio passa sì per i punti dati, ma ha forti oscillazioni al di fuori di essi. come nell'esempio qui sotto.



Questo non è il comportamento che ci aspettiamo da una funzione “semplice” che passi per i punti marcati in blu!

Polinomi di Lagrange Possiamo dare una formula esplicita per la soluzione del problema dell'interpolazione polinomiale. Fissati i nodi distinti $x_0, x_1, \dots, x_d$, definiamo i *polinomi di Lagrange*

$$L_k(x) = \frac{\prod_{j \neq k} (x - x_j)}{\prod_{j \neq k} (x_k - x_j)}, \quad k = 1, 2, \dots, d.$$

Ad esempio, se i nodi sono $x_0 = 1, x_1 = 2, x_2 = 4$, abbiamo

$$L_0(x) = \frac{(x-2)(x-4)}{(1-2)(1-4)}, \quad L_1(x) = \frac{(x-1)(x-4)}{(2-1)(2-4)}, \quad L_2(x) = \frac{(x-1)(x-2)}{(4-1)(4-2)}.$$

Notare che il denominatore non si annulla mai se i nodi sono distinti, e che sono tutti polinomi di grado d , visto che il numeratore è il prodotto di d fattori di grado 1. Inoltre, vale il seguente risultato.

Lemma 6.3. *Si ha*

$$L_k(x_i) = \begin{cases} 1 & i = k, \\ 0 & \text{altrimenti.} \end{cases}$$

Dimostrazione. Sostituendo $x = x_k$, numeratore e denominatore diventano identici, quindi il polinomio vale 1. Sostituendo $x = x_i$ con $i \neq k$, uno dei fattori nella produttoria al numeratore diventa $(x_i - x_i)$, quindi $L_k(x_i) = 0$. $\square$

Quindi il k -esimo polinomio di Lagrange fornisce la soluzione al problema di interpolazione con $\mathbf{y} = \mathbf{e}_k$ (k -esimo vettore della base canonica). Facendo una combinazione lineare di queste soluzioni, possiamo ottenere la soluzione a un problema di interpolazione polinomiale con un vettore $\mathbf{y}$ generico.

Teorema 6.4. *Siano $(x_0, y_0), \dots, (x_d, y_d)$ dati. La soluzione del problema di interpolazione polinomiale (cioè l'unico polinomio p di grado $\leq d$ tale che $p(x_i) = y_i$ per ogni $i = 0, 1, \dots, d$) è data da*

$$p(x) = \sum_{k=0}^d y_k L_k(x).$$

Dimostrazione. Sappiamo già che la soluzione del problema è unica. Basta verificare che $p(x_i) = y_i$ per ogni i (cosa che segue dal Lemma 6.3) e che $p(x)$ ha grado minore o uguale a d (perché è una combinazione lineare dei polinomi di Lagrange, che hanno tutti grado d). $\square$

Resto dell'interpolazione In alcuni casi, sappiamo che i punti da interpolare (x_i, y_i) appartengono esattamente al grafico di una certa funzione f , cioè, $y_i = f(x_i)$ per ogni $i = 0, \dots, d$. Si parla in questo caso di “polinomio di interpolazione della funzione f ”. Questa f tipicamente non è un polinomio, ma potrebbe essere una funzione definita da una formula più complicata. Ha senso quindi cercare di “rimpiazzarla” con un polinomio più semplice, se possiamo assicurarci che l'errore commesso nell'interpolazione è piccolo. Il risultato seguente ci dà un modo di stimare questo errore, in funzione delle derivate della funzione f .

Teorema 6.5 (Resto dell'interpolazione). *Sia $f \in \mathcal{C}^{d+1}([a, b])$, $x_0, x_1, \dots, x_d$ nodi distinti in $[a, b]$, e $p(x)$ il polinomio di interpolazione (di grado al più d) di f sui nodi dati. Allora per ogni $x \in [a, b]$ esiste un punto $\xi \in (a, b)$ tale che*

$$f(x) - p(x) = \frac{f^{(d+1)}(\xi)}{(d+1)!} (x - x_0)(x - x_1) \dots (x - x_d). \quad (6.2)$$

Non lo dimostriamo ma facciamo qualche commento.

- Questo risultato assomiglia a uno sviluppo di Taylor con resto di Lagrange: abbiamo che $f(x)$ è uguale a un polinomio $p(x)$ (che arriva ad avere potenze fino al grado d) più un resto che dipende dalla derivata $d + 1$ -esima. Qui abbiamo $d + 1$ punti, anziché un “centro” solo dello sviluppo di Taylor, e nella formula per il resto $\frac{f^{(d+1)}(\xi)}{(d+1)!} (x - x_0)^{d+1}$, il termine $(x - x_0)^{d+1}$ viene rimpiazzato da $\prod_{j=0}^d (x - x_j)$. (Questo è un ottimo modo per ricordarsi la formula.)

- Non ci stupisce che ci siano dei fattori $(x - x_i)$ nel membro di destra: difatti, se sostituiamo $x = x_i$, per un qualche $i = 0, 1, \dots, d$, il termine di sinistra si annulla, quindi deve annullarsi anche quello di destra.
- Un caso speciale in cui possiamo verificare il risultato direttamente è quello in cui $f(x)$ è essa stessa un polinomio di grado minore o uguale a d . In questo caso il polinomio di interpolazione coincide con $f(x)$ stessa, quindi il termine di sinistra è nullo per ogni x ; e anche il termine di destra si annulla perché per un polinomio di grado al più d abbiamo $f^{(d+1)} \equiv 0$.

Errore dell'interpolazione Dal teorema del resto dell'interpolazione segue una stima per l'errore massimo (differenza tra $f(x)$ e $p(x)$), cioè

$$|f(x) - p(x)| \leq \frac{C_{d+1}}{(d+1)!} (b-a)^{d+1},$$

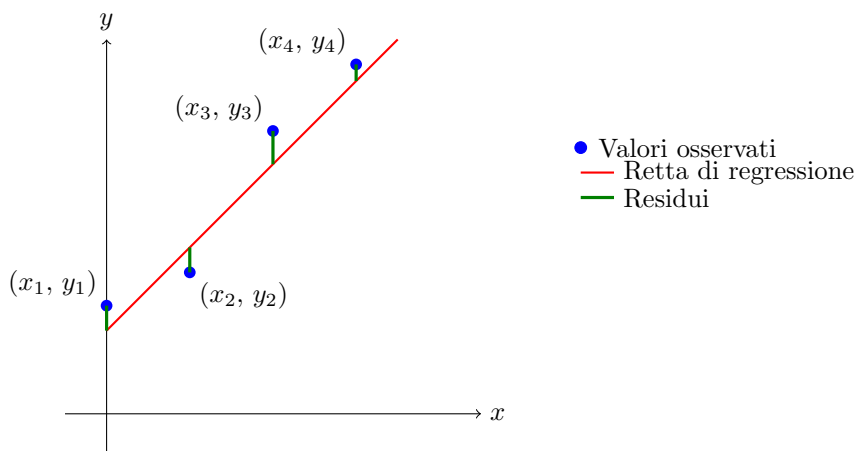
dove $C_{d+1} = \max_{x \in [a,b]} |f^{(d+1)}(x)|$.

6.3 Altri problemi di interpolazione e approssimazione

Approssimazione / fit Come abbiamo visto, quando il grado d è alto, il polinomio di interpolazione può avere forti oscillazioni, e la matrice di Vandermonde V può essere mal condizionata. In molti casi, anche se abbiamo tanti punti $(x_0, y_0), \dots, (x_m, y_m)$, conviene fissare un grado $d < m$ e prendere il polinomio di grado minore o uguale a d che più si avvicina ai punti dati. Questo corrisponde a risolvere un sistema lineare sovradeterminato nel senso dei minimi quadrati: difatti, si ha

$$\begin{aligned} & \min_{\alpha \in \mathbb{R}^{d+1}} |p(x_0) - y_0|^2 + |p(x_1) - y_1|^2 + \dots + |p(x_m) - y_m|^2 \\ &= \min_{\alpha \in \mathbb{R}^{d+1}} \left\| \begin{bmatrix} 1 & x_0 & x_0^2 & \dots & x_0^d \\ 1 & x_1 & x_1^2 & \dots & x_1^d \\ 1 & x_2 & x_2^2 & \dots & x_2^d \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_m & x_m^2 & \dots & x_m^d \end{bmatrix} \begin{bmatrix} \alpha_0 \\ \alpha_1 \\ \alpha_2 \\ \vdots \\ \alpha_d \end{bmatrix} - \begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix} \right\|_2^2. \end{aligned}$$

Vediamo in figura un esempio con quattro punti e un polinomio di grado $d = 1$. La retta non passa esattamente per i punti dati, ma vicino ad essi, in modo da minimizzare (al variare di tutte le possibili rette) la somma dei segmenti verticali disegnati in verde, i residui.



Questo problema si chiama *approssimazione di funzioni*, o, dall'inglese, *fitting*. Il problema risultante è un problema ai minimi quadrati, e si può risolvere con gli algoritmi che abbiamo già visto: equazioni normali, fattorizzazione QR, funzione “backslash” di Matlab.

Retta dei minimi quadrati Vediamo come esempio il caso particolare $d = 1$, in cui il polinomio è una retta, detta a volte *retta dei minimi quadrati* o *retta di regressione*. A differenza del problema dell'interpolazione, i punti sono più di due, quindi non esiste una retta che passa esattamente per tutti questi punti. Abbiamo

$$X = \begin{bmatrix} 1 & x_0 \\ 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_m \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}, \quad \boldsymbol{\alpha} = \begin{bmatrix} \alpha_1 \\ \alpha_2 \end{bmatrix} = (X^T X)^{-1} X^T \mathbf{y},$$

dove l'ultima formula corrisponde ad usare le equazioni normali per risolvere questo problema. Possiamo ottenere una formula più esplicita espandendo i conti e usando la formula per l'inversa di una matrice 2×2 . Potreste aver visto queste formule se avete già incontrato la retta di regressione lineare in altri corsi, per esempio parlando di statistica; per noi rifare questo conto non è particolarmente interessante.

Approfondimento: interpolazione con funzioni più generali Quello dell'interpolazione polinomiale è un problema apparentemente non-lineare, ma che sorprendentemente si riduce a un sistema di equazioni lineari: anche se nella formulazione del problema compaiono polinomi di grado più alto di 1, la dipendenza dai coefficienti incogniti $\alpha_0, \dots, \alpha_d$ è lineare.

Possiamo risolvere nello stesso modo anche un problema più generale: fissiamo n funzioni $\phi_1(x), \dots, \phi_n(x)$, e cerchiamo coefficienti incogniti $\alpha_1, \dots, \alpha_n$ in modo da ottenere una funzione

$$\phi(x) = \alpha_1 \phi_1(x) + \alpha_2 \phi_2(x) + \dots + \alpha_n \phi_n(x) \quad (6.3)$$

che passi per certi punti.

Più nel dettaglio, supponiamo di avere m coppie di valori $(x_1, y_1), \dots, (x_m, y_m)$. Di nuovo, assumiamo che gli x_i siano *distinti*.

Se imponiamo le condizioni $\phi(x_i) = y_i$ per $i = 1, 2, \dots, m$, abbiamo il sistema lineare di m equazioni in n incognite

$$\underbrace{\begin{bmatrix} \phi_1(x_1) & \phi_2(x_1) & \dots & \phi_n(x_1) \\ \phi_1(x_2) & \phi_2(x_2) & \dots & \phi_n(x_2) \\ \vdots & \vdots & \ddots & \vdots \\ \phi_1(x_m) & \phi_2(x_m) & \dots & \phi_n(x_m) \end{bmatrix}}_{=X} \underbrace{\begin{bmatrix} \alpha_1 \\ \alpha_2 \\ \vdots \\ \alpha_n \end{bmatrix}}_{=\alpha} = \underbrace{\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}}_{=\mathbf{y}}.$$

Se $m = n$, cioè se ho tanti punti quante funzioni incognite, questo è un sistema lineare quadrato e posso risolverlo esattamente (ammesso che la matrice X sia invertibile) (*interpolazione*). Se $m > n$, allora ho un sistema con più equazioni che incognite, e posso risolverlo nel senso dei minimi quadrati (*approssimazione/fit*), calcolando $\min \|X\alpha - \mathbf{y}\|$. Vediamo un esempio.

Polinomi trigonometrici Se desideriamo interpolare (o approssimare) una funzione che sappiamo essere periodica di periodo 2π , viene naturale scegliere una base composta di funzioni che sono anch'esse periodiche, per esempio

$$\begin{aligned} \phi_1(x) &= 1, & \phi_3(x) &= \cos x, \\ \phi_2(x) &= \sin x, & \phi_5(x) &= \cos 2x, \\ \phi_4(x) &= \sin 2x, & & \\ \vdots & & \vdots & \\ \phi_{2k}(x) &= \sin kx, & \phi_{2k+1}(x) &= \cos kx. \end{aligned}$$

Queste sono, in un certo senso, le funzioni periodiche “più semplici” di periodo 2π . Non vediamo i dettagli qui, ma è un approccio molto usato, e collegato alla cosiddetta *trasformata di Fourier*, che forse avrete occasione di incontrare in futuro in altri corsi.